Lecture 04 – Nonparametric Regression

Stat 5376: Statistical Machine Learning

Ruiqi Liu

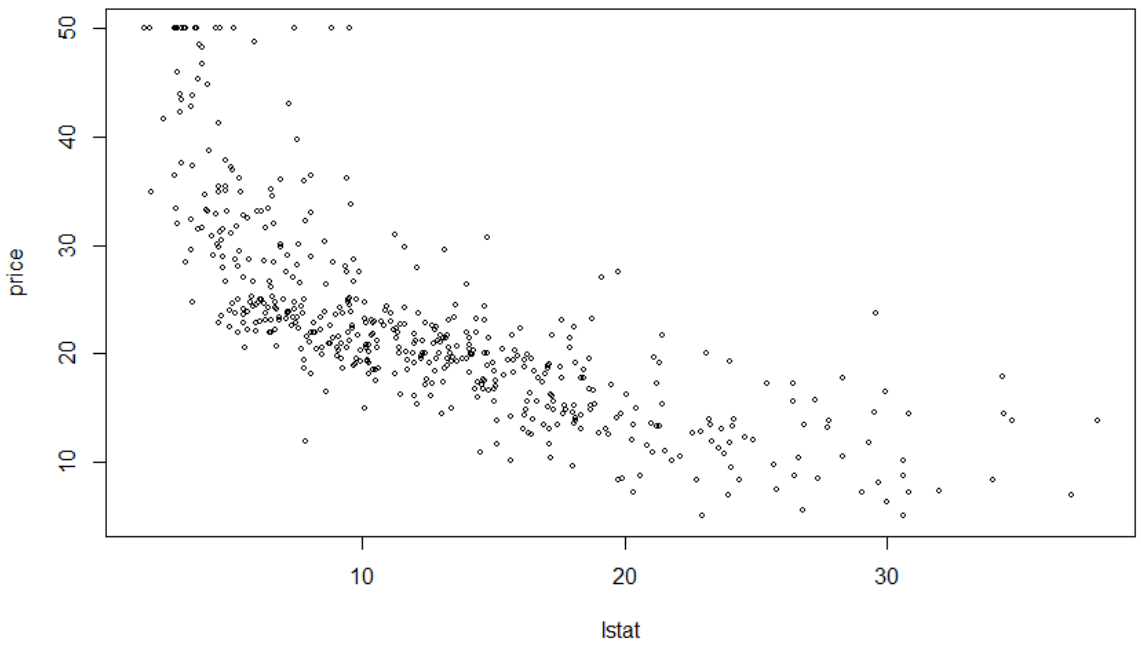
Department of Mathematics and Statistics
Texas Tech University

http://www.math.ttu.edu/~ruiqliu/

Disclaimer: These notes have not been subjected to the usual scrutiny reserved for formal publications. They may be distributed outside this class only with the permission of the Instructor.

Moving Beyond Linearity

- Boston Housing dataset.
- lstat: lower status of population.
- price: median price of the houses.
- Linear model seems not adequate.



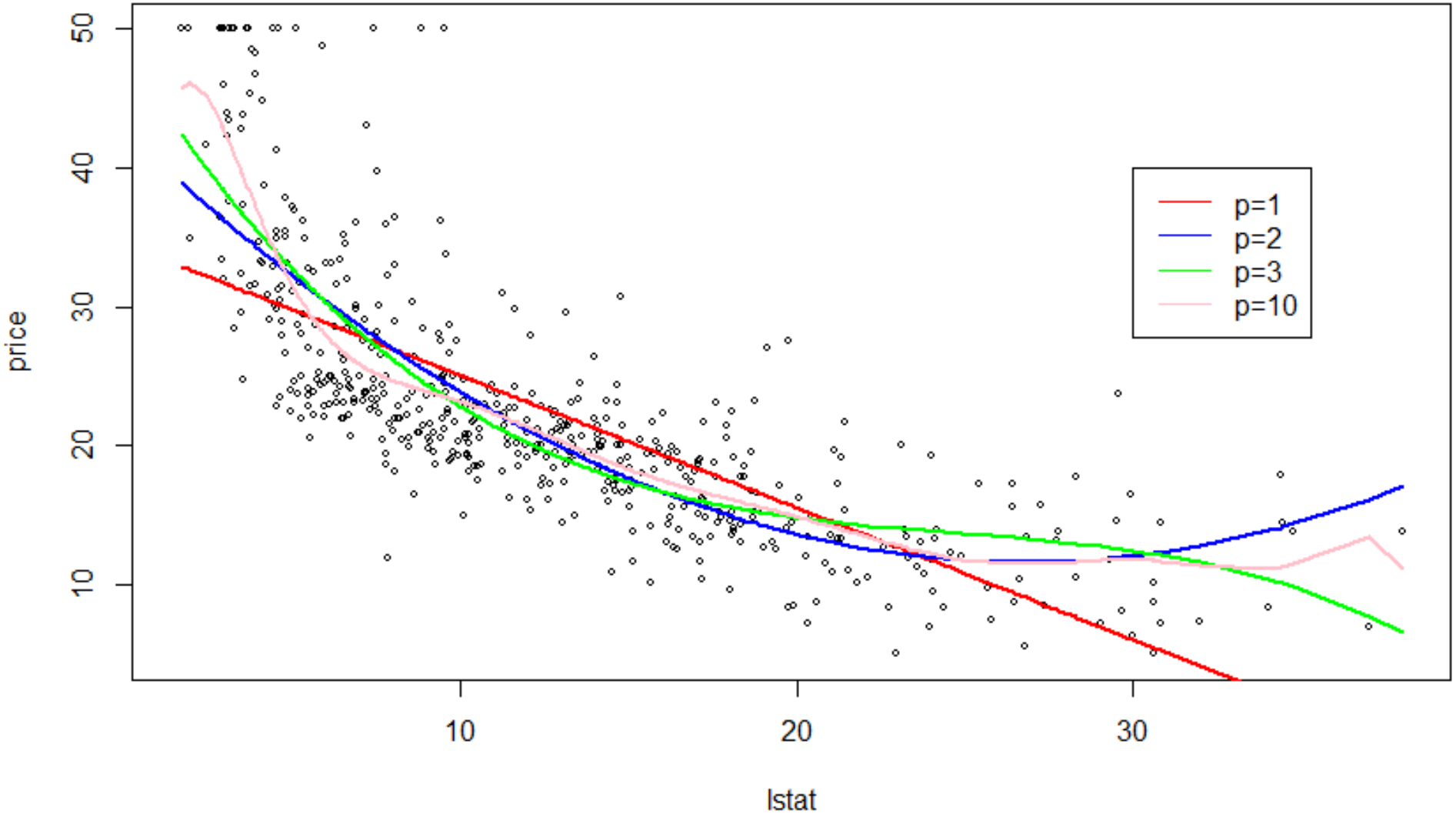
Polynomial Regression

- A standard method is to use polynomial regression.
- $y = \beta_0 + \beta_1 x + \ldots + \beta_p x^p$ with $y =$price, $x =$lstat.

Polynomial Regression

- A standard method is to use polynomial regression.
- $y = \sum_{i=1}^{p} \beta_i x^p$ with $y =$price, $x =$lstat.
- Try $p = 1$(linear model), $p = 2, 3, 10$.
-
```
library(MASS)
data(Boston)
data=Boston
lstat=data$lstat
price=data$medv
index=order(lstat)
lstat=lstat[index]
price=price[index]

lmod=lm(price~lstat)
plot(lstat,price,cex=0.5)
lines(lstat,lmod$fitted.values, col="red",lwd=2)

p2mod=lm(price~lstat+I(lstat^2))
lines(lstat,p2mod$fitted.values,col="blue",lwd=2)

p3mod=lm(price~lstat+I(lstat^2)+I(lstat^3))
lines(lstat,p3mod$fitted.values,col="green",lwd=2)

p10mod=lm(price~lstat+I(lstat^2)+I(lstat^3)+
I(lstat^4)+I(lstat^5)+I(lstat^6)+I(lstat^7)+I(lstat^8)+I(lstat^9)+I(lstat^10))
lines(lstat,p10mod$fitted.values,col="pink",lwd=2)
```

Polynomial Regression



Polynomial Regression

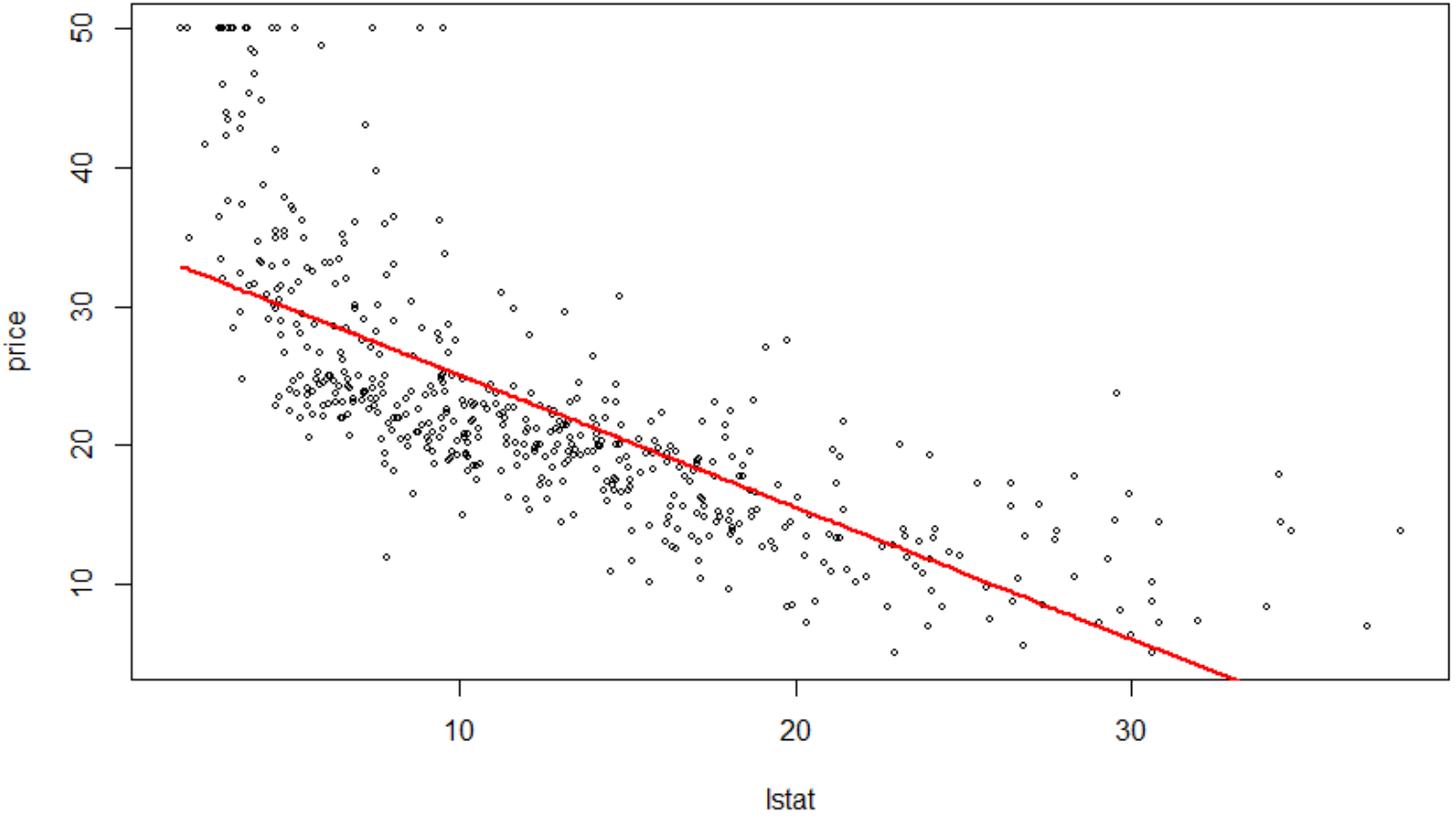
- In the regression problem with one predictor x:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_p x^p + \varepsilon;$$

- It can allow us to model expected features of the relationship;
- Choice of p is important.
- Small p: under-fitting.
- Large p: over-fitting.

Choice of Degree p: Under-fitting

- In Boston Housing example, when $p = 1$, linear model under-fits the model.
- Under-fitting: It happens when the specified model is not large enough.



Choice of Degree p: Over-fitting

- To over-fitting issue, let's see a simple example;
- Let's generate $n = 6$ data points from the model

$$y_i = 1 + 2x_i + \varepsilon_i, \quad \varepsilon_i \sim N(0, 1^2), \quad i = 1, \ldots, 6.$$

- Lets first fit a regression model (1)

$$y = \beta_0 + \beta_1 x$$

- Lets first fit a regression model (2)

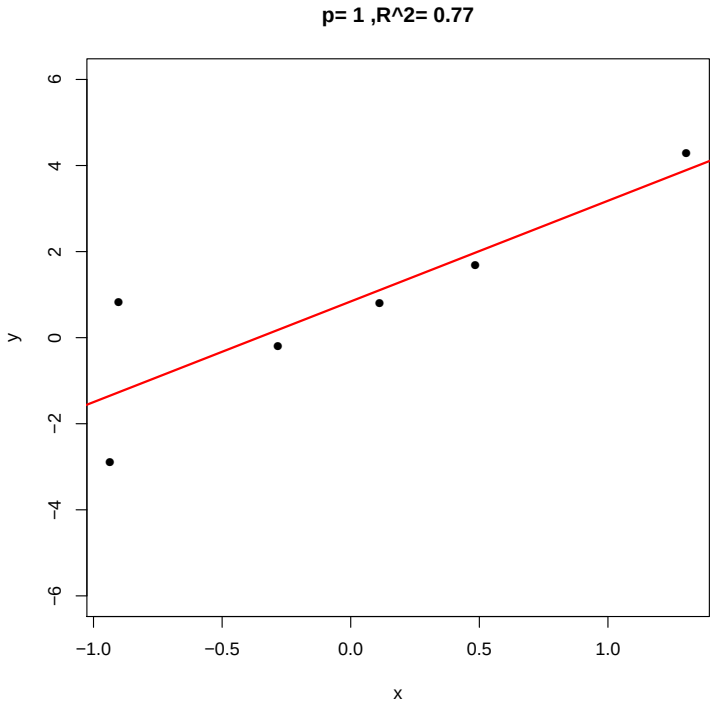
$$y = \beta_0 + \beta_1 x + \beta_2 x^2$$

- $\vdots$
- Lets first fit a regression model (5)

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_5 x^5$$

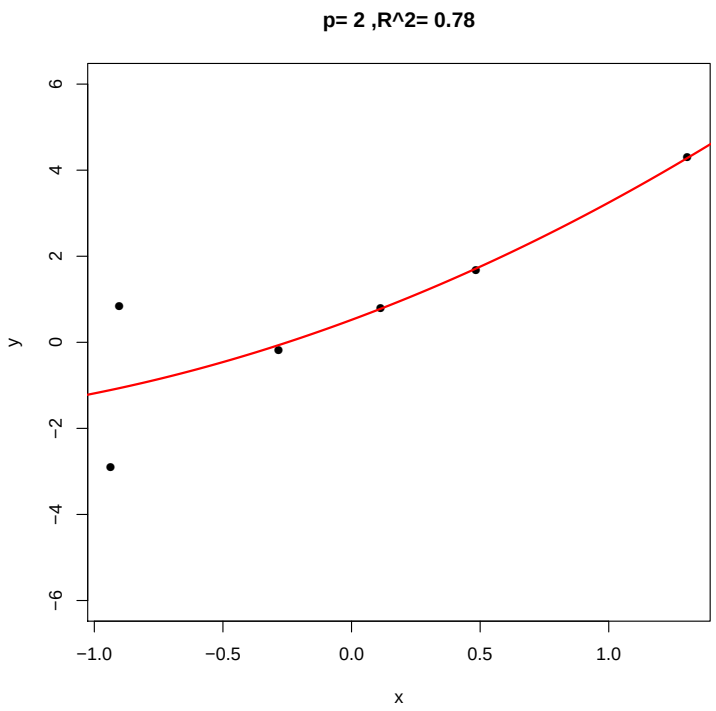
- Add a new predictor x^p one at a time, $p = 1, \ldots, 5$.

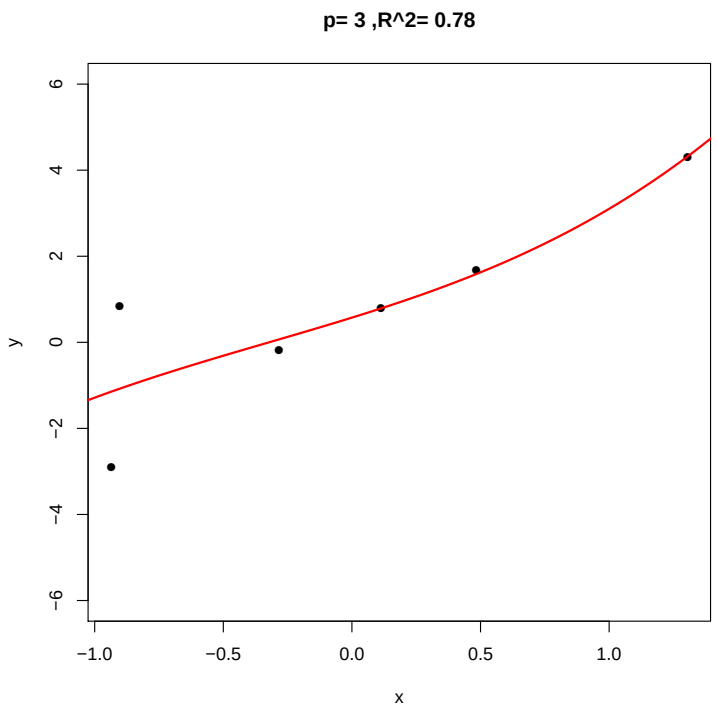
Choice of Degree p: Over-fitting



- $p = 1$ fits the model reasonably well;

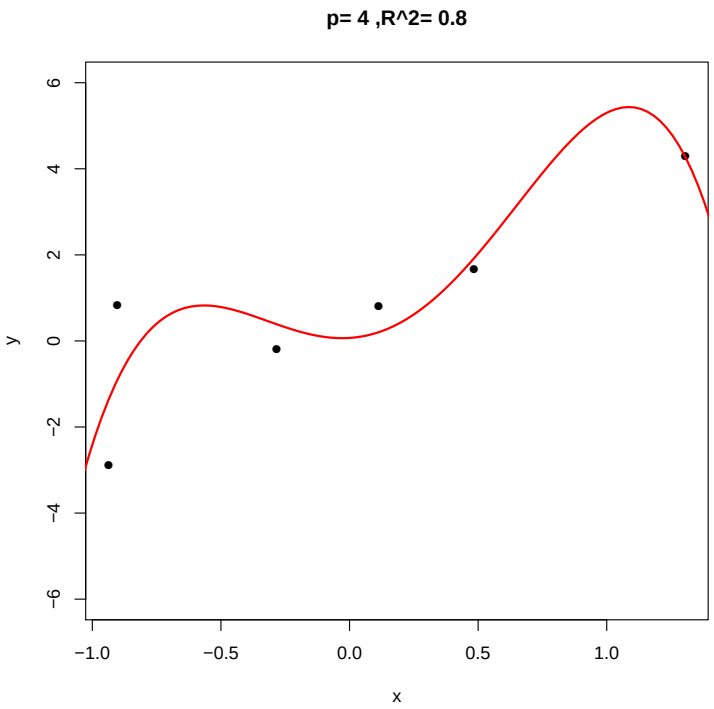
Problem: Over-fitting

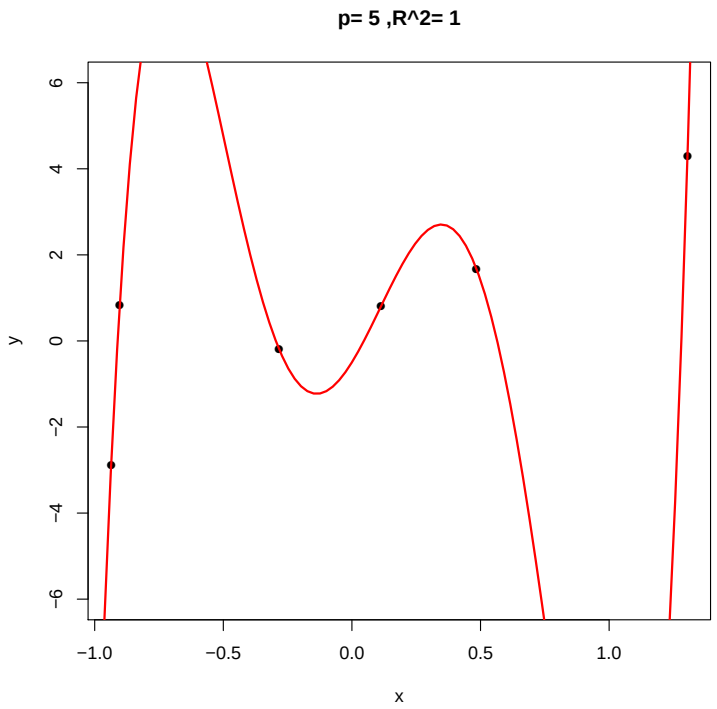




- $p = 2, 3$ R^2 increase a little bit;
- The model fit looks ok!

Problem: Over-fitting





- $p = 6$: $R^2 = 1$;
- But the model fit is horrible!
- This is the case of overfitting!

Problem: Over-fitting

- Over-fitting: usually happens when your sample size n is small but the number of parameter p is large!
- For polynomial regression, the over-fitting happens because
  - when your model is complicated enough (i.e. p large enough), you can fit the error term ε very well!
  - This is not what you want! You want to fit the signal $\mathbf{X}\boldsymbol{\beta}$ but not ε;
  - Over-fitting means your model can fit the “training data" really well, but when you try to use your model to do prediction on another data set, the performance could be horrible!

Polynomial Regression

- In the regression problem with one predictor x:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_p x^p + \varepsilon;$$

- There are two common ways to choose p in polynomial regression:
  - Foward: keep adding terms until the added term is not statistically significant;
  - Backward: we can start with a large d and eliminate non-statistically significant terms starting with the highest order term.
- Personally, I would favor the forward strategy;

Savings Data

- Consider “savings" dataset in R package “faraway".
- Let’s see if we can use polynomial regression on the “ddpi" variable in the savings
- First, fit a linear model:

```
summary(lm(sr ~ ddpi,savings))
Coefficients:
Estimate Std. Error t value Pr(>|t|)
(Intercept)   7.8830     1.0110   7.797 4.46e-10 ***
ddpi          0.4758     0.2146   2.217   0.0314 *
```

- The p-value of “ddpi" is significant so move on to a quadratic term:

```
summary(lm(sr ~ ddpi+I(ddpi^2),savings))
Coefficients:
Estimate Std. Error t value Pr(>|t|)
(Intercept)  5.13038    1.43472   3.576 0.000821 ***
ddpi         1.75752    0.53772   3.268 0.002026 **
I(ddpi^2)   -0.09299    0.03612  -2.574 0.013262 *
```

- Again the p-value of $ddpi^2$ is significant so move on to a cubic term:

Savings Data

- Fit the cubic model

```
summary(lm(sr ~ ddpi+I(ddpi^2)+I(ddpi^3),savings))
Coefficients:
Estimate Std. Error t value Pr(>|t|)
(Intercept)  5.145e+00  2.199e+00   2.340   0.0237 *
ddpi         1.746e+00  1.380e+00   1.265   0.2123
I(ddpi^2)   -9.097e-02  2.256e-01  -0.403   0.6886
I(ddpi^3)   -8.497e-05  9.374e-03  -0.009   0.9928
```

- The p-value of $ddpi^3$ is not significant so stick with the quadratic. STOP.
- Usually it is a bad idea to eliminate lower order terms from the model before the higher order terms even if they are not statistically significant;
- In this case, we do not eliminate ddpi and $ddpi^2$ even though they are insignificant; only eliminate $ddpi^3$;
- Think about it: Setting the intercept to zero means the regression passes through the origin;
- Setting the linear term to zero would also put some constraint on your model!

Orthogonal Polynomials

- You have to refit the model each time a term is removed which is inconvenient;
- Can we do this just once? Yes, using Orthogonal polynomials!
- Define polynomials

$$z_1 = a_1 + b_1 x \qquad \text{first order polynomial}$$

$$z_2 = a_2 + b_2 x + c_2 x^2 \qquad \text{second order polynomial}$$

$$z_3 = a_3 + b_3 x + c_3 x^2 + d_3 x^3 \qquad \text{third order polynomial}$$

- If all coefficients a, b, c are chosen so that $z_i^T z_j = 0$ when $i \neq j$, then z_1, z_2, z_3 are called Orthogonal polynomials;
- The *poly()* function constructs orthogonal polynomials;

Savings Data

- Fit the cubic model using orthogonal polynomials

```
lmod <- lm(sr ~ poly(ddpi,4),savings)
sumary(lmod)
                 Estimate Std. Error t value Pr(>|t|)
(Intercept)       9.67100    0.58460  16.543   <2e-16 ***
poly(ddpi, 4)1    9.55899    4.13376   2.312   0.0254 *
poly(ddpi, 4)2  -10.49988    4.13376  -2.540   0.0146 *
poly(ddpi, 4)3   -0.03737    4.13376  -0.009   0.9928
poly(ddpi, 4)4    3.61197    4.13376   0.874   0.3869
```

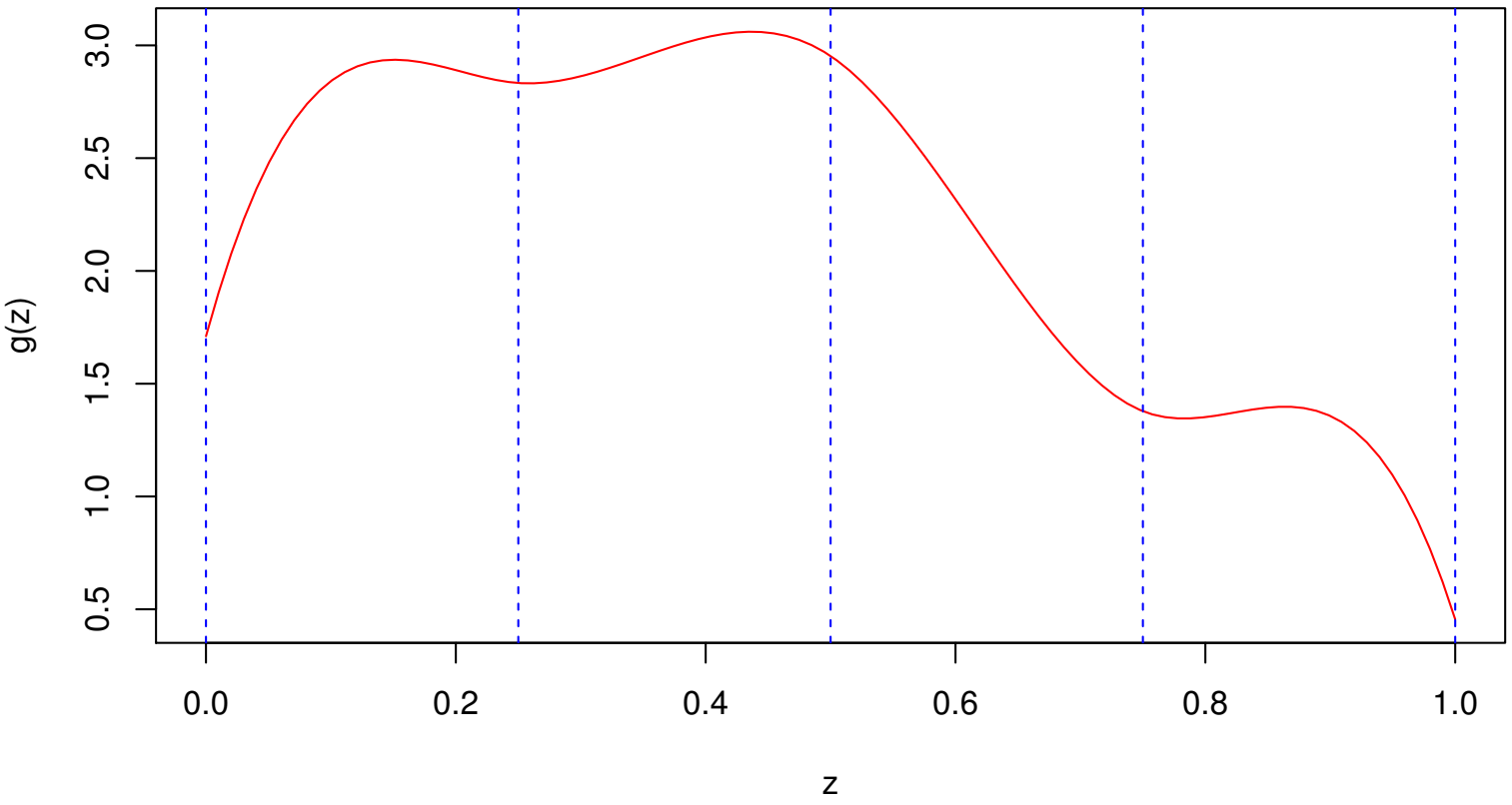
- Fit the quadratic model using orthogonal polynomials

```
lmod <- lm(sr ~ poly(ddpi,2),savings)
sumary(lmod)
                Estimate Std. Error t value Pr(>|t|)
(Intercept)       9.6710     0.5769  16.765   <2e-16 ***
poly(ddpi, 2)1    9.5590     4.0790   2.343   0.0234 *
poly(ddpi, 2)2  -10.4999     4.0790  -2.574   0.0133 *
```

- Same Conclusion: Quadratic model is best, with just this summary.

Spline

- A spline $g(x)$ with degree p during interval $[a, b]$ with knots $a < \kappa_1 < \kappa_2 < \ldots < \kappa_k < b$ is a piecewise polynomial during each intervals which connects nicely at knots.
- Spline during $[0, 1]$ with $p = 3$ and three knots $0.25, 0.5, 0.75$.



Truncated Power Basis

- The truncated power basis of degree p is defined as

$$1, x, \cdots, x^p, (x-\kappa_1)_+^p, \cdots, (x-\kappa_K)_+^p,$$

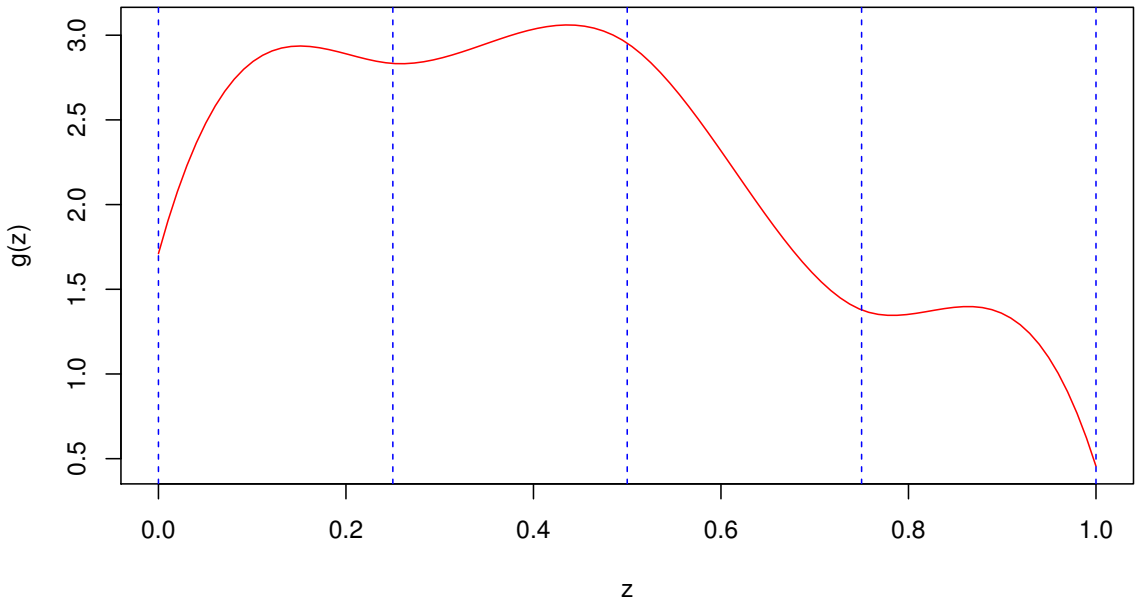
  here $(x-\kappa_i)_+ = \max(0, x-\kappa_i)$.
- In total, there are $K+p+1$ basis functions (DF).
- Any spline $g(x)$ during $[a,b]$ with degree p and knots $a < \kappa_1 < \kappa_2 < \ldots < \kappa_k < b$ can be write as

$$g(x) = \beta_0 + \beta_1 x + \cdots + \beta_p x^p + \sum_{k=1}^{K} b_k (x-\kappa_k)_+^p$$

- The function $(x-\kappa)_+^p$ has $p-1$ continuous derivatives
- Higher values of p lead to smoother spline functions $g(x)$.
- The pth derivative of $g(x)$ is NOT continuous (a step function): b_k is the jump in the pth derivative at the knot κ_k;

Number of Basis Functions

- Example with $K = 3, p = 3$:



- During each interval, it is a polynomial with order p: need $(K+1)(p+1)$ parameters.
- Connect nicely at knots, $g_-^{(s)}(\kappa_k) = g_+^{(s)}(\kappa_k)$ for all $k = 1, \ldots, K$ and $s = 0, \ldots, p-1$: there is Kp constraints.
- In total, we need $(K+1)(p+1) - Kp = K + p + 1$ parameters.
- Specially, for cubic spline with $p = 3$, we need $K + 4$ parameters.

B-Spline Basis

- One issue of truncated power basis is the difference of the magnitude for the basis is large.
- Need a more “balanced" basis.
- Given knots $\kappa_1 < \kappa_2 < \ldots < \kappa_K$ and interval $[a, b]$. We denote $\kappa_i = a$ if $i \leq 0$ and $\kappa_i = b$ if $i \geq K + 1$.
- The B-spline basis of degree 0 is defined as

$$B_{i,1}(x) = \begin{cases} 1 & \text{if } \kappa_i \leq x < \kappa_{i+1} \\ 0 & \text{otherwise} \end{cases} \text{ for } i = 0, 1, \ldots, K.$$

- The B-spline basis of degree p is defined

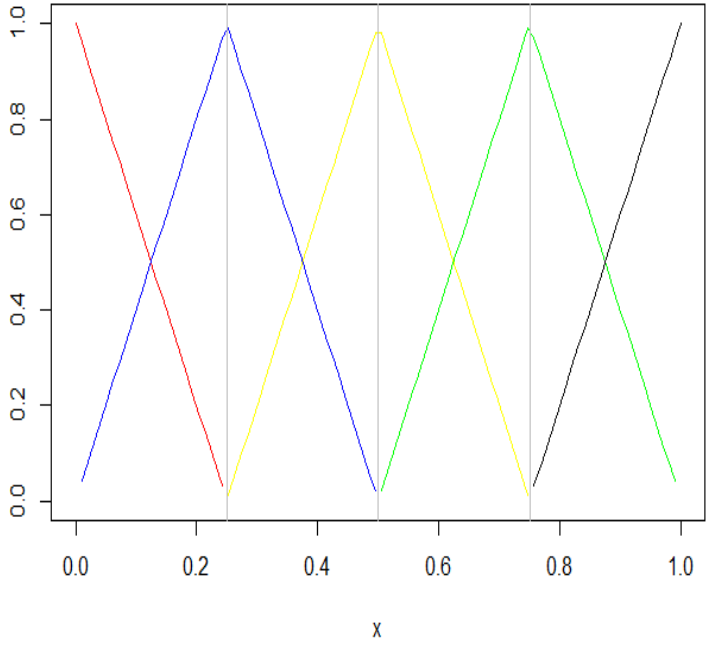
$$B_{i,p}(x) = \frac{x - \kappa_i}{\kappa_{i+p} - \kappa_i} B_{i,p-1}(x) + \frac{\kappa_{i+p+1} - x}{\kappa_{i+p+1} - \kappa_{i+1}} B_{i+1,p-1}(x),$$

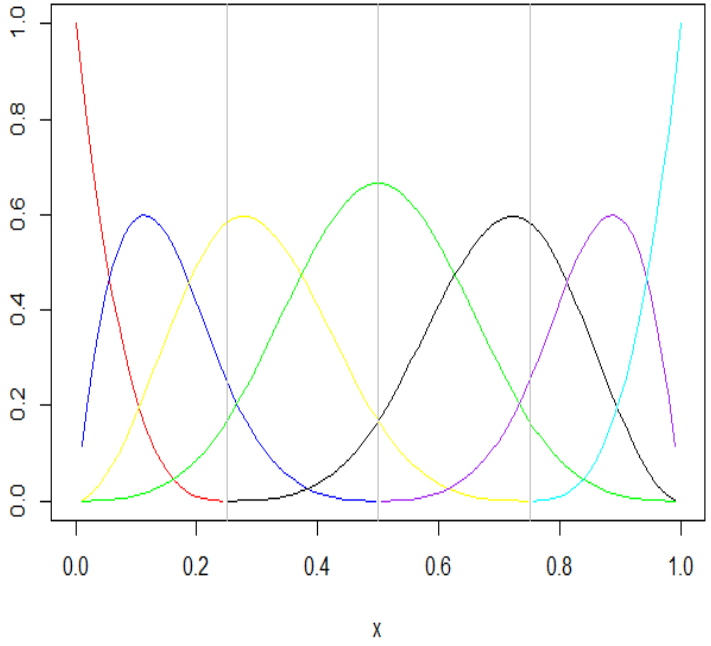
  for $i = -p, -p + 1, \ldots, K$.
- Any spline $g(x)$ during $[a, b]$ with degree p and knots $a < \kappa_1 < \kappa_2 < \ldots < \kappa_k < b$ can be write as

$$g(x) = \sum_{i=-p}^{K} \beta_i B_{i,p}(x).$$

B-Spline Basis

- Knots $(0.25, 0.5, 0.75)$ during $[0, 1]$.
- Linear Spline $(p = 1)$ and Cubic Spline $(p = 3)$:





- Number of basis functions (DF) $K + p + 1 = 5$ and 7.

Practical Implementation

- The nonparametric regression model

$$y_i = f(x_i) + \varepsilon_i, \quad \mathbb{E}(\varepsilon_i) = 0, \quad i = 1, \ldots, n$$

- Use the spline function based on truncated power basis

$$y_i \approx \sum_{i=-p}^{K} \beta_i B_{i,p}(x) + \varepsilon_i$$

- It can be written as a linear model

$$\boldsymbol{Y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon},$$

with $\boldsymbol{\beta} = (\beta_{-p}, \ldots, \beta_K)^T$ and

$$\mathbf{X} = \begin{pmatrix} B_{-p}(x_1) & & \cdots & B_K(x_1) \\ B_{-p}(x_2) & & \cdots & B_K(x_2) \\ \vdots & \vdots \quad \vdots & \vdots & \vdots \\ B_{-p}(x_n) & & \cdots & B_K(x_n) \end{pmatrix}$$

Wage Data

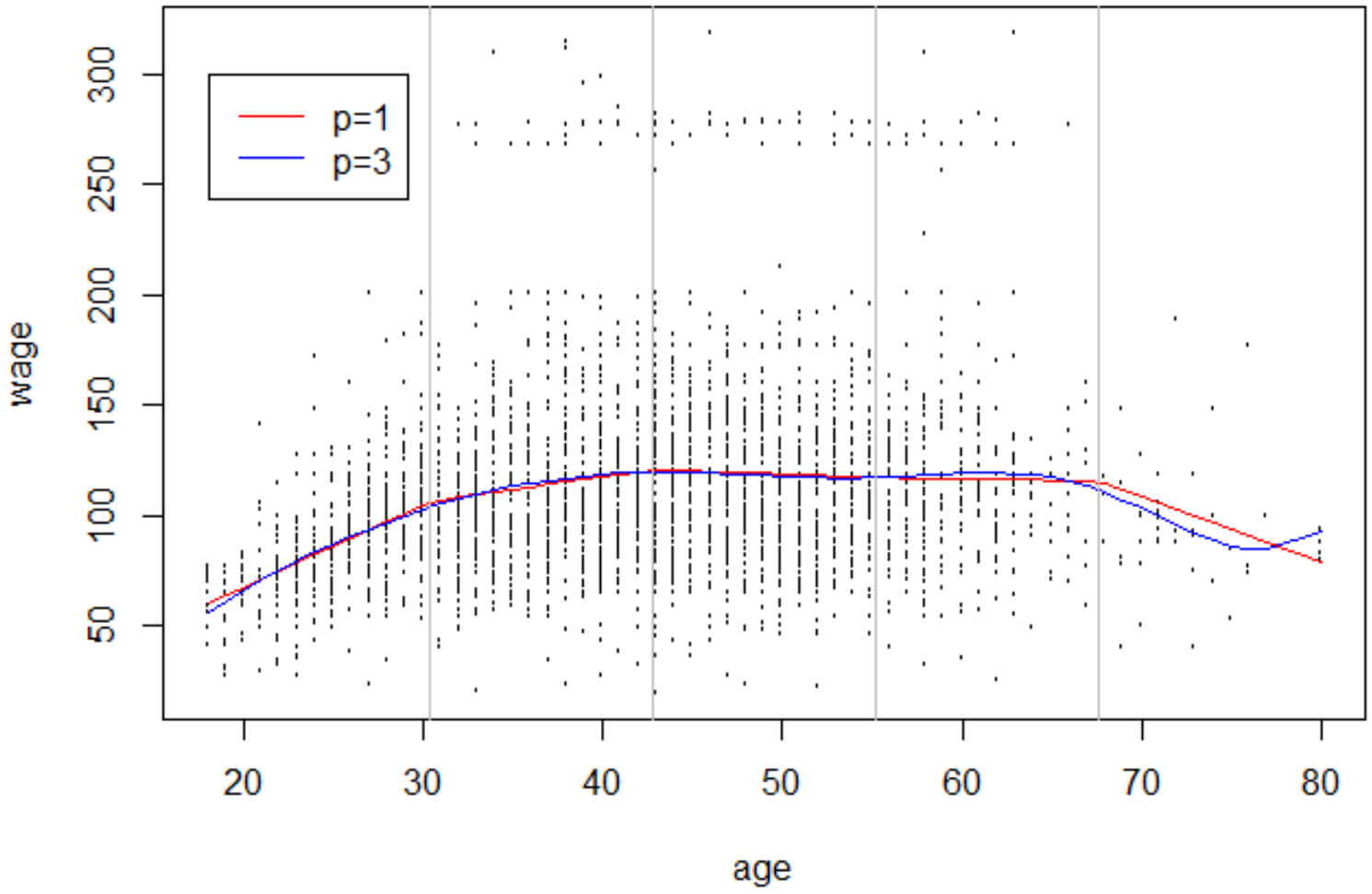
- Consider “Wage" dataset in R package “ISLR".
- $y =$wage and $x =$age.
- Four Knots are chosen to be equally spaced during the $[\min(x), \max(x)]$.
-
```
library(ISLR)
library(splines)
data("Wage")
age=Wage$age
wage=Wage$wage
index=order(age)
age=age[index]
wage=wage[index]

knots=seq(min(age),max(age), length.out=6)
knots=knots[-length(knots)]
knots=knots[-1]

range(age)

[1] 18 80

knots

[1] 30.4 42.8 55.2 67.6
```

Wage Data

- ```
  plot(age, wage, cex=0.3)
  sp1.mod=lm(wage~bs(age,knots=knots,degree=1))
  lines(age,sp1.mod$fitted.values,col="red")

  sp3.mod=lm(wage~bs(age,knots=knots,degree=3))
  lines(age,sp3.mod$fitted.values,col="blue")

  abline(v=knots,col="gray")

  legend(18,300, c("p=1","p=3"),col=c("red","blue"),lty=1)
  ```
- ```
  ?bs

  bs(x, df = NULL, knots = NULL, degree = 3, intercept = FALSE,
     Boundary.knots = range(x))
  ```
- By default, bs() choose the range of age as the boundary points.
- By default, there is no intercept, so bs() will generate $K + p$ basis instead of $K + p + 1$. The intercept is added to the regression in lm().
- By default, bs() will generate a cubic spline basis.

Wage Data

- B-Spline regression with 4 knots.



Natural Cubic Spline

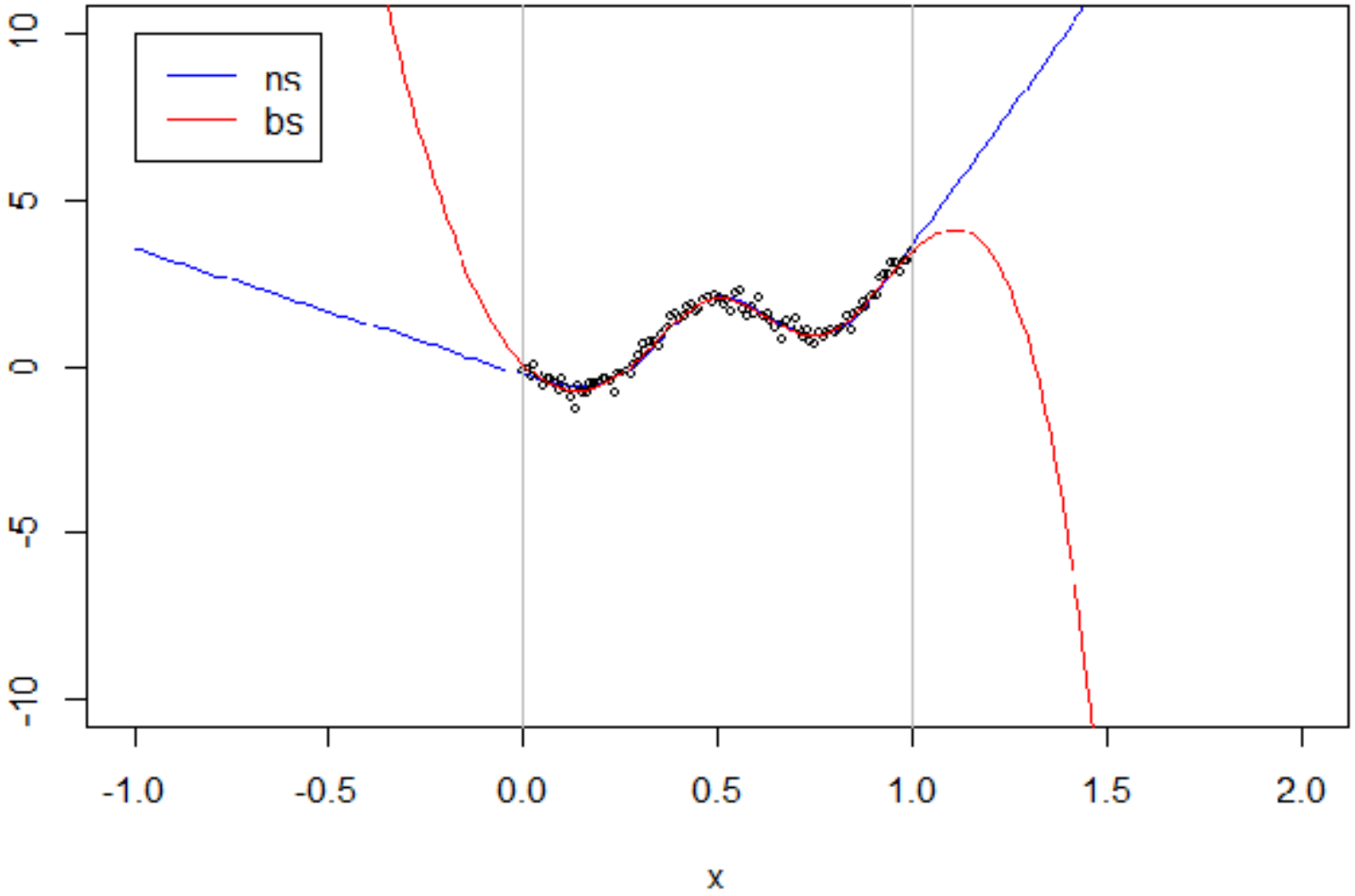
- Suppose we have a knot locations $\kappa_1, \cdots, \kappa_K$ and interval $[a, b]$;
- A natural cubic spline $g(x)$ during $[a, b]$ is
  - $g(x)$ is a cubic spline $(p = 3)$ during $[a, b]$ with knots above.
  - $g(x)$ is a linear function during $(-\infty, a)$ and (b, ∞).
- An equivalent definition is $g'''_-(a) = g''(a) = g'''_+(b) = g''(b) = 0$.
- Let $\kappa_0 = a$ and $\kappa_{K+1} = b$, a cubic spline during $(-\infty, \infty)$ with knots $\kappa_0, \ldots, \kappa_{K+1}$ can be written as

$$g(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \sum_{k=0}^{K+1} b_k (x - \kappa_k)_+^3.$$

- For $x \in (-\infty, a)$, $g''(x) = 2\beta_2 + 6\beta_3 x = 0$, since it is linear.
- So $\beta_2 = 0$ and $\beta_3 = 0$.
- For $x \in (b, \infty)$, $g''(x) = 2\beta_2 + 6\beta_3 x + \sum_{k=0}^{K+1} 6 b_k (x - \kappa_k) = 0$, since it is linear.
- So $\sum_{k=0}^{K+1} b_k = 0$ and $\sum_{k=0}^{K+1} b_k \kappa_k = 0$.
- Number of basis functions(DF) is $(K + 2) + p + 1 - 4 = K + 2$.

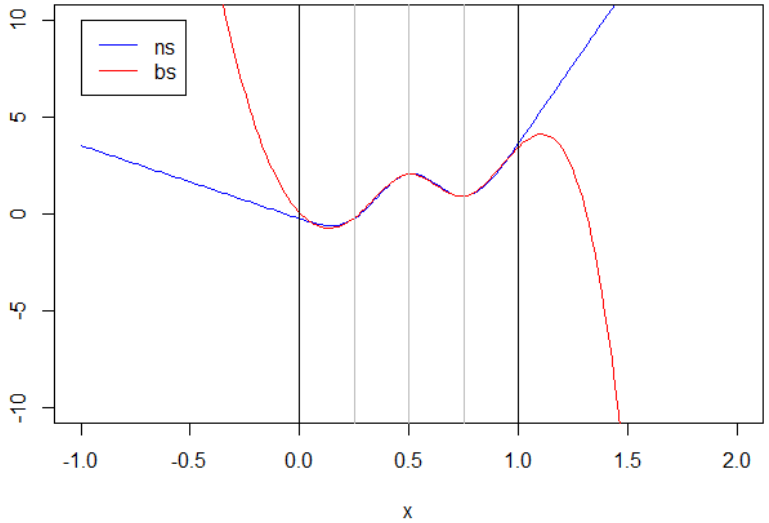
Cubic Spline and Natural Cubic Spline

- Behaviour outside observed interval for natural cubic spline and cubic spline.



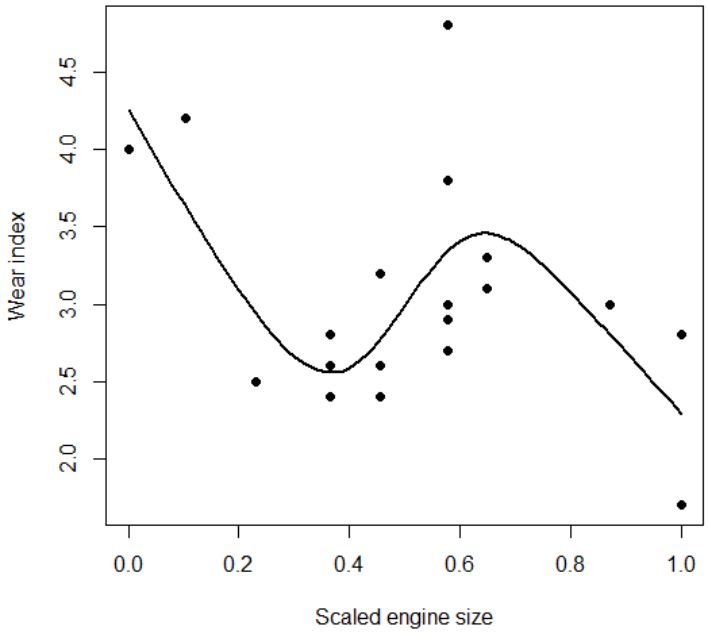
Cubic Spline and Natural Cubic Spline

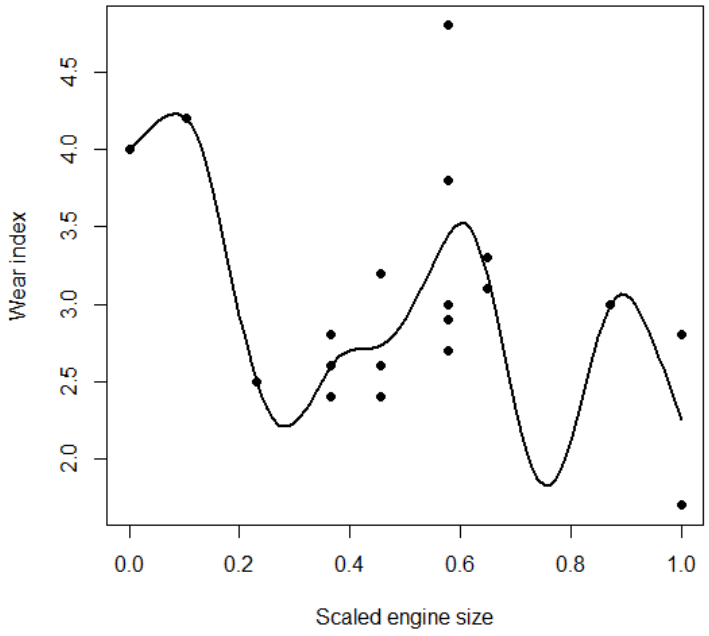
- Example with knots at $0.25, 0.5, 0.75$ and boundary points 0 and 1:



- During each interval, it is a polynomial with order 3: need $(K+1)(3+1)$ parameters.
- Outside boundary, it is linear function: need $2 \times 2 = 4$ parameters.
- Connect nicely at knots, $g_-^{(s)}(\kappa_k) = g_+^{(s)}(\kappa_k)$ for all $k = 1, \ldots, K$ and $s = 0, \ldots, 2$: there are $3K$ constraints.
- Connect nicely at boundary, $g_-^{(s)}(a) = g_+^{(s)}(a)$ and $g_-^{(s)}(b) = g_+^{(s)}(b)$ for $s = 0, \ldots, 2$: there are 6 constraints.
- In total, we need $(K+1)(3+1) + 4 - 3K - 6 = K + 2$ parameters.

Choice of Number and Locations of Knots

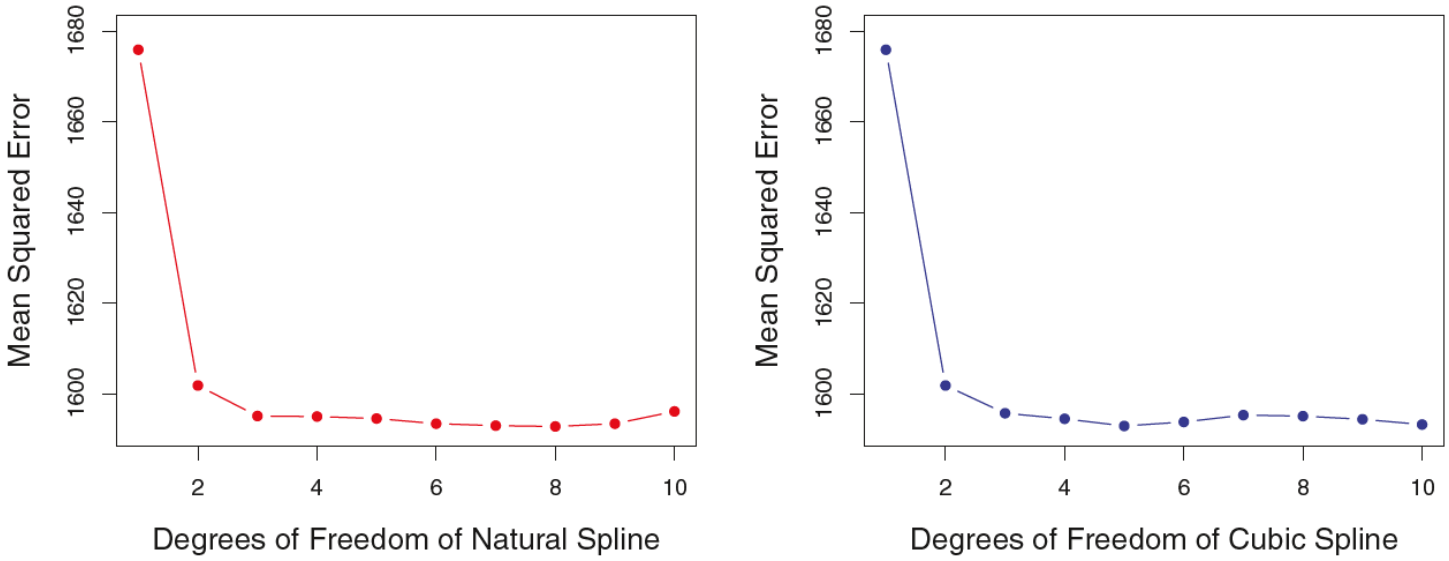




- The number of knots and knot locations matter:
  - Left: natural cubic splines with $K = 4$ knots;
  - Right: natural cubic splines with $K = 7$ knots;
- The regression spline is most flexible in regions that contain a lot of knots
  - One can equally spaced knots within a region, i.e., $[0, R]$;
  - One can also place knots at quantiles of observed predictor x;

Cross Validation

- How many knots (K) should we use?
- One option is to use cross-validation (CV);
  - remove a portion of the data (say 10 %);
  - fit a spline with a certain number of knots to the remaining data;
  - use the spline to make predictions for the held-out portion;
  - repeat this process multiple times until each observation has been left out once
  - compute the overall cross-validated prediction error;
- The graphed cross-validation error will typically look like following



- Degrees of freedom: number of basis functions used by the spline (is related to the number of knots K but different);

Wage Data

- We don't need to always specify the knot locations, use df$=?$
-
```
library(ISLR)
library(splines)
data("Wage")

age=Wage$age
wage=Wage$wage
index=order(age)
age=age[index]
wage=wage[index]
fit=lm(wage~bs(age,df=6))
 ## You can find the locations of the knots used
> attr(bs(age ,df=6) ,"knots")
25%   50%   75%
33.75 42.00 51.00
```
- $K = 3, p = 3$, so DF=$K + p = 6$ (no intercept).
- In this case R chooses knots at ages 33.8, 42.0, and 51.0, which correspond to the 25th, 50th, and 75th percentiles of "age".

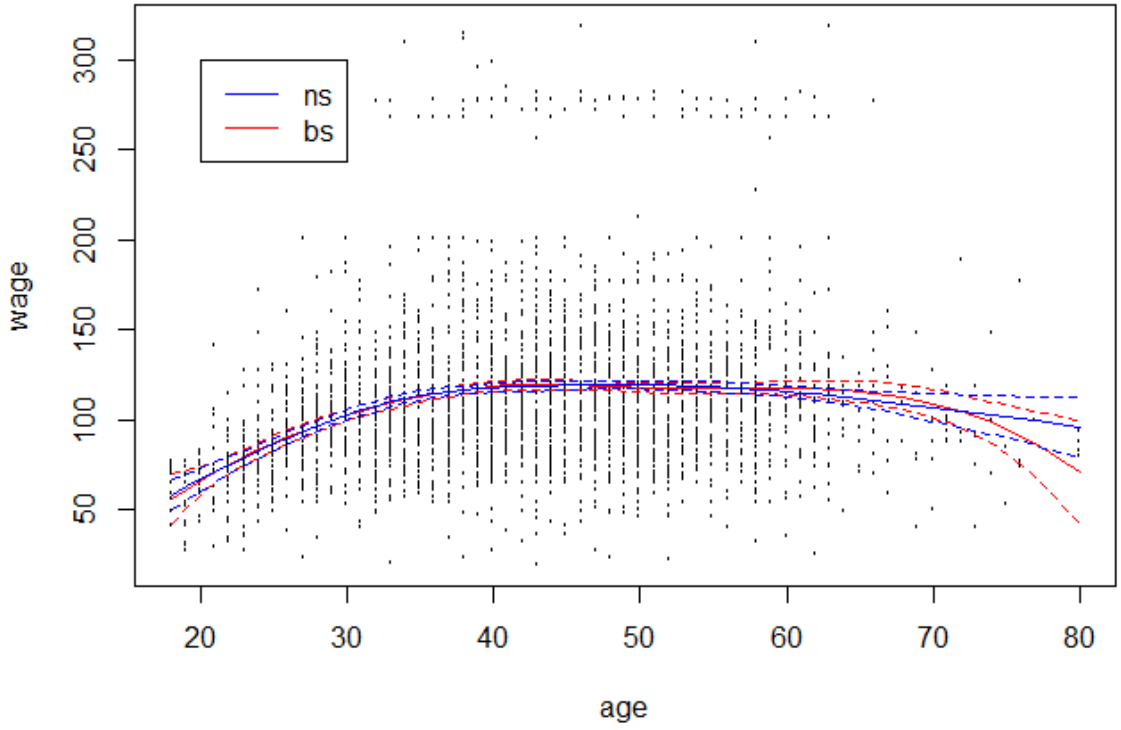
Wage Data

- We can also fit the natural spline using the ns() function.

```
fit2=lm(wage~ns(age,df=4))

plot(age, wage, cex=0.3)
lines(age,fit$fitted.values,col="red")
lines(age,fit2$fitted.values,col="blue")
legend(20,300,c("ns","bs"),col=c("blue","red"),lty=1)

bsse=predict(fit,se=T)$se.fit
nsse=predict(fit2,se=T)$se.fit
bsci1=fit$fitted.values+1.96*bsse
bsci2=fit$fitted.values-1.96*bsse
nsci1=fit2$fitted.values+1.96*nsse
nsci2=fit2$fitted.values-1.96*nsse
lines(age,bsci1,col="red",lty=2)
lines(age,bsci2,col="red",lty=2)
lines(age,nsci1,col="blue",lty=2)
lines(age,nsci2,col="blue",lty=2)
```

Wage Data

- 95% confidence interval:



- What difference do you see between the cubic spline and the natural cubic spline?

Noiseless Data – Interpolation

Suppose we have 10 data points as follows, what do you think is the best way to recover the true underlying function?

- The best way is to connect the dots—(This is called interpolation!)
- But there are many ways to connect the dots, which way is the best way? See following two, which is better? why?
- Is there a "best" way?

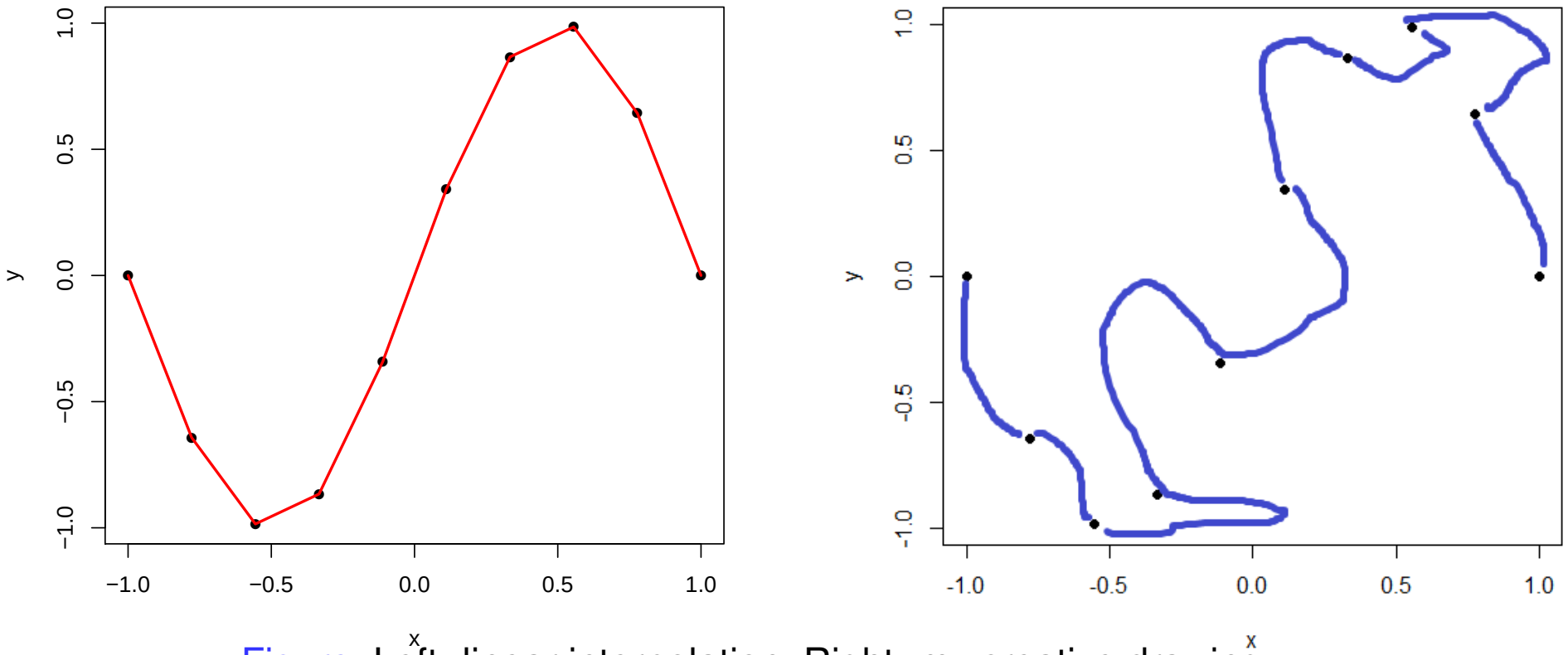


Figure: Left: linear interpolation; Right: my creative drawing

Natural Cubic Splines are Smoothest Interpolators

- Of all functions that are continuous on $[x_1, x_n]$ and have absolutely continuous first derivatives, say $f(x)$, we define the roughness measure as

$$J(f) = \int_{x_1}^{x_n} f''(x)^2 \, dx.$$

- Among all such $f(x)$ functions that also interpolate points $(x_1, y_1), \ldots, (x_n, y_n)$, the natural cubic spline $g(x)$ is the most smooth one;

$$\int_{x_1}^{x_n} g''(x)^2 \, dx \leq \int_{x_1}^{x_n} f''(x)^2 \, dx$$

  for any smooth function $f(x)$;
- In other words, any interpolate that is not identical to the natural cubic spline $g(x)$ will have a higher integrated squared second derivative.
- In a sense, the natural cubic spline is the best way to connect dots (interpolation)!

Smoothing Spline – Noisy Data

- Unfortunately, in Statistics, our data are usually noisy: that is, your response y_i's are measured with errors.
- When data are noisy, interpolation is a bad idea, see right panel of the following plot;

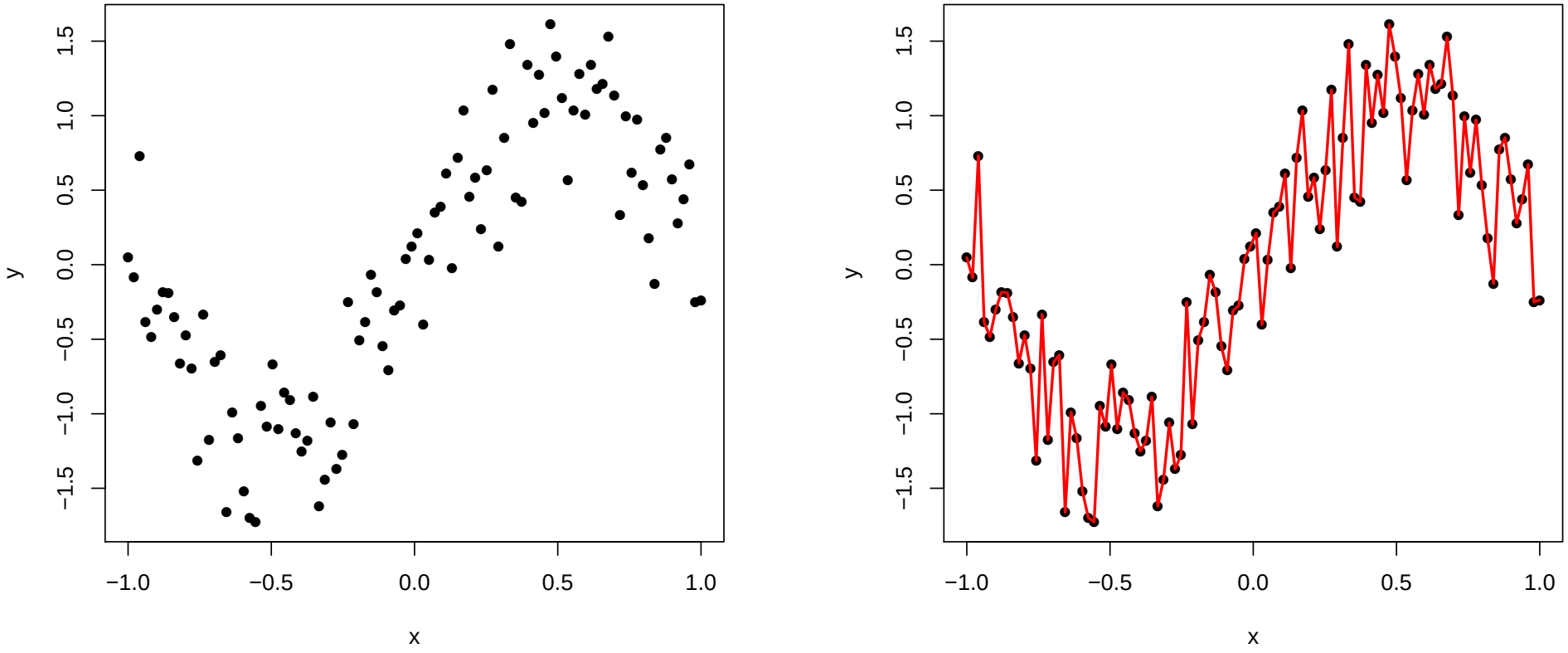


Figure: Left: Noisy data; Right: interpolate the data

Smoothing Spline – Noisy Data

- When data are noisy, it is generally more useful to smooth x_i, y_i data;
- Rather than forcing $g(x_i) = y_i$ (interpolation), it's better to treat the $g(x_i)$ function that minimizes

$$\sum_{i=1}^{n} \{y_i - g(x_i)\}^2 + \lambda \int g''(x)^2 \, dx;$$

- The above equation takes the "Loss+Penalty" formulation (similar to ridge and Lasso regression)
  - $\sum_{i=1}^{n} \{y_i - g(x_i)\}^2$: a loss function that encourages $g(\cdot)$ to fit the data well;
  - $\lambda \int g''(x)^2 \, dx$: a penalty term that penalizes the "roughness" of the function $g(\cdot)$;
- λ is a tunning parameter, used to control the relative weight to be given to the conflicting goals of matching the data and producing a smooth $g(x)$;
  - When $\lambda = 0$: g will be very jumpy and will exactly interpolate the training observations.
  - When $\lambda \to \infty$: $g(\cdot)$ will be perfectly smooth–a straight line (whose $\int g''(x)^2 \, dx = 0$);
- Notice that, we need to find the minimizer over all the functions with second derivatives (not only splines). Is it possible?

Smoothing Spline – Solution

- The solution to the minimization problem

$$\sum_{i=1}^{n}\{y_i - g(x_i)\}^2 + \lambda \int g''(x)^2\, dx;$$

  is a natural cubic spline with knots at $x_1, \cdots, x_n$!
- Sketch of the Proof:
  - Let g be any function with second derivative and let f be the natural cubic spline interpolates $(x_i, g(x_i))$.
  - Clearly, g also interpolates $(x_i, g(x_i))$.
  - So $\sum_{i=1}^{n}\{y_i - g(x_i)\}^2 = \sum_{i=1}^{n}\{y_i - f(x_i)\}^2$ as $f(x_i) = g(x_i)$.
  - However, $\int g''(x)^2\, dx \geq \int f''(x)^2\, dx$ by properties of natural cubic spline.
- Advantage: We don't need to choose the knots!
- All observed data are knots! If you have n observed data points, you need n basis functions!
- Disadvantage: we have an extra tuning parameter λ to shrink some of the coefficients toward 0! (another way to select knots.)

Representation Theorem

- We have seen many good properties of the (cubic) smoothing spline $g(x)$, but how can we estimate $g(x)$?
- To do that, we must first be able to write down the mathematical expression of $g(x)$. Is it possible to do so?
- In fact, there are more than one-way to represent the cubic spline $g(x)$ using some basis functions;
- As a example $g(x) = \alpha_1 b_1(x) + \alpha_2 b_2(x) + \sum_{i=1}^{n} \beta_i R(x_i, x)$, where $b_1(x) = 1, b_2(x) = x$ and

$$R(x,z) = \frac{1}{4}\left[(z-\frac{1}{2})^2 - \frac{1}{12}\right]\left[(x-\frac{1}{2})^2 - \frac{1}{12}\right] - \frac{1}{24}\left[(|x-z|-\frac{1}{2})^4 - \frac{1}{2}(|x-z|-\frac{1}{2})^2 + \frac{7}{240}\right].$$

- $R(x,z)$ is in fact a “reproducing kernel".

Smoothing Splines Regression

- Recall the regression model

$$y_i = f(x_i) + \varepsilon_i$$

- $f(x)$ is a general “smooth function", no closed-form expression;
- However, the assumption is that $f(x)$ can be well approximated by some natural cubic spline $g(x)$, that is

$$f(x) \approx g(x) = \alpha_1 b_1(x) + \alpha_2 b_2(x) + \sum_{j=1}^{n} \beta_j R(x_j, x)$$

- Then, the regression model can be approximated by

$$y_i \approx \alpha_1 b_1(x) + \alpha_2 b_2(x) + \sum_{j=1}^{n} \beta_j R(x_j, x_i) + \varepsilon_i$$

  which is a linear model!
- However, we have $n + 2$ parameters! (more than data points!)
- So, the linear regression minimizing $\sum_{i=1}^{n} \{y_i - g(x_i)\}^2$ is not going to work!
- This is the reason why we need the penalty $\lambda \int g''(x)^2 \, dx$.

Smoothing Splines Regression

- With the representation

$$g(x) = \alpha_1 b_1(x) + \alpha_2 b_2(x) + \sum_{j=1}^{n} \beta_j R(x_j, x),$$

the regression model can be written as

$$\boldsymbol{Y}_{n\times 1} = \boldsymbol{X}_{n\times(n+2)}\boldsymbol{\beta}_{(n+2)\times 1} + \boldsymbol{\varepsilon}_{n\times 1}$$

where the ith row of the model matrix $\boldsymbol{X}$ is

$$\boldsymbol{X}_i = [1, x_i, R(x_1, x_i), R(x_2, x_i), \cdots, R(x_n, x_i)]_{1\times(n+2)},$$

for $i = 1, \ldots, n$ and

$$\boldsymbol{\beta} = (\alpha_1, \alpha_2, \beta_1, \ldots, \beta_n)^T.$$

Smoothing Splines Regression

- Recall the the spline function $g(x)$ was estimated by minimizing

$$\sum_{i=1}^{n}\{y_i - g(x_i)\}^2 + \lambda \int g''(x)^2\, dx;$$

- $\lambda \to \infty$, leads to a straight line estimate for $g(x)$;
- $\lambda \to 0$, close to “interpolation";
- The first part $\sum_{i=1}^{n}\{y_i - g(x_i)\}^2 = (\boldsymbol{Y} - \boldsymbol{X}\boldsymbol{\beta})^T(\boldsymbol{Y} - \boldsymbol{X}\boldsymbol{\beta})$;
- The second part, by the Reproducing-Kernel-Hilbert-Space (RKHS) theory, one can show that

$$\int g''(x)^2\, dx = \boldsymbol{\beta}^T \boldsymbol{S}\boldsymbol{\beta}, \quad \boldsymbol{S} = \begin{pmatrix} \mathbf{0}_{2\times 2} & \mathbf{0}_{2\times n} \\ \mathbf{0}_{n\times 2} & \boldsymbol{R}_{n\times n} \end{pmatrix}$$

where the $(i,j)th$ element of $\boldsymbol{R}_{n\times n}$ is $R(x_i, x_j)$;

Smoothing Splines Regression

- Recall the the spline function $g(x)$ was estimated by minimizing

$$(\boldsymbol{Y} - \boldsymbol{X}\boldsymbol{\beta})^T(\boldsymbol{Y} - \boldsymbol{X}\boldsymbol{\beta}) + \lambda\boldsymbol{\beta}^T\boldsymbol{S}\boldsymbol{\beta};$$

- The solution to the above minimization problem is

$$\widehat{\boldsymbol{\beta}}(\lambda) = (\boldsymbol{X}^T\boldsymbol{X} + \lambda\boldsymbol{S})^{-1}\boldsymbol{X}^T\boldsymbol{Y}.$$

- Then, the hat matrix $\boldsymbol{A}(\lambda)$ can be written as

$$\boldsymbol{A}(\lambda) = \boldsymbol{X}(\boldsymbol{X}^T\boldsymbol{X} + \lambda\boldsymbol{S})^{-1}\boldsymbol{X}^T$$

  recall that $\widehat{\boldsymbol{Y}} = \boldsymbol{A}(\lambda)\boldsymbol{Y}$.
- How to choose an optimal λ?

Choice of λ

- The choice of λ is critical for a good function estimator $g(x)$ for $f(x)$;
- How to choose an optimal λ?
- Cross-validation will do the trick; for the same reason we discussed in the kernel smoothing;
- Generalized cross-validation (GCV):

$$\mathrm{GCV}(\lambda) = \frac{\frac{1}{n}(\boldsymbol{Y} - \boldsymbol{X}\widehat{\boldsymbol{\beta}}(\lambda))^T(\boldsymbol{Y} - \boldsymbol{X}\widehat{\boldsymbol{\beta}}(\lambda))}{\{1 - \frac{1}{n}tr(\boldsymbol{A}(\lambda))\}^2}$$

- GCV is different from CV. GCV is deterministic but CV includes randomness during data splitting.
- We choose the λ with smallest GCV(λ).
- Effective degrees of freedom for linear smoothers (could be fractions):

$$\mathrm{DF}_\lambda = tr(\boldsymbol{A}(\lambda)),$$

  which can be used to show complexity of the fitted spline using λ;

Degree of Freedom

- For linear regression with $x_1, \ldots, x_n \in \mathbb{R}$ and $y_1, \ldots, y_n \in \mathbb{R}$, we minimize
$$(\boldsymbol{Y} - \boldsymbol{X}\boldsymbol{\beta})^T(\boldsymbol{Y} - \boldsymbol{X}\boldsymbol{\beta}),$$
where
$$\boldsymbol{Y} = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix} \text{ and } \boldsymbol{X} = \begin{pmatrix} 1 & x_1 \\ \vdots & \vdots \\ 1 & x_n \end{pmatrix}$$
- The solution to the above minimization problem is
$$\widehat{\boldsymbol{\beta}} = (\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\boldsymbol{Y}.$$
- Then, the hat matrix $\boldsymbol{A}$ can be written as
$$\boldsymbol{A} = \boldsymbol{X}(\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T$$
recall that $\widehat{\boldsymbol{Y}} = \boldsymbol{A}\boldsymbol{Y}$.
- So DF$= tr(A) = 1 + 1 = 2$ (1 covariates + intercept).
- For B-spline regression with K knots and degree p, DF$= K + p + 1$.
- For natural cubic spline regression with K knots, DF$= K + 2$.

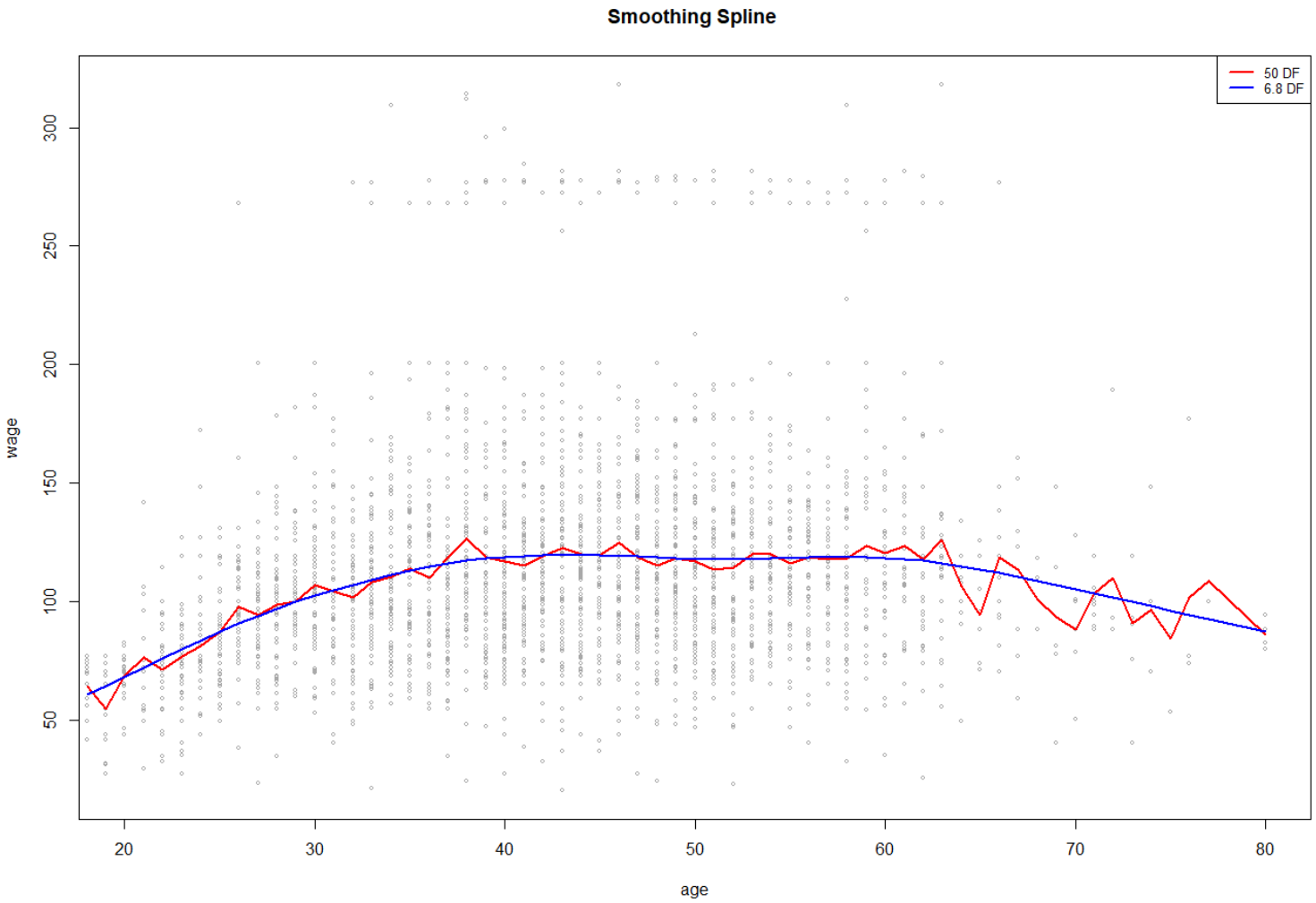
Wage Data

- The R function *smooth.spline()*: fit a smoothing spline;
  - Option *cv = TRUE*: use cross-validation to choose λ;
  - Option *cv = FALSE*: use GCV to choose λ;
  - *all.knots = TRUE*: all distinct points in x are used as knots

```
library(splines)
attach(Wage)
plot(age ,wage ,xlim=c(20,80),cex =.5, col ="darkgrey")
title ("Smoothing Spline ")
fit=smooth.spline(age,wage,df=50) ##Choose lambda by fixing df=50
fit2=smooth.spline(age,wage,cv=TRUE) ##Choose lambda by cross-validation
> fit2$df  ###Find the effective degrees of freedom
[1] 6.8
lines(fit ,col ="red",lwd =2)
lines(fit2 ,col ="blue",lwd =2)
legend ("topright",legend =c("50 DF " ,"6.8 DF"),
col=c("red","blue"),lty =1, lwd =2, cex =.8)
```

Wage Data

- Smoothing spline fit for “Wage" v.s. “age";



- What difference do you see between the smoothing splines using different λ's?

Challenges in Multiple Linear Model

- The multiple linear regression model assumes structure

$$Y = \beta_0 + \beta_1 X_1 + \cdots + \beta_D X_D + \varepsilon$$

- If your data fit the linear model well, we can celebrate!
- However, it rarely happen...Even in the linear model framework, we need to do extra work
  - Transformation of the response variable?
  - Transformation of the predictors?
  - Polynomials regressions?
  - Interactions among predictors?
  - Variable selection?
- A lot of work...!
- Which makes you wonder: is there a “magical regression machine" that I can throw in bunch of data and gives you an automatic fit?

Magical Regression Machine

- The “magical regression model" should look like

$$Y = f(X_1, \ldots, X_D) + \varepsilon$$

- $f(\cdot)$ is any D dimensional function;
- However, there is no way to fit such a function;
- mainly due to the “curse of dimensionality":
  - the number of model parameters inflate;
  - the data distribute very unevenly in high dimensional space; (which is really bad for spline basis or kernel smoother, why?)

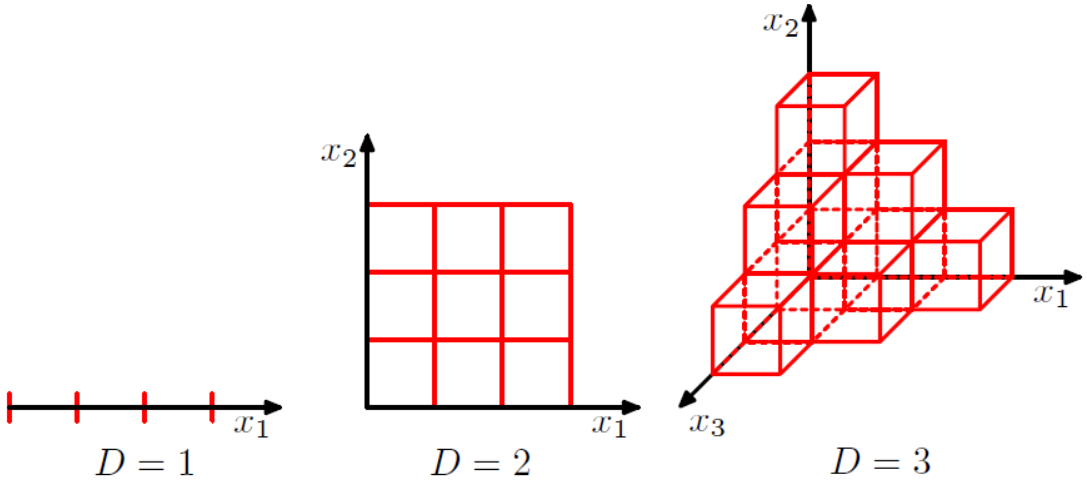


Figure: Illustration of the curse of dimensionality: the number of regions of a regular grid grows exponentially with the dimensionality D of the space.

Number of Parameter Inflates

- If we have D predictors;
- Consider the polynomial model with coefficients up to order 3:

$$Y = \beta_0 + \sum_{j=1}^{D} \beta_j X_j + \sum_{j=1}^{D} \sum_{K=1}^{D} \beta_{jk} X_j X_k + \sum_{j=1}^{D} \sum_{K=1}^{D} \sum_{l=1}^{D} \beta_{jkl} X_j X_k X_l$$

- the number of independent coefficients (not all of the coefficients are independent due to interchange symmetries amongst the X variables) grows proportionally to D^3;
- For a polynomial of order q, the growth in the number of coefficients is like D^q;
- the number of parameters quickly run out of control.

Tensor Product Spline

- If we have $D = 2$ predictors X_1, X_2;
- B-spline basis: $B_i(X_1)$'s, $B_i(X_2)$'s, $\mathrm{DF} = K + p + 1$ for each
- Tensor product basis: $B_{ij}(X_1, X_2) = B_i(X_1)B_j(X_2)$, $\mathrm{DF} = (K + p + 1)^2$
- the number of independent coefficients (not all of the coefficients are independent due to interchange symmetries amongst the X variables) grows proportionally to $(K + p + 1)^D$;
- the number of parameters quickly run out of control.
- OLS works poorly, need penalty

Airquality Data: $D = 3$

- Split the data

```
library(splines)
library(glmnet)
set.seed(1)

data=airquality[!(rowSums(is.na(airquality))),]
n=dim(data)[1]
index_train=sample(1:n,round(n*0.8))
index_test=setdiff(1:n,index_train)
x_all=data[,2:4]
for (i in 1:3){
    r=range(x_all[,i])
    x_all[,i]=(x_all[,i]-r[1])/(r[2]-r[1])
}
y=log(data$Ozone)[index_train]
x=x_all[index_train,]
```

Airquality Data: $D = 3$

- Construct tensor product basis for training

```
p=dim(x)[2]
b.list=list()
for (i in 1:p){ # make basis for each dimension
    bsi=bs(x[,i], Boundary.knots=c(0,1), knots=c(0.25, 0.5, 0.75), degree=3,
    intercept = TRUE)
    b.list[[length(b.list)+1]] =  bsi
}
tb=c()
for (i1 in 1:dim(b.list[[1]])[2]){  # make tensor product basis
    for (i2 in 1:dim(b.list[[2]])[2]){
        for (i3 in 1:dim(b.list[[3]])[2]){
            bsi=b.list[[1]][,i1]*b.list[[2]][,i2]*b.list[[3]][,i3]
            tb=cbind(tb,bsi)
        }
    }
}
#fit ols
lmod=lm(y~tb-1)
#fit lasso
cv.lasso=cv.glmnet(tb,y,intercept = FALSE, standardize=FALSE)
cf=coef(cv.lasso,s=cv.lasso$lambda.min)[-1]
#refit ols
index=which(cf!=0)
refit=lm(y~tb[,index]-1)
```

Airquality Data: $D = 3$

- Construct tensor product basis for testing

```
y_test=log(data$Ozone)[index_test]
x_test=x_all[index_test,]
p=dim(x_test)[2]
b.list=list()
for (i in 1:p){ # make basis for each dimension
    bsi=bs(x_test[,i], Boundary.knots=c(0,1), knots=c(0.25, 0.5, 0.75),
    degree=3, intercept = TRUE)
    b.list[[length(b.list)+1]] =  bsi
}
tb=c()
for (i1 in 1:dim(b.list[[1]])[2]){  # make tensor product basis
    for (i2 in 1:dim(b.list[[2]])[2]){
        for (i3 in 1:dim(b.list[[3]])[2]){
            bsi=b.list[[1]][,i1]*b.list[[2]][,i2]*b.list[[3]][,i3]
            tb=cbind(tb,bsi)
        }
    }
}
```

Airquality Data: $D = 3$

- Calculate testing error

```
cf1=lmod$coefficients
cf1[which(is.na(cf1))]=0
ypre1=tb%*%cf1

cf2=coef(cv.lasso,s=cv.lasso$lambda.min)[-1]
ypre2=tb%*%cf2

index=which(cf!=0)
cf3=rep(0,dim(tb)[2])
cf3[index]=refit$coefficients
ypre3=tb%*%cf3

e1=sqrt(mean((y_test-ypre1)^2))
e2=sqrt(mean((y_test-ypre2)^2))
e3=sqrt(mean((y_test-ypre3)^2))
c(e1,e2,e3)

240616204911346496, 1.35824715656813, 1.304110996539

summary(y_test)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  2.398   2.802   3.478   3.442   3.880   4.804
```

Reproducing Kernel Hilbert Space

- Consider a function space H.
- Any element $f \in H$ is a function $x \to f(x)$ for $x \in \mathcal{X}$.
- $H = C^2[0,1] = \{f : \int_0^1 |f^{(2)}(x)|^2 dx < \infty\}$ with $\mathcal{X} = [0,1]$.
- A kernel $R(\cdot,\cdot)$ is a function taking two argument: $(x,y) \to R(x,y)$ and $R(x,y) = R(y,x)$.
- Gaussian Kernel: $R(x,y) = \frac{1}{\sqrt{2\pi\sigma}} e^{-\frac{(x-y)^2}{2\sigma^2}}$
- Polynomial Kernel: $R(x,y) = (1+xy)^d$.
- A function space H is called Reproducing Kernel Hilbert Space (RKHS) if there is a kernel $R(\cdot,\cdot)$ such that

$$< R_x, f >= f(x) \text{ for any } x \text{ and any } f \in H,$$

  here $< g, f >$ is the inner product and $y \to R_x(y) = R(x,y)$ is a function.
- R is called reproducing kernel.

Reproducing Kernel on $C^2[0,1]$

- Recall $H = C^2[0,1] = \{f : \int_0^1 |f^{(2)}(x)|^2 dx < \infty\}$ with $\mathcal{X} = [0,1]$.
- Define inner product on H:

$$< f, g >= f(0)g(0) + f^{(1)}(0)g^{(1)}(0) + \int_0^1 f^{(2)}(t)g^{(2)}(t)dt.$$

- Define kernel:

$$R(x,y) = R_x(y) = 1 + xy + \int_0^1 (x-u)_+(y-u)_+ du$$

- To show R is a reproducing kernel for H, we need $< R_x, f >= f(x)$ for all $x \in [0,1]$ and $f \in C^2[0,1]$.
- How to show?

Reproducing Kernel on $C^2[0,1]$

- Notice that

$$
\begin{aligned}
R_x(y) &= 1 + xy + \int_0^1 (x-u)_+(y-u)_+ du \\
&= \begin{cases} 1 + xy + \int_0^y (x-u)(y-u)du & \text{if } y < x \\ 1 + xy + \int_0^x (x-u)(y-u)du & \text{if } y > x \end{cases}
\end{aligned}
$$

- it follows that
  - $R_x(0) = 1$
  - $R_x^{(1)}(y) = \begin{cases} x + xy - \frac{1}{2}y^2 & \text{if } y < x \\ x + 1 + \frac{1}{2}x^2 & \text{if } y > x \end{cases}$, so $R_x^{(1)}(0) = x$.
  - $R_x^{(2)}(y) = \begin{cases} x - y & \text{if } y < x \\ 0 & \text{if } y > x \end{cases}$, so $R_x^{(2)}(y) = (x-y)_+$.
- Therefore, we have

$$
\begin{aligned}
< R_x, f > &= R_x(0)f(0) + R_x^{(1)}(0)f^{(1)}(0) + \int_0^1 R_x^{(2)}(t)f^{(2)}(t)dt \\
&= f(0) + f^{(1)}(0)x + \int_0^1 (x-t)_+ f^{(2)}(t)dt
\end{aligned}
$$

Reproducing Kernel on $C^2[0,1]$

- Taylor Expansion with Integral Remainder:

$$\begin{aligned} f(x) &= f(0) + f^{(1)}(0)x + \int_0^x (x-t) f^{(2)}(t)dt \\ &= f(0) + f^{(1)}(0)x + \int_0^1 (x-t)_+ f^{(2)}(t)dt. \end{aligned}$$

- So we prove $< R_x, f >= f(x)$ for any $x \in [0,1]$ and $f \in C^2[0,1]$.
- R is a reproducing kernel for $C^2[0,1]$.
- $R(x,y) = \underbrace{1+xy}_{R_0(x,y):\text{linear}} + \underbrace{\int_0^1 (x-u)_+(y-u)_+ du}_{R_1(x,y):\text{nonlinear}}$

Decomposition of Reproducing Kernel (Optional)

- $R(x,y) = \underbrace{1+xy}_{R_0(x,y)} + \underbrace{\int_0^1 (x-u)_+(y-u)_+ du}_{R_1(x,y)}$
- R_0 is the reproducing kernel for $H_0 = \{f \in H : f^{(2)}(x) = 0 \text{ for all } x\}$
- R_1 is the reproducing kernel for $H_1 = \{f \in H : f(0) = f^{(1)}(0) = 0\}$
- H_0 and H_1 are orthogonal: $< f_0, f_1 >= 0$ for all $f_0 \in H_0, f_1 \in H_1$.
- $H = H_0 + H_1$: any $f \in H$, we have $f = f_0 + f_1$ for a unique pair of $f_0 \in H_0, f_1 \in H_1$.

Decomposition of Reproducing Kerne (Optional)I

- Consider the smoothing spline regression problem:

$$\sum_{i=1}^{n}\{y_i - f(x_i)\}^2 + \lambda \int_0^1 |f^{(2)}(x)|^2\, dx;$$

- Minimize the problem over $H = C^2[0,1]$.
- We consider a subspace $H_{11} = \{g(x) = \sum_{i=1}^{n} a_i R_1(x_i, x),\ \text{with } \alpha_i \in \mathbb{R}\}$
- H_{11} is a subspace of $H_1 = \{f \in H : f(0) = f^{(1)}(0) = 0\}$.
- Let H_{12} be the subspace of H_1 which is orthogonal to H_{11}.
  - H_{11} and H_{12} are orthogonal: $< f_{11}, f_{12} >= 0$ for all $f_{11} \in H_{11}, f_{12} \in H_{12}$.
  - $H_1 = H_{11} + H_{12}$: any $f_1 \in H_1$, we have $f_1 = f_{11} + f_{12}$ for a unique pair of $f_{11} \in H_{11}, f_{12} \in H_{12}$.

Decomposition of Reproducing Kernel (Optional)

- For any $f \in H$, we have $f = f_0 + f_1 = f_0 + f_{11} + f_{12}$.
- Notice that $f_0 \in H_0 = \{f \in H : f^{(2)}(x) = 0 \text{ for all } x\}$, it follows

$$\int_0^1 |f^{(2)}(x)|^2 = \int_0^1 |f_1^{(2)}(x)|^2$$

- Also $f_1 \in H_1 = \{f \in H : f(0) = f^{(1)}(0) = 0\}$, we have

$$\int_0^1 |f_1^{(2)}(x)|^2 =< f_1, f_1 >$$

- H_{11} and H_{12} are orthogonal, and R_1 is a reproducing kernel of H_1:

$$f_{12}(x_i) =< R_1(x_i, \cdot), f_{12}(\cdot) >= 0$$

and

$$\begin{aligned} < f_1, f_1 > &= < f_{11} + f_{12}, f_{11} + f_{12} > \\ &= < f_{11}, f_{11} > + < f_{12}, f_{12} > +2 < f_{11}, f_{12} > \\ &= < f_{11}, f_{11} > + < f_{12}, f_{12} > . \end{aligned}$$

Decomposition of Reproducing Kernel (Optional)

- Combining the above, we show that

$$
\begin{aligned}
& \sum_{i=1}^{n}\{y_i - f(x_i)\}^2 + \lambda \int_0^1 |f^{(2)}(x)|^2\, dx \\
= & \sum_{i=1}^{n}\{y_i - f_0(x_i) - f_{11}(x_i) - f_{12}(x_i)\}^2 + \lambda < f_{11}, f_{11} > + \lambda < f_{12}, f_{12} > \\
= & \sum_{i=1}^{n}\{y_i - f_0(x_i) - f_{11}(x_i)\}^2 + \lambda < f_{11}, f_{11} > + \lambda < f_{12}, f_{12} >,
\end{aligned}
$$

- So it will be minimized with $f_{12} = 0$.
- The solution will have the form

$$
f(x) = f_0(x) + f_{11}(x) = \alpha_1 + \alpha_2 x + \sum_{i=1}^{n} \beta_i R_1(x_i, x)
$$

Decomposition of Reproducing Kernel (Optional)

- Because $f_{12} = 0$, so $f_1 = f_{11}$.
- We verify

$$\begin{aligned}
\int_0^1 |f^{(2)}(x)|^2 =< f_1, f_1 > &= < f_{11}, f_{11} > \\
&= < \sum_{i=1}^n \beta_i R_1(x_i, \cdot), \sum_{i=1}^n \beta_i R_1(x_i, \cdot) > \\
&= \sum_{i=1}^n \sum_{j=1}^n \beta_i \beta_j R_1(x_i, x_j)
\end{aligned}$$

Smoothing Spline – Kernel Algorithm

- With the representation

$$f(x) = \alpha_1 + \alpha_2 x + \sum_{j=1}^{n} \beta_j R_1(x_j, x),$$

the regression model can be written as

$$\boldsymbol{Y}_{n\times 1} = \boldsymbol{X}_{n\times(n+2)}\boldsymbol{\beta}_{(n+2)\times 1} + \boldsymbol{\varepsilon}_{n\times 1}$$

where the ith row of the model matrix $\boldsymbol{X}$ is

$$\boldsymbol{X}_i = [1, x_i, R_1(x_1, x_i), R_1(x_2, x_i), \cdots, R_1(x_n, x_i)]_{1\times(n+2)},$$

for $i = 1, \ldots, n$ and

$$\boldsymbol{\beta} = (\alpha_1, \alpha_2, \beta_1, \ldots, \beta_n)^T.$$

Smoothing Spline – Kernel Algorithm

- Recall the the spline function $f(x)$ was estimated by minimizing

$$\sum_{i=1}^{n}\{y_i - f(x_i)\}^2 + \lambda \int |f^{(2)}(x)|^2 \, dx;$$

- The first part $\sum_{i=1}^{n}\{y_i - f(x_i)\}^2 = (\boldsymbol{Y} - \boldsymbol{X}\boldsymbol{\beta})^T(\boldsymbol{Y} - \boldsymbol{X}\boldsymbol{\beta})$;

Smoothing Spline – Kernel Algorithm

- Recall

$$f(x) = \alpha_1 + \alpha_2 x + \underbrace{\sum_{j=1}^{n} \beta_j R_1(x_j, x)}_{f_{11}},$$

$$< f, g > = f(0)g(0) + f^{(1)}(0)g^{(1)}(0) + \int_0^1 f^{(2)}(t)g^{(2)}(t)dt,$$

$$R_1(x, y) = \int_0^1 (x-u)_+(y-u)_+ du \text{ with } R_1(x,0) = R_1^{(1)}(x,0) = 0.$$

- The second part, we verify

$$\begin{aligned}
\int_0^1 |f^{(2)}(x)|^2 = \int_0^1 |f_{11}^{(2)}(x)|^2 &= < f_{11}, f_{11} > \\
&= < \sum_{i=1}^{n} \beta_i R_1(x_i, \cdot), \sum_{i=1}^{n} \beta_i R_1(x_i, \cdot) > \\
&= \sum_{i=1}^{n}\sum_{j=1}^{n} \beta_i \beta_j R_1(x_i, x_j) \\
&= \boldsymbol{\beta}^T \boldsymbol{S} \boldsymbol{\beta}, \qquad \boldsymbol{S} = \begin{pmatrix} \mathbf{0}_{2\times 2} & \mathbf{0}_{2\times n} \\ \mathbf{0}_{n\times 2} & \boldsymbol{R}_{n\times n} \end{pmatrix}
\end{aligned}$$

where the $(i,j)th$ element of $\boldsymbol{R}_{n\times n}$ is $R_1(x_i, x_j)$;

- The loss function becomes $(\boldsymbol{Y} - \boldsymbol{X}\boldsymbol{\beta})^T(\boldsymbol{Y} - \boldsymbol{X}\boldsymbol{\beta}) + \lambda \boldsymbol{\beta}^T \boldsymbol{S} \boldsymbol{\beta}$

Another Reproducing Kernel on $C^2[0,1]$

- Reproducing kernel is not unique.
- If we define inner product on $H = C^2[0,1]$ as

$$< f, g >= \int_0^1 f(t)dt \int_0^1 g(t)dt + \int_0^1 f^{(1)}(t)dt \int_0^1 g^{(1)}(t)dt + \int_0^1 f^{(2)}(t)g^{(2)}(t)dt$$

- we can show $H = H_0 + H_1$ with
  - $H_0 = \{f \in H : f^{(2)}(x) = 0 \text{ for all } x\}$
  - $H_1 = \{f \in H : \int_0^1 f(t)dt = \int_0^1 f^{(1)}(t)dt = 0\}$ with reproducing kernel

$$\begin{aligned} R_1(x,y) = & \frac{1}{4}\left[(x-\frac{1}{2})^2 - \frac{1}{12}\right]\left[(y-\frac{1}{2})^2 - \frac{1}{12}\right] \\ & - \frac{1}{24}\left[(|x-y| - \frac{1}{2})^4 - \frac{1}{2}(|x-y| - \frac{1}{2})^2 + \frac{7}{240}\right]. \end{aligned}$$

- By same argument, the solution of smoothing spline regression has the form:

$$f(x) = \alpha_1 + \alpha_2 x + \sum_{i=1}^{n} \beta_i R_1(x_i, x).$$

From Kernel to RKHS

- If H is a RKHS, then there is a reproducing kernel R.
- If we have kernel R, we can construct a RKHS!
- Consider the function has the form: $f(x) = \sum_{i=1}^{\infty} a_i R(x_i, x)$ with $x_i \in \mathcal{X}$ and $a_i \in \mathbb{R}$.
- Suppose $f(x) = \sum_{i=1}^{\infty} a_i R(x_i, x)$
- and $g(x) = \sum_{i=1}^{\infty} b_i R(y_i, x)$
- Define inner product

$$< f, g >= \sum_{i=1}^{\infty} \sum_{j=1}^{\infty} a_i b_j R(x_i, y_j).$$

- Define function space:

$$H = \{f(x) = \sum_{i=1}^{\infty} a_i R(x_i, x) : x_i \in \mathcal{X} \text{ with } < f, f >= \sum_{i=1}^{\infty} \sum_{j=1}^{\infty} a_i a_j R(x_i, x_j) < \infty\}$$

- It can be shown that R is the reproducing kernel of H.

From Kernel to RKHS

- Polynomial Kernel: $R(x, y) = (1 + xy)^d$.
- $H = \{f : f(x) = a_0 + a_1 x + \ldots + a_d x^d\}$
- Gaussian Kernel: $R(x, y) = \frac{1}{\sqrt{2\pi\sigma}} e^{-\frac{(x-y)^2}{2\sigma^2}}$, with $x, y \in \mathcal{X}$
  - H is complicate in this case.
  - Functions in H are smooth (infinite times differentiable).
  - H is small: it does not contain any polynomial.
  - H is large: for any continuous function g and interval $[a, b]$, there is a sequence $f_n \in H$ such that

$$\sup_{x \in [a,b]} |f_n(x) - g(x)| \to 0.$$

Kernel Ridge Regression

- Loss function

$$\sum_{i=1}^{n}\{y_i - f(x_i)\}^2 + \lambda < f, f >;$$

- Similar argument can show that, the solution should have the form

$$f(x) = f_{11}(x) = \sum_{i=1}^{n} \beta_i R(x_i, x).$$

- The loss function becomes

$$(\boldsymbol{Y} - \boldsymbol{R}\boldsymbol{\beta})^T(\boldsymbol{Y} - \boldsymbol{R}\boldsymbol{\beta}) + \lambda\boldsymbol{\beta}^T\boldsymbol{R}\boldsymbol{\beta},$$

where $\boldsymbol{R} = [R(x_i, x_j)] \in \mathbb{R}^{n\times n}$.

Smoothing Spline ANOVA

- Model with q-way interaction:

$$f(x_1, \ldots, x_p) = \text{constant} + \sum_{i=1}^{p} g_i(x_i) + \sum_{1 \le i_1 < i_2 \le p}^{p} g_{i_1, i_2}(x_{i_1}, x_{i_2}) +$$
$$\ldots + \sum_{1 \le i_1 < \ldots < i_q \le p}^{p} g_{i_1, \ldots, i_q}(x_{i_1}, \ldots, x_{i_q})$$

- Example: $g_2(x_2)$ models the main effect
- Example: $g_{1,2,4}(x_1, x_2, x_4)$ models the three-way interaction
- Example: $q = 1$ becomes additive model

Smoothing Spline ANOVA

- Recall $R(x,y) = 1 + \underbrace{xy + \int_0^1 (x-u)_+(y-u)_+ du}_{\widetilde{R}(x,y)}$
- We can use $\widetilde{R}_1(x_1, \cdot)$ to model $g_1(x_1)$ (removing intercept)
- Kernel for x_1, x_2:

$$\widetilde{R}_{1,2}((x_1,x_2),(y_1,y_2)) = \widetilde{R}(x_1,y_1)\widetilde{R}(x_2,y_2)$$

- We can use $\widetilde{R}_{1,2}((x_1,x_2),\cdot)$ to model $g_{1,2}(x_1,x_2)$
- Kernel for x_1, x_2, x_4:

$$\widetilde{R}_{1,2,4}((x_1,x_2,x_4),(y_1,y_2,y_4)) = \widetilde{R}(x_1,y_1)\widetilde{R}(x_2,y_2)\widetilde{R}(x_4,y_4)$$

- We can use $\widetilde{R}_{1,2,4}((x_1,x_2,x_4),\cdot)$ to model $g_{1,2,4}(x_1,x_2,x_4)$

Smoothing Spline ANOVA

- Model with q-way interaction:

$$f(x_1, \ldots, x_p) = \text{constant} + \sum_{i=1}^{p} g_i(x_i) + \sum_{1 \le i_1 < i_2 \le p}^{p} g_{i_1, i_2}(x_{i_1}, x_{i_2}) +$$
$$\ldots + \sum_{1 \le i_1 < \ldots < i_q \le p}^{p} g_{i_1, \ldots, i_q}(x_{i_1}, \ldots, x_{i_q}).$$

- Kernel with q-way interaction:

$$R^{(q)}(\boldsymbol{x}, \cdot) = 1 + \sum_{i=1}^{p} \widetilde{R}_i(x_i, \cdot) + \sum_{1 \le i_1 < i_2 \le p}^{p} \widetilde{R}_{i_1, i_2}((x_{i_1}, x_{i_2}), \cdot) +$$
$$\ldots + \sum_{1 \le i_1 < \ldots < i_q \le p}^{p} \widetilde{R}_{i_1, \ldots, i_q}((x_{i_1}, \ldots, x_{i_q}), \cdot).$$

Here $\boldsymbol{x} = (x_1, \ldots, x_p)$.

Kernel Ridge Regression

- The loss function becomes

$$\widehat{\boldsymbol{\beta}} = \underset{\boldsymbol{\beta}}{\operatorname{argmin}} (\boldsymbol{Y} - \boldsymbol{R}\boldsymbol{\beta})^T (\boldsymbol{Y} - \boldsymbol{R}\boldsymbol{\beta}) + \lambda \boldsymbol{\beta}^T \boldsymbol{R}\boldsymbol{\beta},$$

  where $\boldsymbol{R} = [R^{(q)}(\boldsymbol{x}_i, \boldsymbol{x}_j)] \in \mathbb{R}^{n \times n}$.
- Estimator: $\widehat{f}(\boldsymbol{x}) = \sum_{i=1}^n \widehat{\beta}_i R^{(q)}(\boldsymbol{x}_i, \boldsymbol{x}) = \Phi^\top(\boldsymbol{x}) \widehat{\boldsymbol{\beta}}$ where

$$\Phi(\boldsymbol{x}) = \begin{pmatrix} R^{(q)}(\boldsymbol{x}_1, \boldsymbol{x}) \\ \dots \\ R^{(q)}(\boldsymbol{x}_n, \boldsymbol{x}) \end{pmatrix}, \widehat{\boldsymbol{\beta}} = \begin{pmatrix} \widehat{\beta}_1, \\ \dots \\ \widehat{\beta}_n \end{pmatrix} \in \mathbb{R}^n.$$

Airquality Data

- Model with three way interaction

```
rm(list=ls())
library(gss)
set.seed(1)
data=airquality[!(rowSums(is.na(airquality))),]
n=dim(data)[1]
index_train=sample(1:n,round(n*0.8))
index_test=setdiff(1:n,index_train)
x_all=data[,2:4]
for (i in 1:3){
    r=range(x_all[,i])
    x_all[,i]=(x_all[,i]-r[1])/(r[2]-r[1])
}
y=log(data$Ozone)[index_train]
x=x_all[index_train,]

data_train=data.frame(x,y)
smod=ssanova(y~Solar.R*Wind*Temp, data=data_train)
```

Airquality Data

- Model with three way interaction

```
smod

Call:
ssanova(formula = y ~ Solar.R * Wind * Temp, data = data_train)
Terms:
[1] "1"                 "Solar.R"           "Wind"
[4] "Temp"              "Solar.R:Wind"      "Solar.R:Temp"
[7] "Wind:Temp"         "Solar.R:Wind:Temp"

Number of unpenalized and penalized terms:
                  Unpenalized Penalized
1                           1         0
Solar.R                     1         1
Wind                        1         1
Temp                        1         1
Solar.R:Wind                1         3
Solar.R:Temp                1         3
Wind:Temp                   1         3
Solar.R:Wind:Temp           1         7
total                       8        19

Smoothing parameters are selected by GCV with alpha=1.4.
```

Airquality Data

- Check testing error

```
y_test=log(data$Ozone)[index_test]
x_test=x_all[index_test,]
data_test=data.frame(x_test)
yhat=predict(smod,data_test,se=TRUE)$fit
error=sqrt(mean((yhat-y_test)^2))
error

0.498310670160604
```

Airquality Data

- Models with two and one way interaction

```
smod1=ssanova(y~Solar.R+Wind+Temp, data=data_train)
yhat=predict(smod1,data_test,se=TRUE)$fit
error1=sqrt(mean((yhat-y_test)^2))
error1

smod2=ssanova(y~Solar.R+Wind+Temp+Solar.R:Wind+Solar.R:Temp
+Wind:Temp, data=data_train)
yhat=predict(smod2,data_test,se=TRUE)$fit
error2=sqrt(mean((yhat-y_test)^2))
error2

c(error1, error2, error)

0.358587175148685, 0.486669497524865, 0.498310670160604
#try multiple data splitting or k-fold cv
```

Spline Meets Regularization

- In B-spline regression, we need to determine the number of knots K.
- By the idea of Lasso, we can tentatively choose a large K (say 100), and then select the basis function.
- The loss function becomes

$$(\boldsymbol{Y} - \mathbf{X}\boldsymbol{\beta})^\top (\boldsymbol{Y} - \mathbf{X}\boldsymbol{\beta}) + \lambda \sum_{j=-p}^{K} |\beta_j|,$$

with $\boldsymbol{\beta} = (\beta_{-p}, \ldots, \beta_K)^T$ and

$$\mathbf{X} = \begin{pmatrix} B_{-p}(x_1) & & \cdots & B_K(x_1) \\ B_{-p}(x_2) & & \cdots & B_K(x_2) \\ \vdots & \vdots \quad \vdots & \vdots & \vdots \\ B_{-p}(x_n) & & \cdots & B_K(x_n) \end{pmatrix}$$

- You transform the problem from selecting K to selecting λ.

Spline Meets Regularization

- Variable selection in additive model.
- Goal: $y = \alpha + f_1(x_1) + f_2(x_2) + \ldots + f_s(x_s) + \epsilon$
- Which f_i's should be removed?
- Each $f_i(x) \approx \sum_{j=-p}^{K-1} \beta_{i,j} B_j(x)$ (removing the intercept).
- We can apply group lasso

$$(\boldsymbol{Y} - \alpha \mathbf{1}_n - \mathbf{X}_1 \boldsymbol{\beta}_1 - \ldots - \mathbf{X}_s \boldsymbol{\beta}_s)^\top (\boldsymbol{Y} - \alpha \mathbf{1}_n - \mathbf{X}_1 \boldsymbol{\beta}_1 - \ldots - \mathbf{X}_s \boldsymbol{\beta}_s) + \lambda \sum_{j=1}^{s} \|\boldsymbol{\beta}_j\|,$$

with $\boldsymbol{\beta}_j = (\beta_{j,-p}, \ldots, \beta_{j,K-1})^T$ and

$$\mathbf{X}_j = \begin{pmatrix} B_{-p}(x_{j,1}) & & \cdots & B_{K-1}(x_{j,1}) \\ B_{-p}(x_{j,2}) & & \cdots & B_{K-1}(x_{j,2}) \\ \vdots & \vdots \quad \vdots & \vdots & \vdots \\ B_{-p}(x_{j,n}) & & \cdots & B_{K-1}(x_{j,n}) \end{pmatrix}$$

Spline Meets Regularization

- Patten Selection.
- Goal: $y = \alpha + f_1(x_1) + f_2(x_2) + \ldots + f_s(x_s) + \epsilon$
- Which f_i's are linear.
- Each $f_i(x) \approx \beta_{i,1}x + \underbrace{\beta_{i,2}x^2 + \cdots + \beta_{i,p}x^p + \sum_{k=1}^{K} b_{i,k}(x-\kappa_k)_+^p}_{\text{nonlinear term}}$
- We can apply group lasso

$$(\boldsymbol{Y} - \alpha\mathbf{1}_n - \mathbf{X}_1\boldsymbol{\beta}_1 - \ldots - \mathbf{X}_s\boldsymbol{\beta}_s)^\top(\boldsymbol{Y} - \alpha\mathbf{1}_n - \mathbf{X}_1\boldsymbol{\beta}_1 - \ldots - \mathbf{X}_s\boldsymbol{\beta}_s) + \lambda\sum_{j=1}^{s}\|\widetilde{\boldsymbol{\beta}}_j\|,$$

where

$$\begin{aligned}
\boldsymbol{\beta}_j &= (\beta_{j,1}, \ldots, \beta_{j,p}, b_{j,1}, \ldots, b_{j,K})^T \\
\widetilde{\boldsymbol{\beta}}_j &= (\beta_{j,2}, \ldots, \beta_{j,p}, b_{j,1}, \ldots, b_{j,K})^T \\
\mathbf{X}_j &= \begin{pmatrix} x_{j,1} & \cdots & (x_{j,1}-\kappa_K)_+^p \\ x_{j,2} & \cdots & (x_{j,2}-\kappa_K)_+^p \\ \vdots & \vdots & \vdots \\ x_{j,n} & \cdots & (x_{j,n}-\kappa_K)_+^p \end{pmatrix}
\end{aligned}$$

Environmental Kuznets Curve

Consider following Environmental Kuznets Curve:

$$y_{it} = f_1(e_{it}) + f_2(\mathrm{gdp}_{it}) + f_3(\mathrm{trade}_{it}) + \alpha_i + error,$$

- y: Log CO_2 emission per capita
- e: Log energy consumption per capita
- gdp: Log GDP per capita
- trade: Log trade per capita
- N=89, T=40

Environmental Kuznets Curve

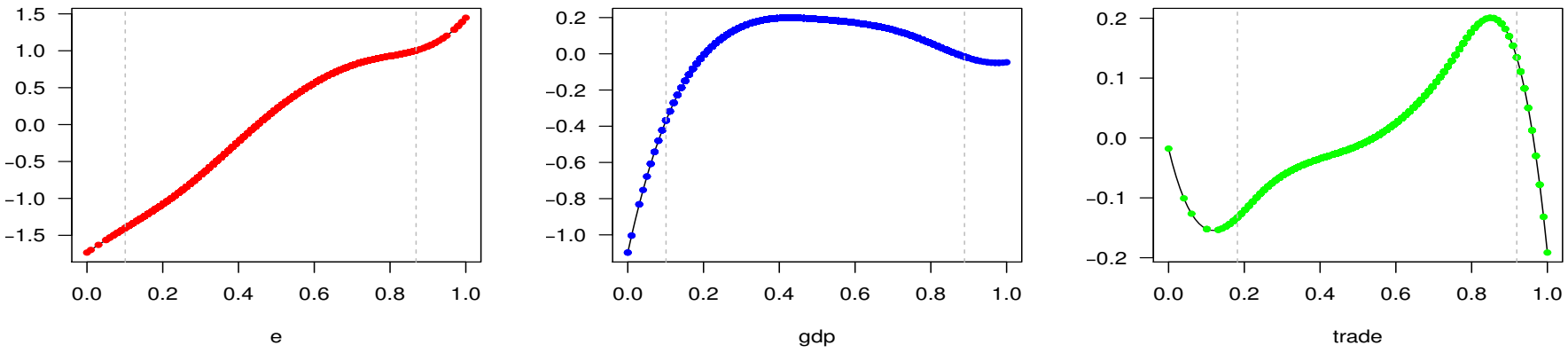
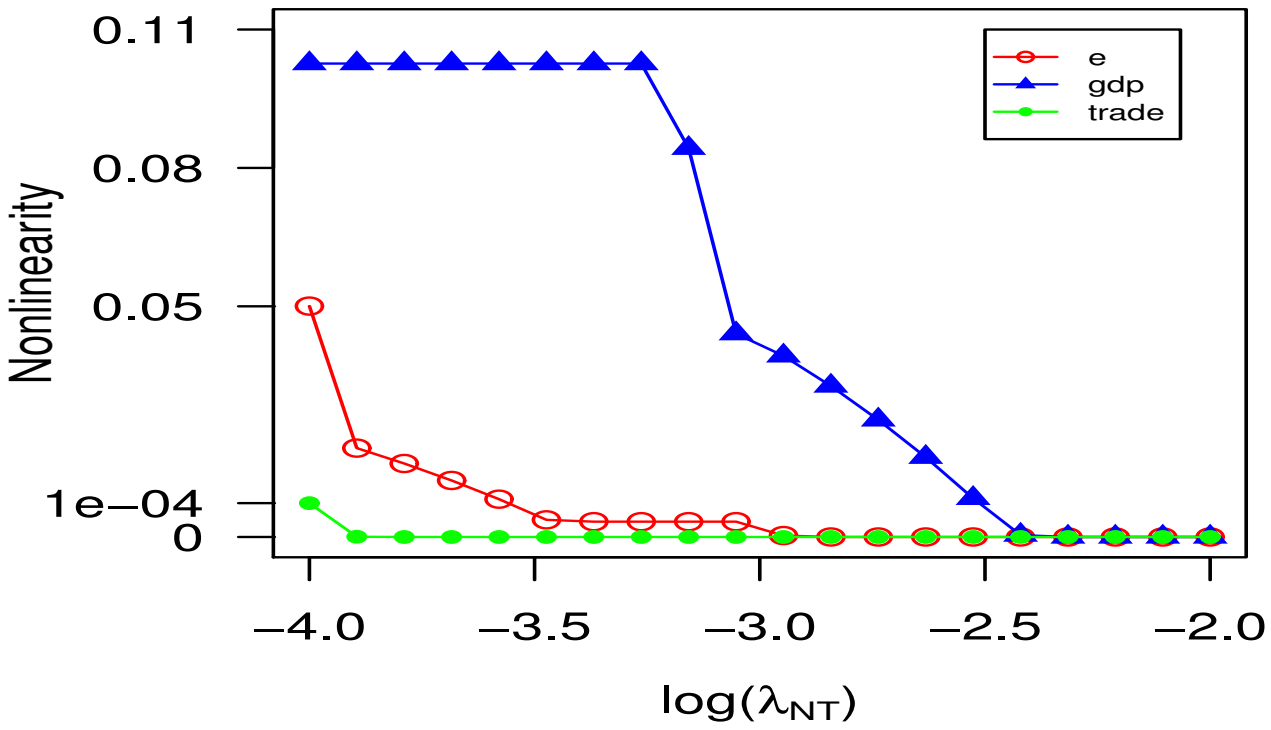


Figure: Estimated curves of f_j's for non-penalized estimators. Two vertical dashed lines indicate the 2.5% to 97.5% pencentile range of each regressor.

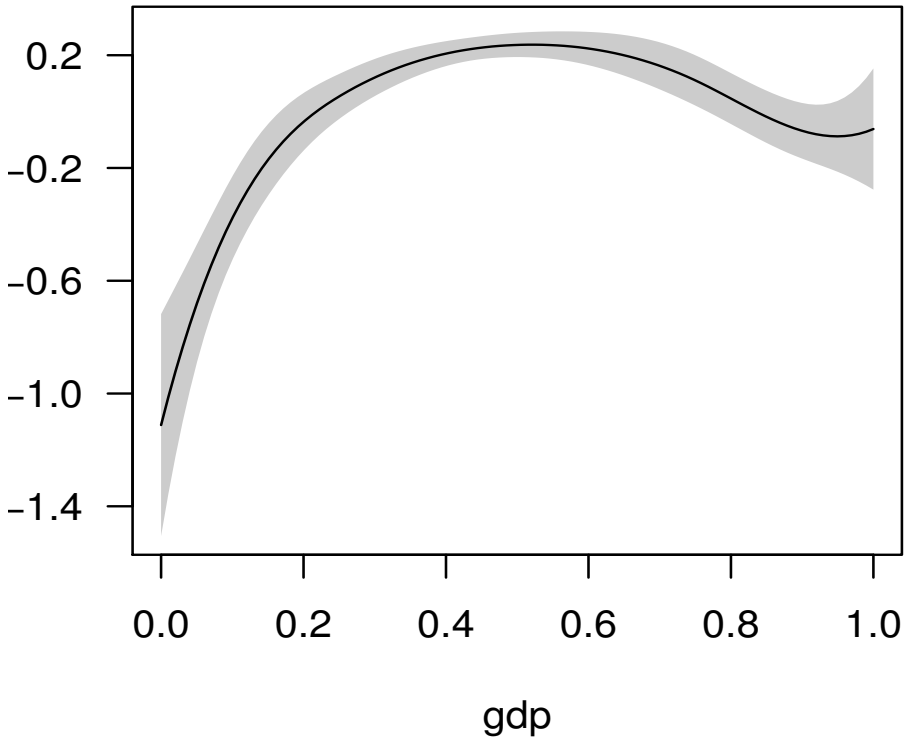
Environmental Kuznets Curve



	Model 1	Model 2	Model 3	Model 4
Linearity	$\emptyset$	trade	trade,e	trade,e,gdp
CV	0.0541	0.0523	0.0489*	0.0563

Environmental Kuznets Curve

	e	trade
Coef	3.452***	0.491***
SE	0.303	0.123



- The inverted U-shape: As a country develops, it shows a increase in metals use followed by a decline owing to regulatory policy.
- Existing literature uses quadratic term (gdp^2).

Environmental Kuznets Curve

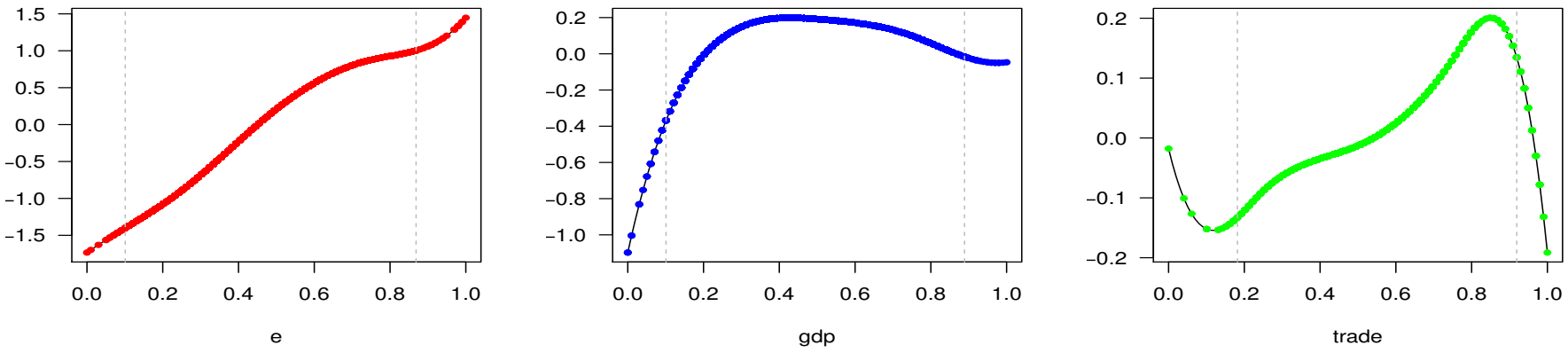
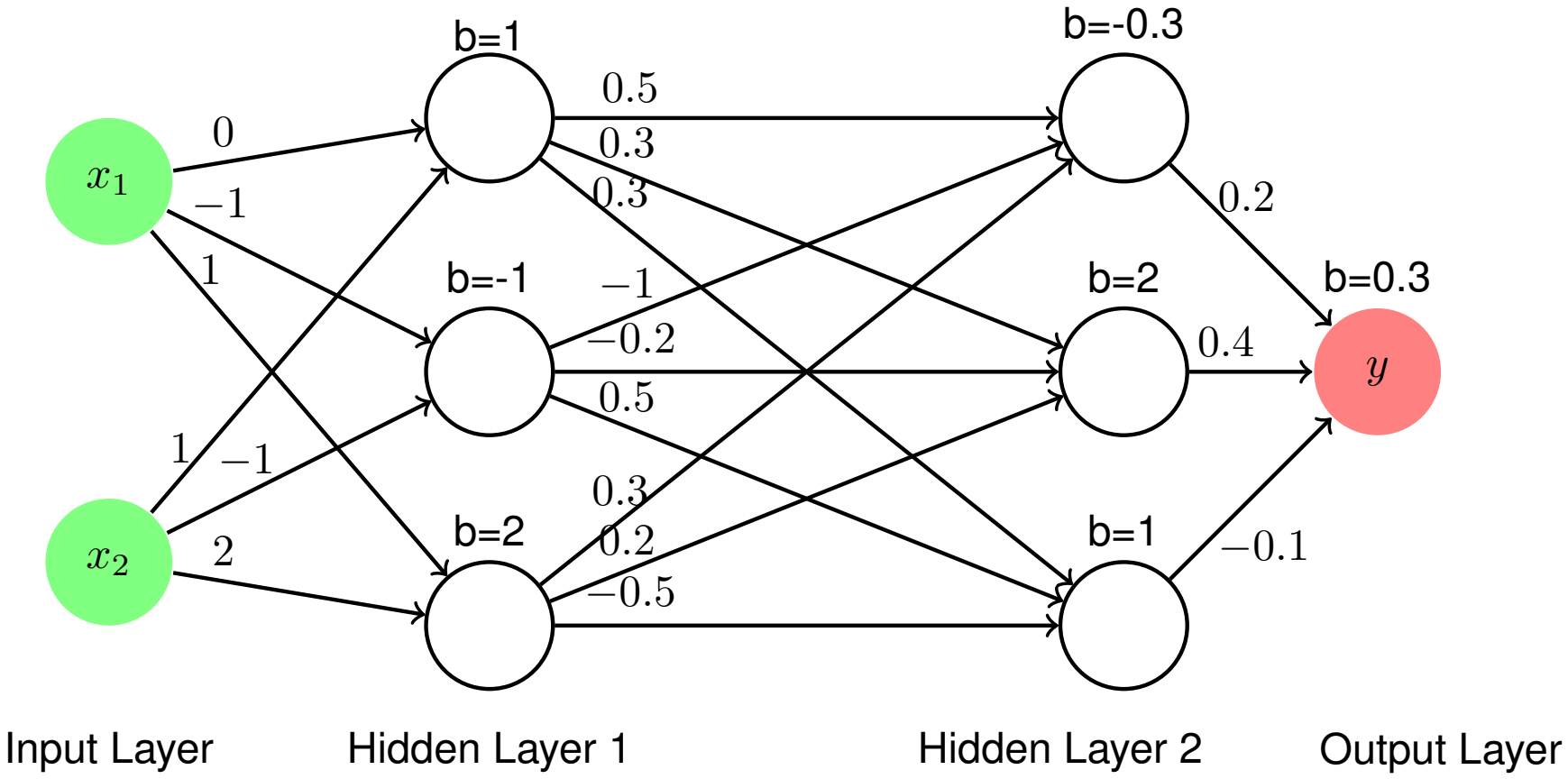


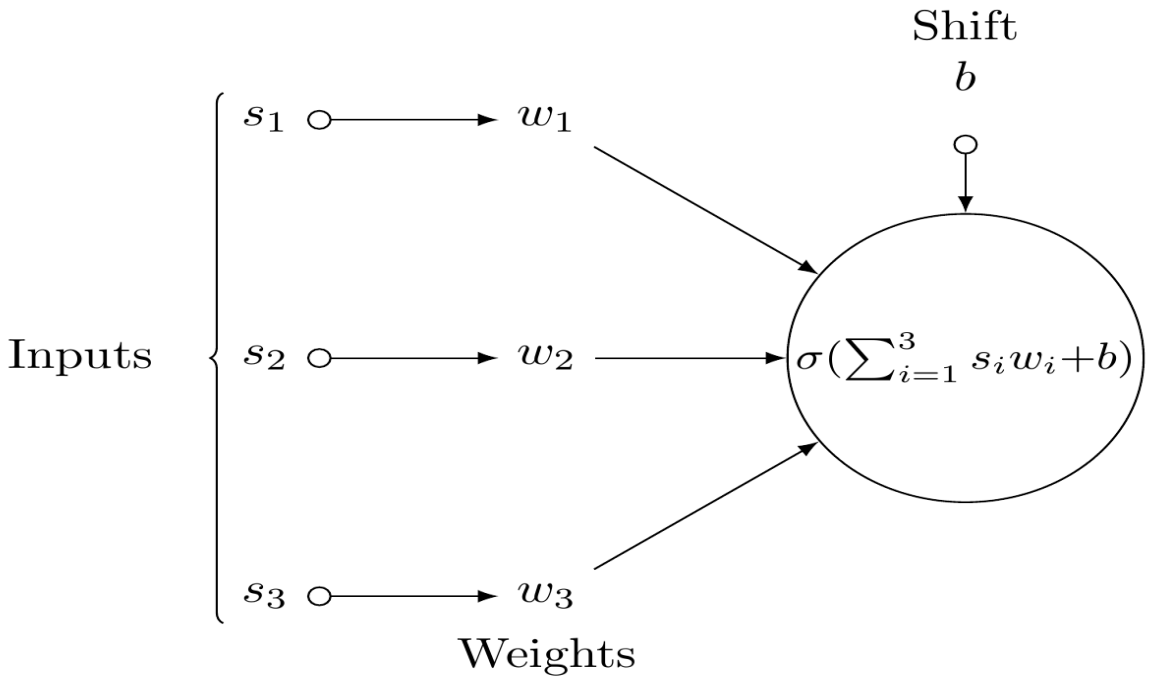
Figure: Estimated curves of f_j's for non-penalized estimators. Two vertical dashed lines indicate the 2.5% to 97.5% pencentile range of each regressor.

Deep Neural Network – Overview

- Fully connected deep neural network (Multilayer perceptron, MLP)
- Use x_1, x_2 to predict $y = f(x_1, x_2)$.



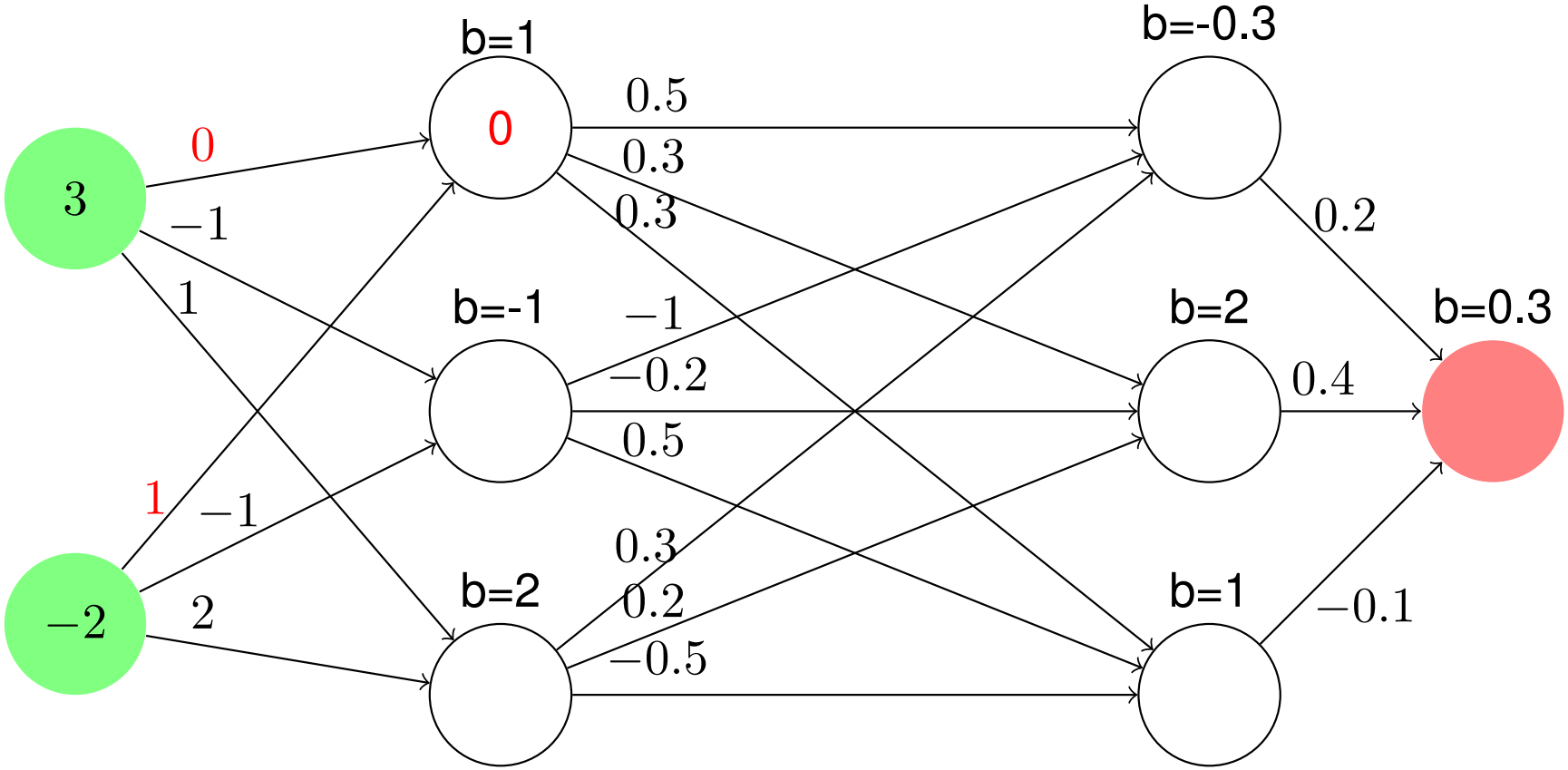
Deep Neural Network – Overview



- Activation function $\sigma(x) = \begin{cases} \frac{1}{1+e^{-x}} & \text{Sigmoid} \\ \tanh^{-1}(x) & \text{Hyperbolic Tangent} \\ \max(x, 0) & \text{Rectifier Linear Unit (ReLU)} \end{cases}$
- Shift: called bias in Pytorch

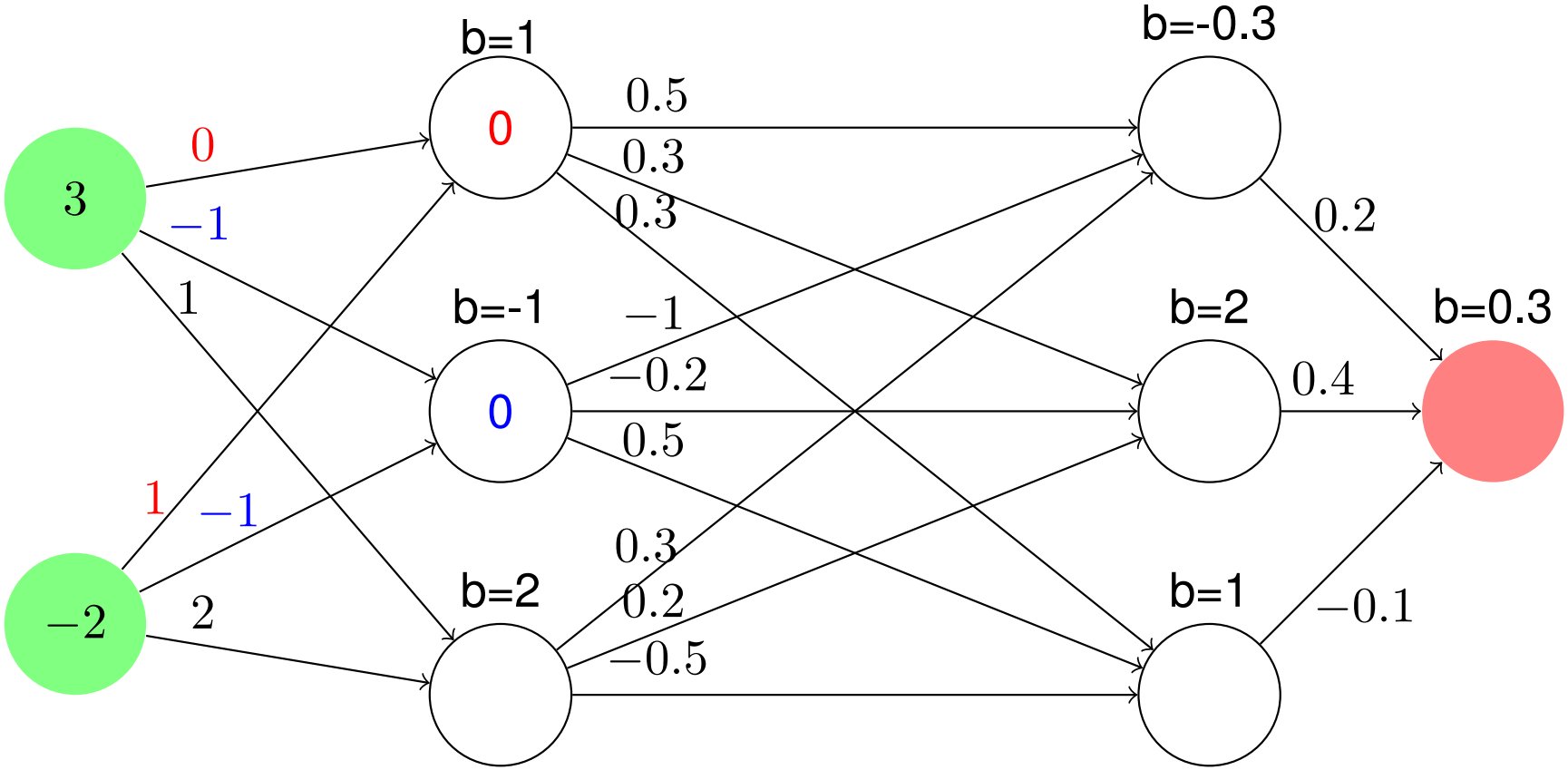
Deep Neural Network – Operation of ReLU

- $\sigma(3 \times 0 + (-2) \times 1 + 1) = \sigma(-1) = 0$



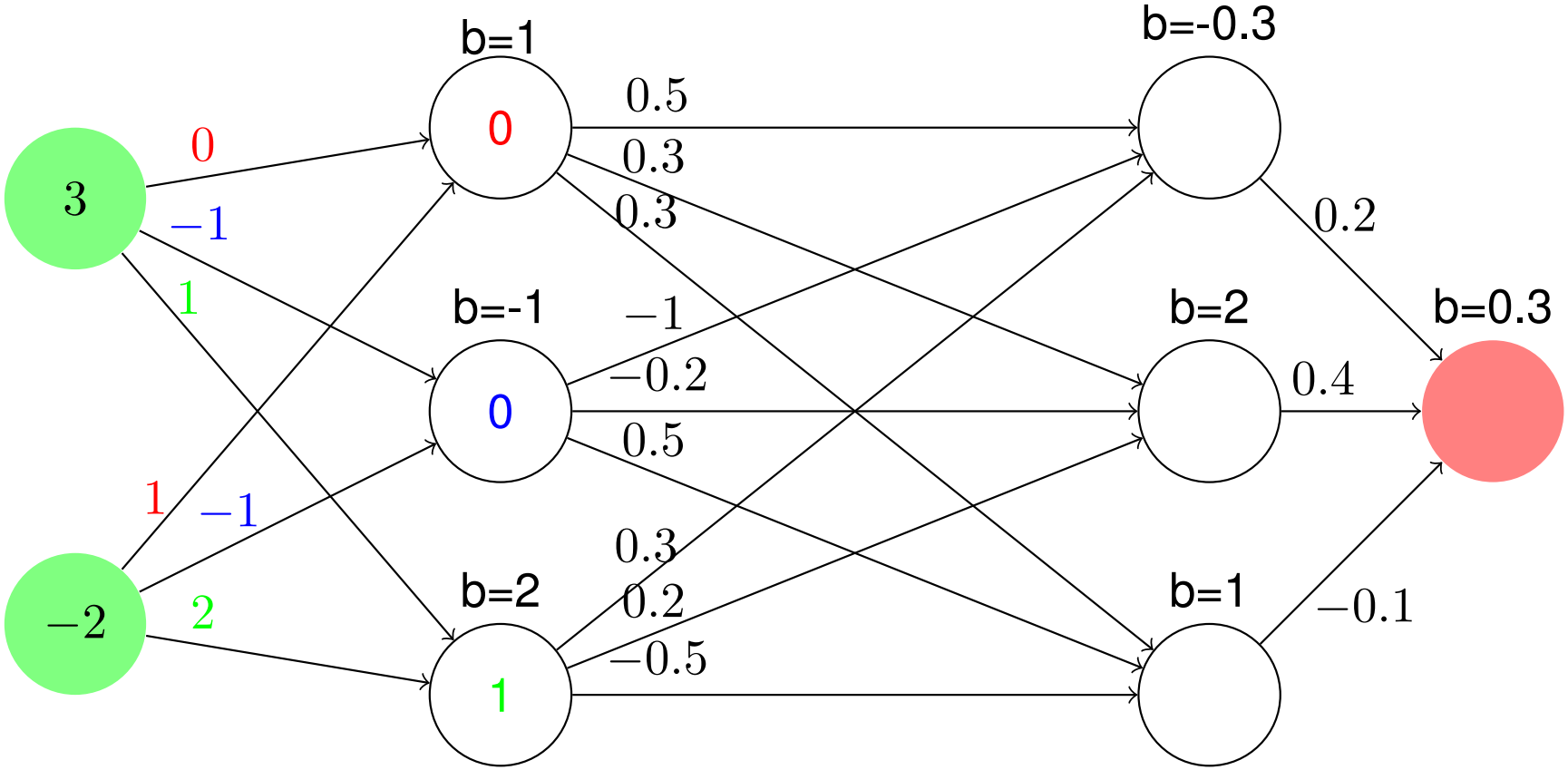
Deep Neural Network – Operation of ReLU

- $\sigma(3 \times (-1) + (-2) \times (-1) - 1) = \sigma(-2) = 0$



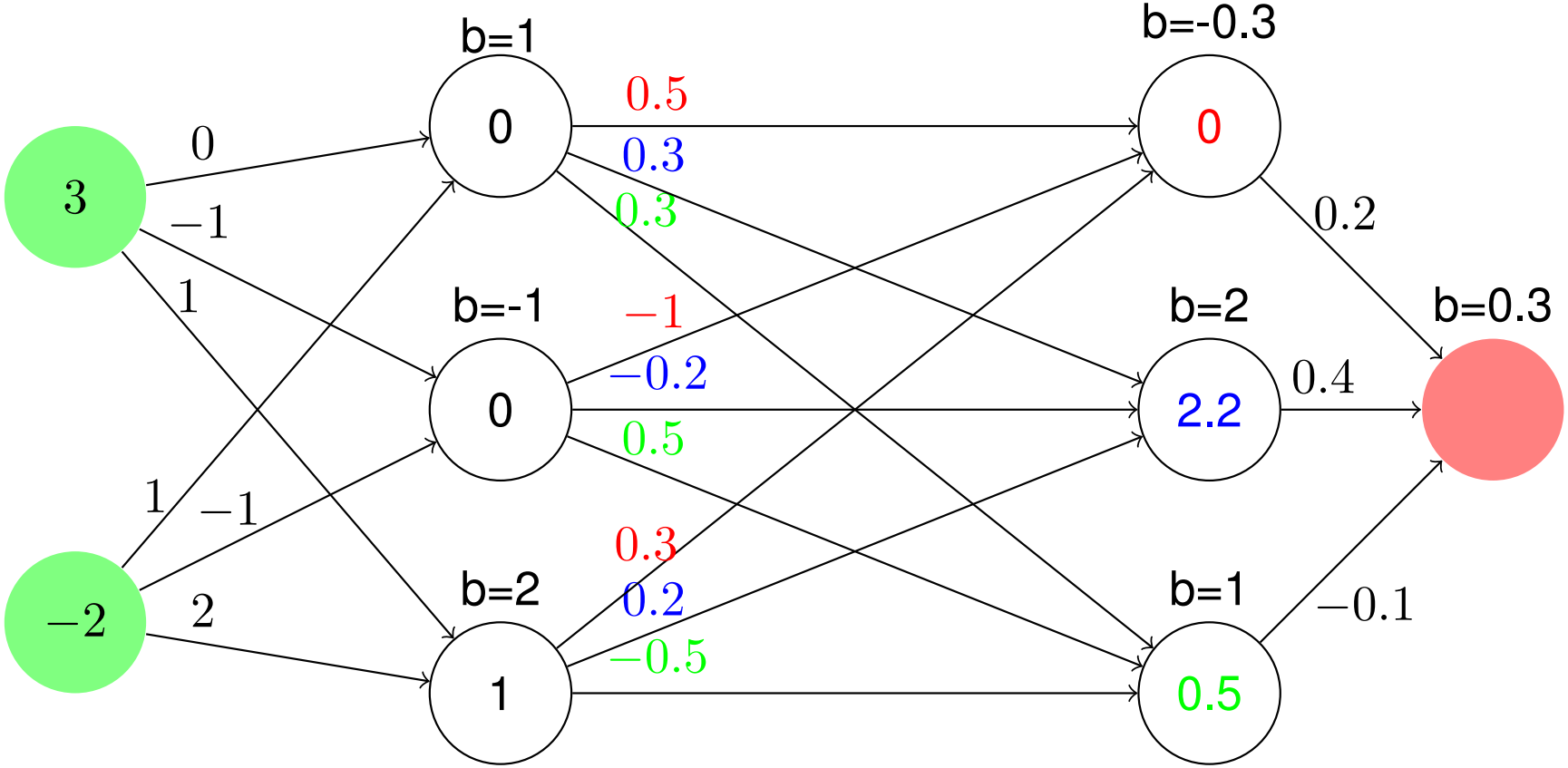
Deep Neural Network – Operation of ReLU

- $\sigma(3 \times 1 + (-2) \times 2 + 2) = \sigma(1) = 1$



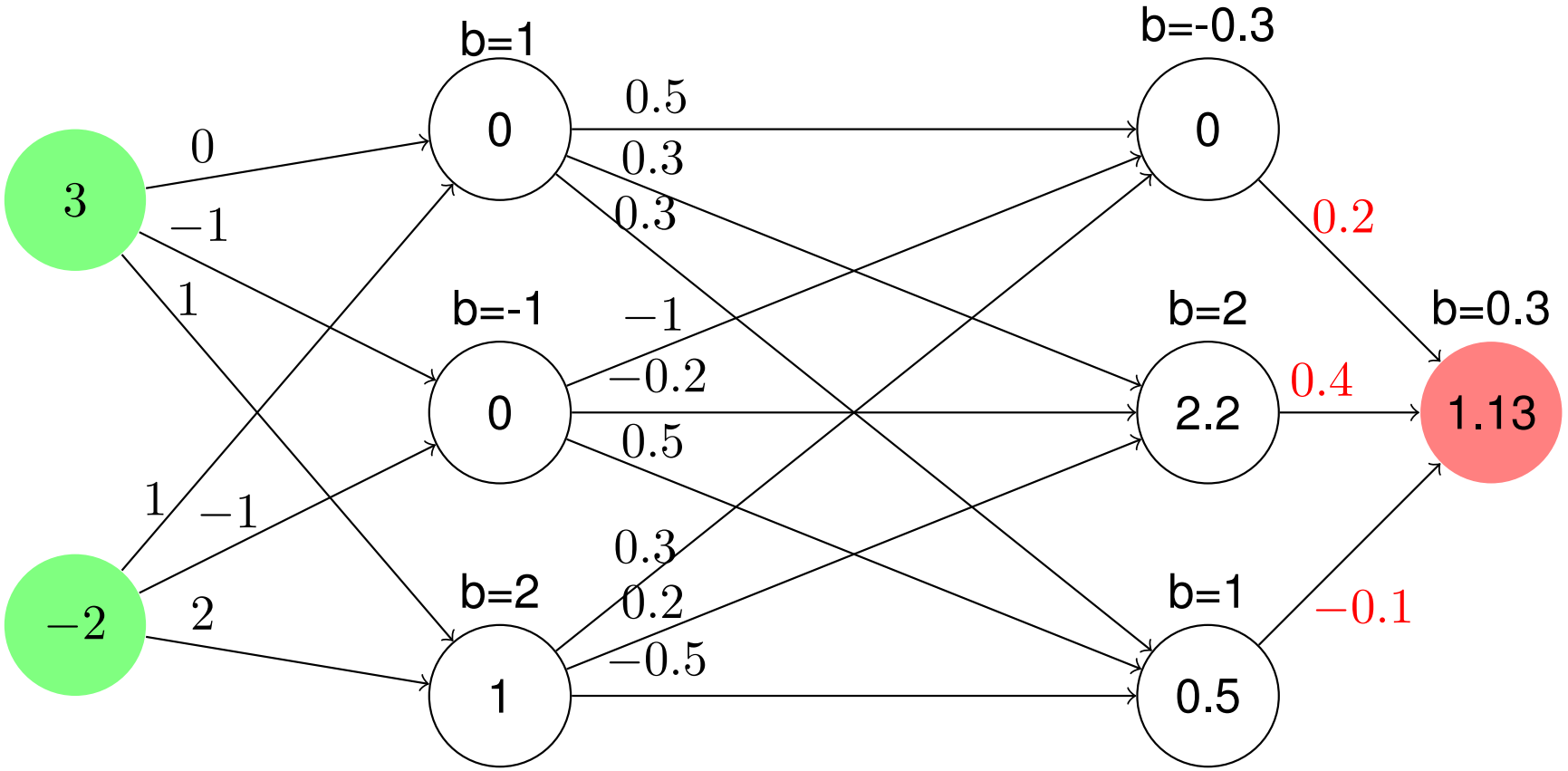
Deep Neural Network – Operation of ReLU

- Forward to next layer



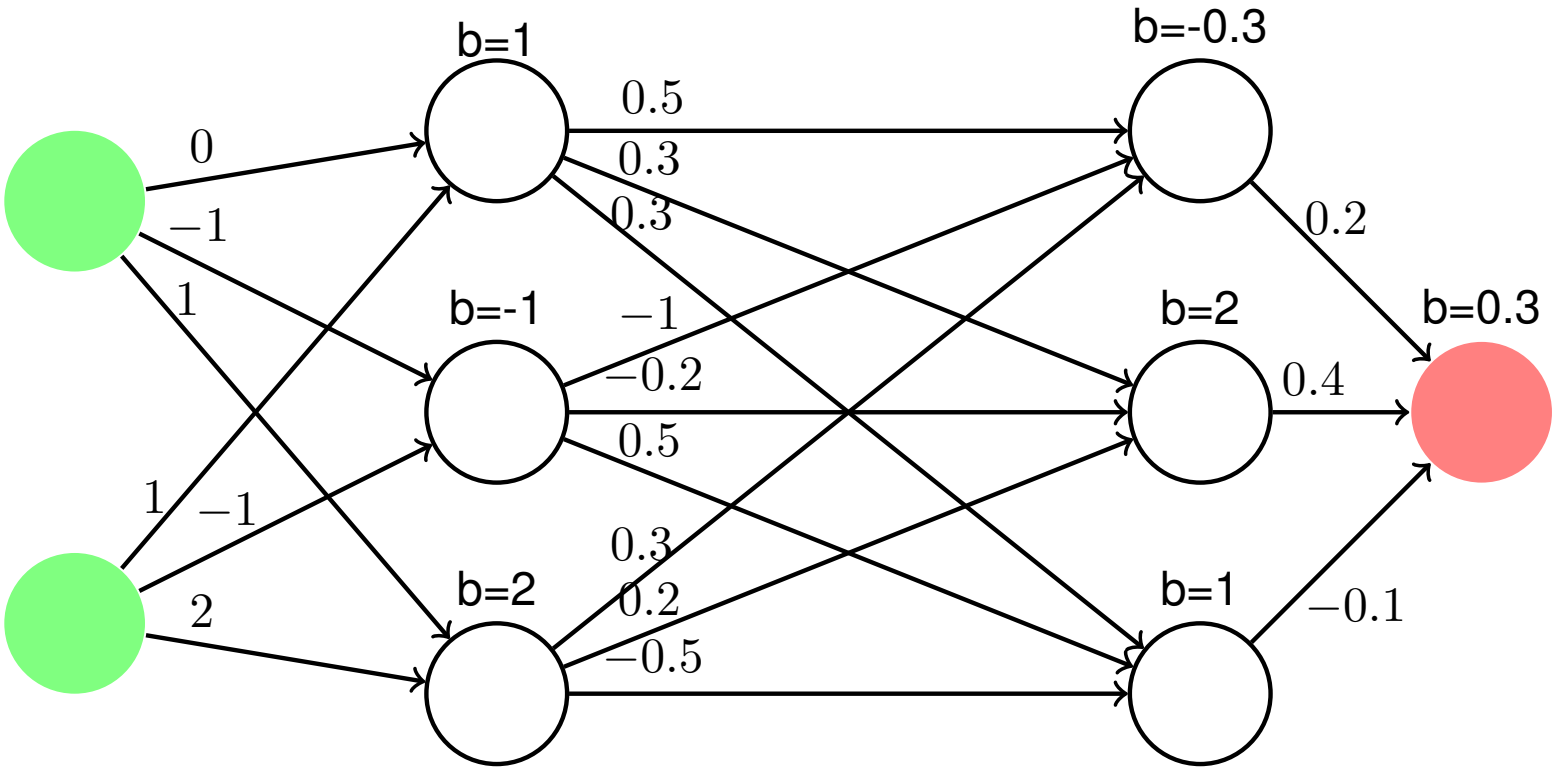
Deep Neural Network – Operation of ReLU

- Forward to next layer



- For neural network $f : \mathbb{R}^2 \to \mathbb{R}$ defined above, $f(3, -2) = 1.13$.

Deep Learning – Example



- Deep neural network (25 parameters):
  - $6 + 9 + 3 = 18$ weights
  - $3 + 3 + 1 = 7$ shifts
- Deep Learning: estimate (learn) these parameters from data.

Deep Learning – Loss Function

- Let $f_\theta(\boldsymbol{x}) = f_\theta(x_1, \ldots, x_p)$ be DNN with parameters θ.
- Observations: $\{(y_i, \boldsymbol{x}_i)\}_{i=1}^n$, $\boldsymbol{x}_i = (x_{i1}, \ldots, x_{ip})$.
- Loss function:

$$\min_\theta \frac{1}{n} \sum_{i=1}^n (y_i - f_\theta(\boldsymbol{x}_i))^2$$

- Train: gradient descent, update all parameters in each loop

Deep Learning – Penalty

- Loss function+Penalty:

$$\min_{\theta} \frac{1}{n} \sum_{i=1}^{n} (y_i - f_\theta(\boldsymbol{x}_i))^2 + \lambda \|\theta\|_2^2$$

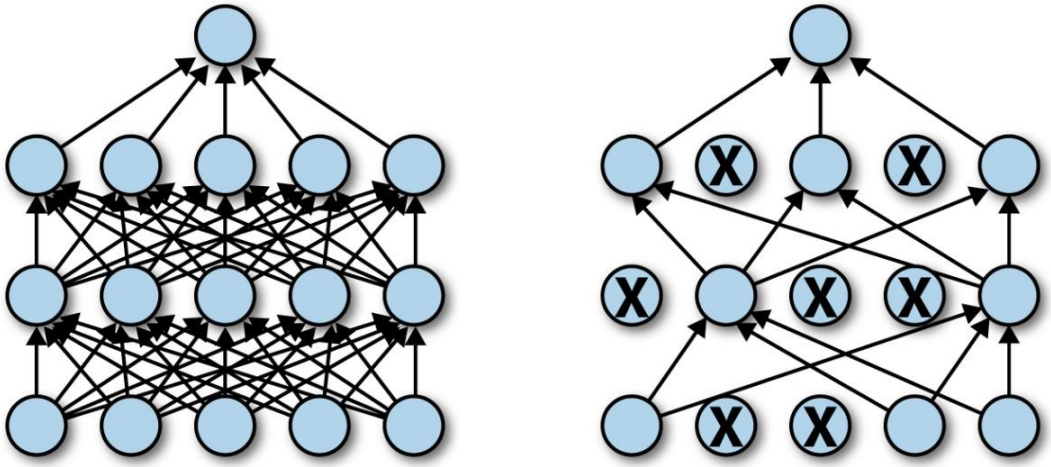
- Train: gradient descent, update all parameters in each loop

Deep Learning – Dropout

- Loss function:

$$\min_{\theta} \frac{1}{n} \sum_{i=1}^{n} (y_i - f_\theta(\boldsymbol{x}_i))^2$$

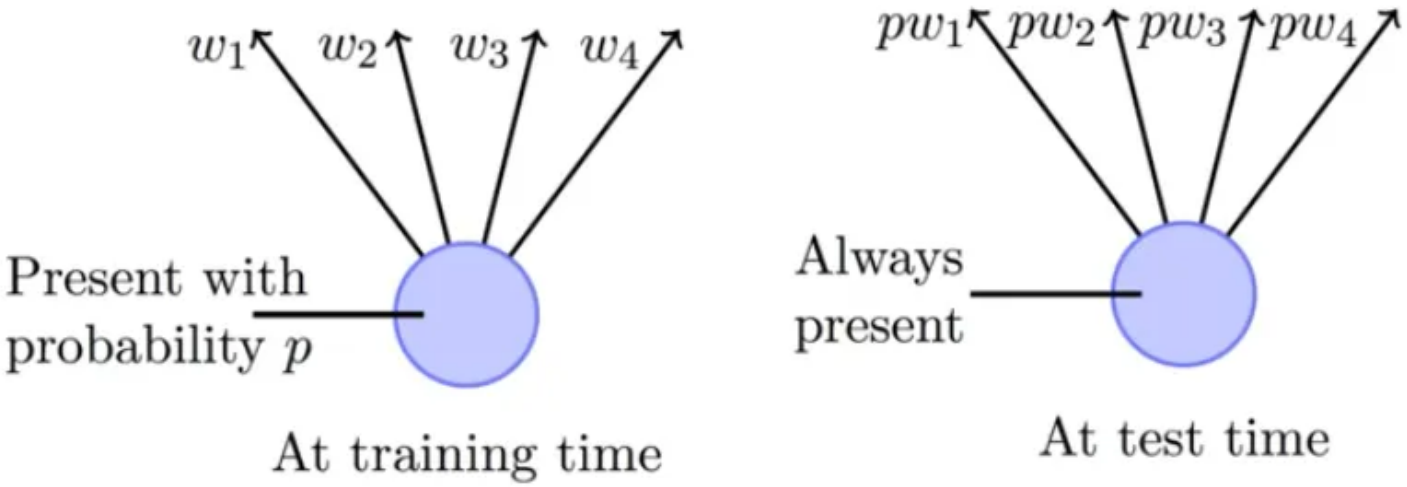
- Train: Drop some nodes and update remaining parameters in each loop



- How to drop some nodes

Deep Learning – Dropout

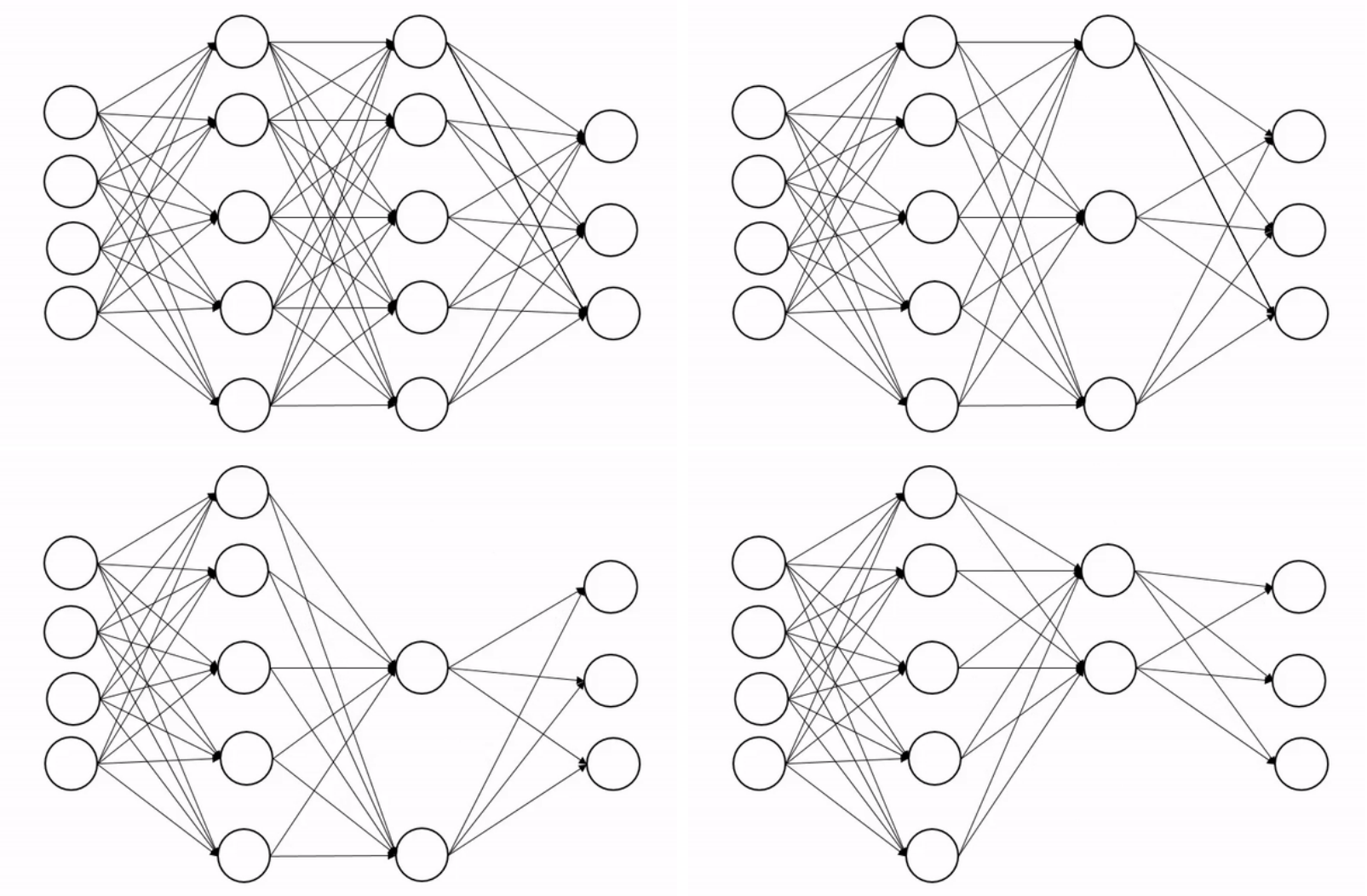
- Randomly set the output as 0:



- Pytorch: model.train(), model.eval();

Deep Learning – Dropout

- Dropout at second hidden layer:



Deep Learning – Coding

- See instruction in class

Additive Model

- The “magical regression model" is not going to work well!
- A nice compromise is the “additive model":

$$Y = \beta_0 + f_1(X_1) + \cdots + f_p(X_p) + \varepsilon,$$

  where the f_j's are smooth arbitrary functions.
- Recall that sometimes, in linear models, we need to find the best transformation of predictors;
- The additive model: the best transformations are determined simultaneously by estimating $f_j(\cdot)$'s;

Partially Linear Additive Model

- The additive model will do poorly when strong interactions among X_j's exist;
- If there is sufficient data, we can add terms such as $f_{ij}(x_i, x_j)$ into the model;
- We can also consider the Partially linear additive model:

$$Y = \beta_0 + f_1(X_1) + \cdots + f_p(X_p) + \boldsymbol{Z}\boldsymbol{\gamma} + \boldsymbol{\varepsilon},$$

- $\boldsymbol{Z}$ is the design matrix for the variables that will not be modeled additively, where some may be quantitative and others qualitative.
- These all can be done using function *gam(·)* in R package “mgcv";
- Being able to use this R-package will significantly promote your level of “statistical analysis"!

Additive Model

- Now suppose that two explanatory variables, X and Z;
- Assume a simple additive model structure

$$y_i = \beta_0 + f_1(x_i) + f_2(z_i) + \varepsilon_i, \quad \varepsilon_i \sim N(0, \sigma^2);$$

- For simplicity, assume that all z_i and x_i lie in $[0, 1]$.
- Identifiability: a pair of functions $\{f_1(x), f_2(z)\}$ give the same model as $\{f_1(x) - c, f_2(z) + c\}$;
- Usually we assume that $Ef_1(x) = Ef_2(z) = 0$;
- This is the reason why the default setting in bs(), ns() is intercept=FALSE.
- Provided the identifiability issue is addressed, the additive model can be
  - represented using regression splines or smoothing splines;
  - estimated by penalized least squares;
  - the smoothing parameter can be estimated by GCV/CV;

  in the same way as the simple univariate model.

Some R Packages

- There are many R packages that can fit the additive model
- “mgcv" package: Wood(2000);
  - Function “gam()" to fit the model;
  - “gam()" works the same as the “glm()" or “lm()" functions.
  - The main difference is that the “gam()" model formula can include smooth terms, “s()" and “te()";
  - Options available for controlling automatic smoothness selection (choose λ_j's);
- “gss" package: Gu(2000);
  - I did not use much of this..
  - It can perform function ANOVA test (i.e. smaller model v.s. bigger model.)
- Other packages: “assist", “gamlss", “vgam", etc.
- all available at cran.r-project.org, you can check out.

Ozone Data

- The “ozone" data is available in R package “faraway";
- A study the relationship between atmospheric ozone concentration and meteorology in the Los Angeles Basin in 1976;
- Response variable: O3, Ozone concentration, (unit:ppm), at Sandbug AFB; 330 observations;
- Nine potential covariates:
  - vh:the altitude at which the pressure is 500 millibars;
  - wind: the wind speed (mph);
  - temp:temperature;
  - hum: the humidity (%)
  - ibh: the temperature inversion base height (feet);
  - dpg: the pressure gradient (mm Hg);
  - ibt: the inversion base temperature (degrees F);
  - vis: the visibility (miles);
  - doy: the day of the year;

Ozone Data

- Our first attempt is to fit a linear model:

```
alm <- lm(O3~.,data=ozone)
> sumary(alm)
Estimate  Std. Error t value  Pr(>|t|)
(Intercept) 20.31357546 29.51930673  0.6881  0.491860
vh          -0.00542713  0.00539854 -1.0053  0.315514
wind        -0.05458319  0.13484250 -0.4048  0.685901
humidity     0.08097411  0.01883942  4.2981 2.289e-05
temp         0.27554918  0.04979118  5.5341 6.519e-08
ibh         -0.00023383  0.00029556 -0.7912  0.429441
dpg         -0.00336287  0.01128052 -0.2981  0.765810
ibt          0.02964114  0.01360881  2.1781  0.030130
vis         -0.00799098  0.00375033 -2.1307  0.033873
doy         -0.00911943  0.00277450 -3.2869  0.001126
n = 330, p = 10, Residual SE = 4.44068, R-Squared = 0.7
>round(vif(alm),2)
vh     wind humidity  temp   ibh   dpg      ibt    vis      doy
5.43   1.36     2.34  8.65  4.74  2.71    18.17   1.48     1.40
```

- Many coefficents insignificant!
- What is the next thing to do?

Ozone Data

- Looks like there is serious multicolinearity!
- remove “ibt"

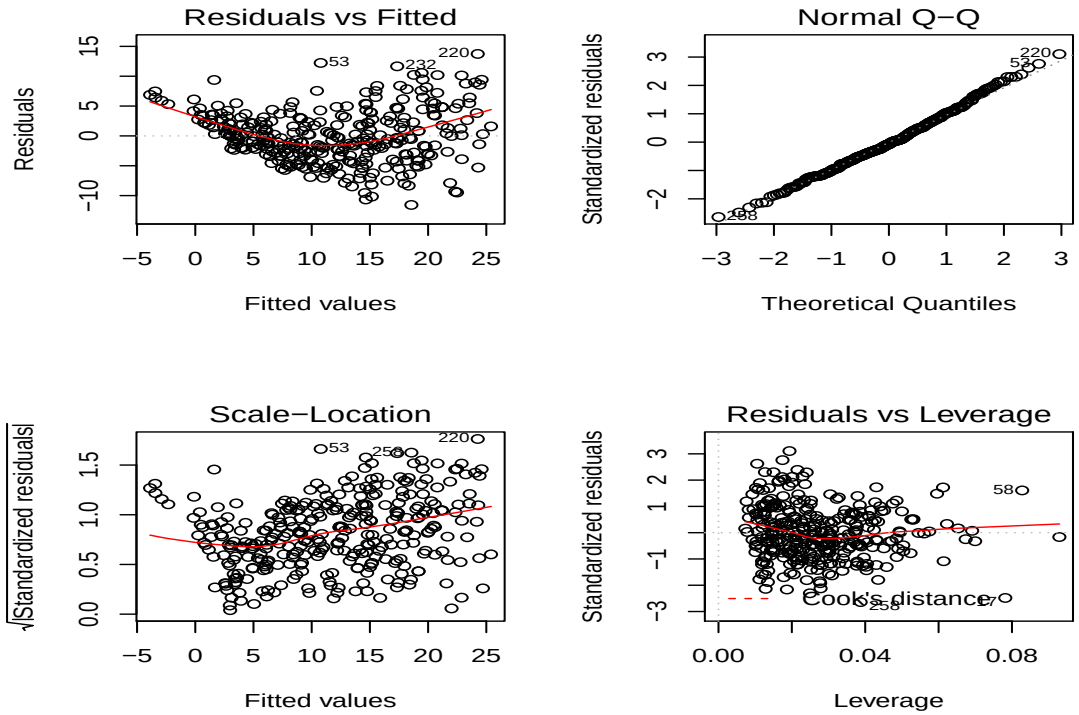
```
alm <- lm(O3~.-ibt,data=ozone)
> sumary(alm)
Estimate  Std. Error t value  Pr(>|t|)
(Intercept) -8.26321236 26.59712656 -0.3107 0.7562450
vh          -0.00022798  0.00487030 -0.0468 0.9626933
wind        -0.02028897  0.13469883 -0.1506 0.8803664
humidity     0.08264442  0.01893327  4.3650 1.716e-05
temp         0.34966485  0.03656055  9.5640 < 2.2e-16
ibh         -0.00073498  0.00018659 -3.9390 0.0001004
dpg         -0.01301875  0.01043321 -1.2478 0.2130073
vis         -0.00829779  0.00376948 -2.2013 0.0284255
doy         -0.00858876  0.00277985 -3.0896 0.0021792
n = 330, p = 9, Residual SE = 4.46650, R-Squared = 0.7
>round(vif(alm),2)
vh     wind humidity   temp   ibh      dpg      vis      doy
4.37   1.34     2.33   4.61   1.87     2.29     1.48     1.39
```

- Looks better; no severe multicolinearity;
- What is the next thing to do?

Ozone Data

- Perform model diagnostic

```
par(mfrow=c(2,2))
plot(alm)
```

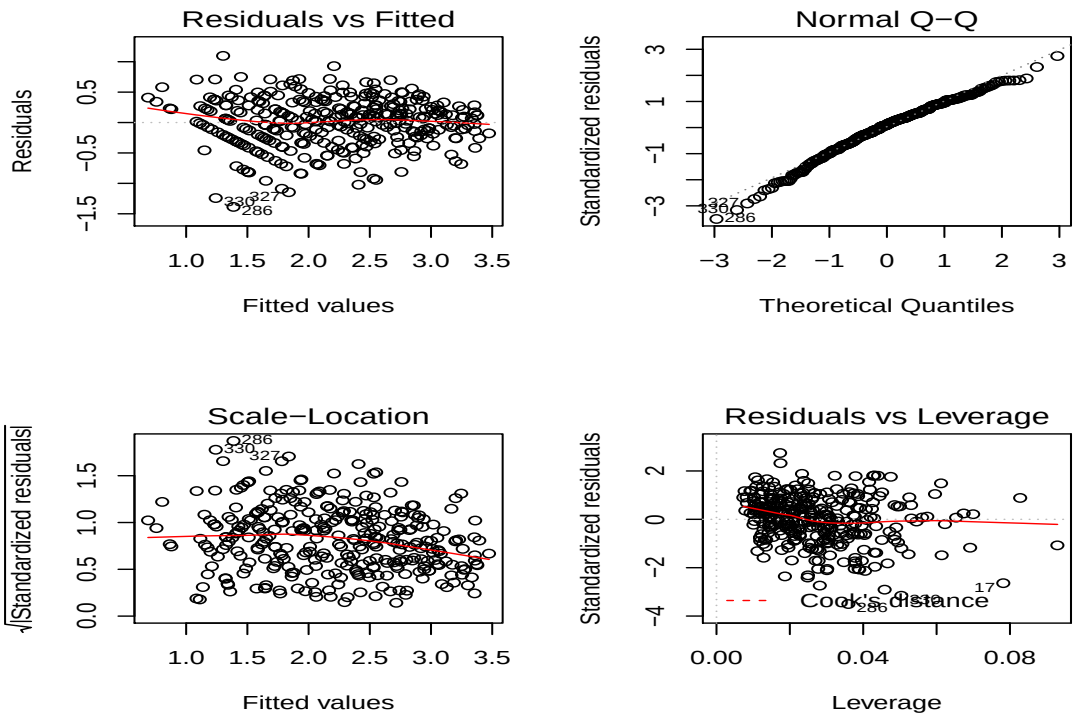


- Potential problem: nonlinear relationship/non-constant variance;
- Normality and outlier/leverage plot look ok;

Ozone Data

- Take a log-transformation of the response;
- Fit the linear regression model again;

```
logalm <- lm(log(O3)~.-ibt,data=ozone)
par(mfrow=c(2,2))
plot(alm)
```



- Looks better than before;
- You can do better than this: partial residual plot, partial regression plot to make transformations of covariate; (but I will stop here)

Ozone Data

- Now we can perform model selection on the linear model

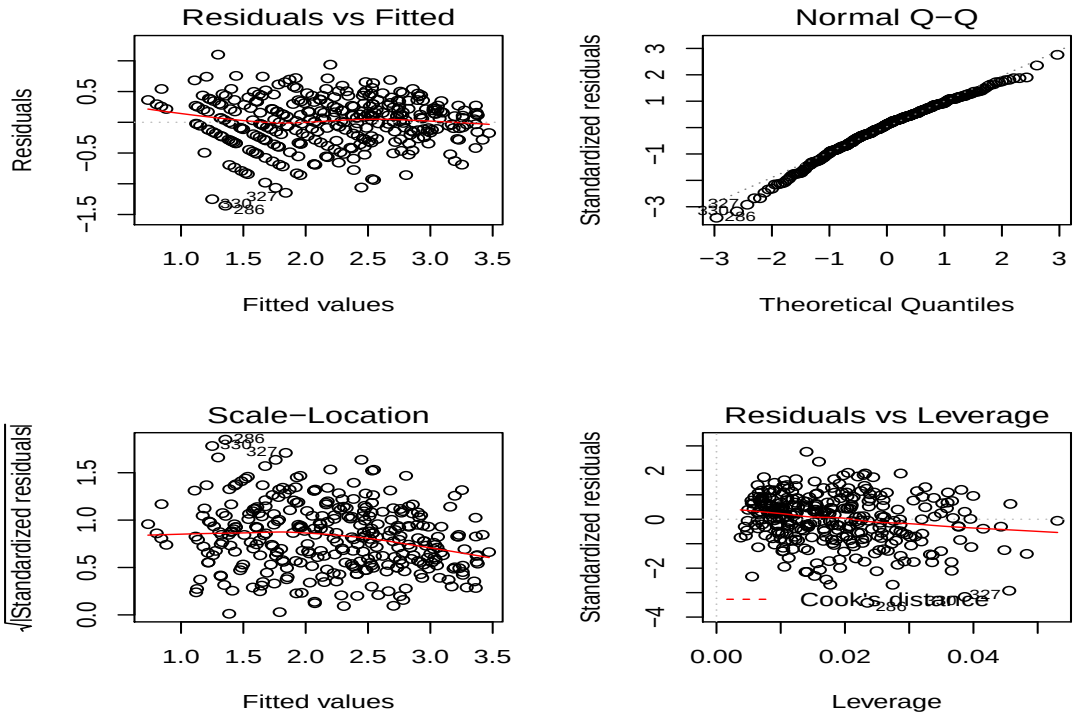
```
logalm <- lm(log(O3)~.-ibt,data=ozone)
obj<-stepwise(logalm,
direction = c("forward/backward"),
criterion = c("BIC"))
#The best model
Step:  AIC=-573.25
log(O3) ~ temp + ibh + humidity + doy + vis
```

- Now fit the final linear model

```
library(Rcmdr)
logalm <- lm(log(O3)~temp+ibh+humidity+doy+vis,data=ozone)
sumary(logalm)
> sumary(logalm)
Estimate  Std. Error t value  Pr(>|t|)
(Intercept)  5.2008e-01  1.6213e-01  3.2078  0.001471
temp         3.1516e-02  1.9756e-03 15.9521 < 2.2e-16
ibh         -9.4699e-05  1.5393e-05 -6.1522 2.252e-09
humidity     5.2382e-03  1.2528e-03  4.1811 3.738e-05
doy         -9.8361e-04  2.2883e-04 -4.2984 2.279e-05
vis         -8.6892e-04  3.3420e-04 -2.6000  0.009750
n = 330, p = 6, Residual SE = 0.40168, R-Squared = 0.72
```

Ozone Data

- The diagnostic plots of the final linear model



- Looks ok...;
- Final model $R^2 = 0.72$ with $p = 6$ covariates.

Ozone Data

- Remove “ibt" to avoid multicollinearity (need check this for additive model);
- Each smooth function can be represented by $s(\cdot, k = 10)$;
- k specifies number of basis functions used for $s()$, default is $k = 10$;
  - Even though the default setting says $k = -1$, the default number of basis functions is depending on another parameter “bs=?".
  - When bs= “tp" (the thin plate spline, a two-dimensional natural cubic spline), $k = -1$ use 10 basis functions.
  - It automatically removes the intercept, so the number of basis functions is 9.
-
```
library(mgcv)
amred <- gam(log(O3)~s(vh,k=10)+s(wind)+s(humidity)
+s(temp)+s(dpg)+s(ibh)+s(vis)+s(doy),data=ozone)
> amred
Formula:
log(O3) ~ s(vh) + s(wind) + s(humidity) + s(temp) + s(dpg) +
s(ibh) + s(vis) + s(doy)
Estimated degrees of freedom:
1.00 1.00 2.38 3.92 3.33 2.70 2.73
4.56  total = 22.62
GCV score: 0.105328
```
- Total number of basis functions used $1 + 9 \times 8 = 73$;
- After penalization, DF$= tr(\boldsymbol{A}(\boldsymbol{\lambda})) = 22.62$;

Ozone Data

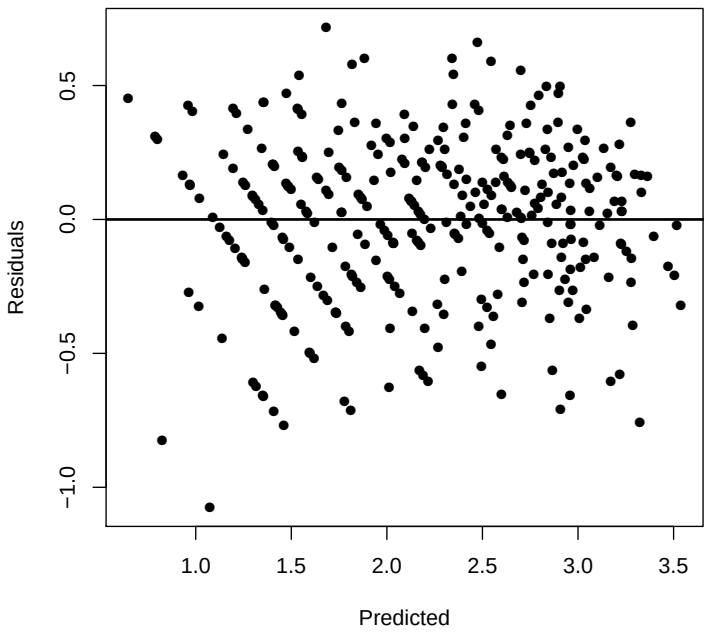
- Check out the fitted coefficients

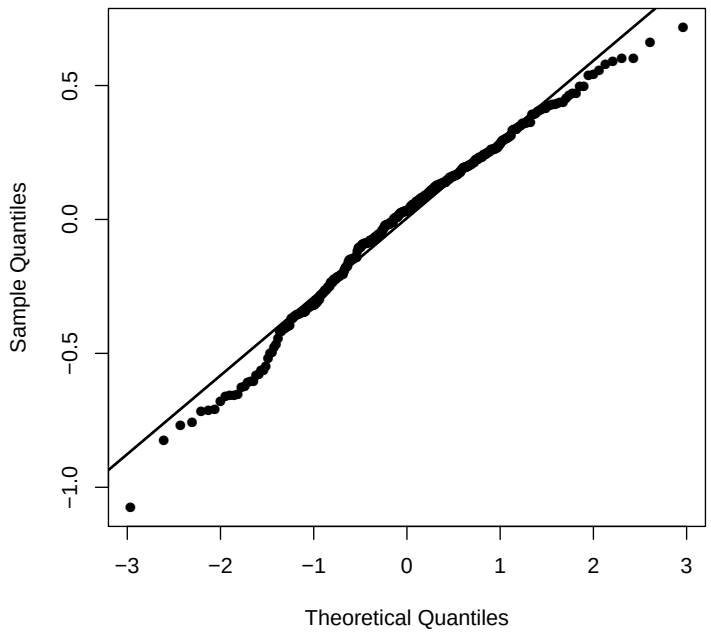
```
> summary(amred)
Parametric coefficients:
Estimate Std. Error t value Pr(>|t|)
(Intercept)  2.21297    0.01724   128.3   <2e-16 ***
Approximate significance of smooth terms:
edf   Ref.df    F  p-value
s(vh)        1.000  1.000 14.729  0.00015 ***
s(wind)      1.000  1.000  9.290  0.00250 **
s(humidity) 2.375  2.989  2.325  0.07504 .
s(temp)      3.922  4.886  6.669 8.54e-06 ***
s(dpg)       3.327  4.225 15.582 4.84e-12 ***
s(ibh)       2.704  3.306  7.948 2.43e-05 ***
s(vis)       2.727  3.409  9.959 1.11e-06 ***
s(doy)       4.562  5.682 26.072  < 2e-16 ***
R-sq.(adj) =  0.825   Deviance explained = 83.6%
GCV = 0.10533  Scale est. = 0.098109  n = 330
```

Ozone Data

- Now we can plot basic diagnostic plots as in the linear models;
- Make use of “predict()" and “residuals()" function as in the linear models;

```
plot(predict(amred), residuals(amred),xlab="Predicted",ylab="Residuals",pch=16)
abline(h=0,lwd=2)
qqnorm(residuals(amred), main="",pch=16)
qqline(residuals(amred),lwd=2)
```



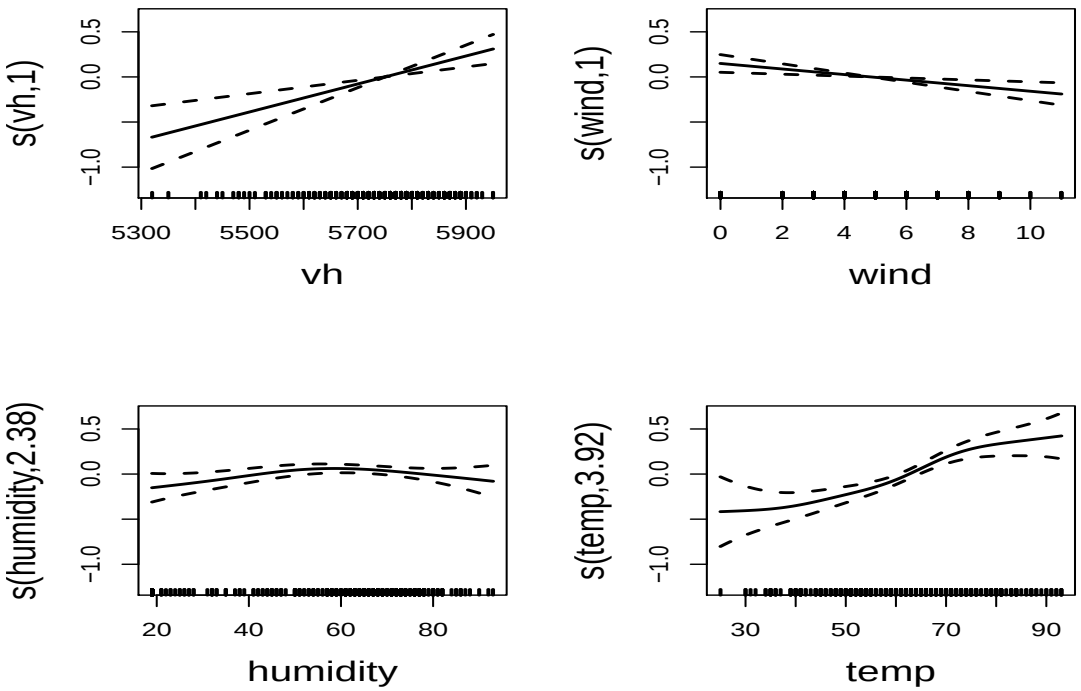


- Looks roughly ok...but with room to improve

Ozone Data

- Now we can visualize the fitting, first four functions

```
par(mfrow=c(2,2))
for(k in 1:4)
plot(amred,select=k,lwd=2,cex.lab=1.5)
```

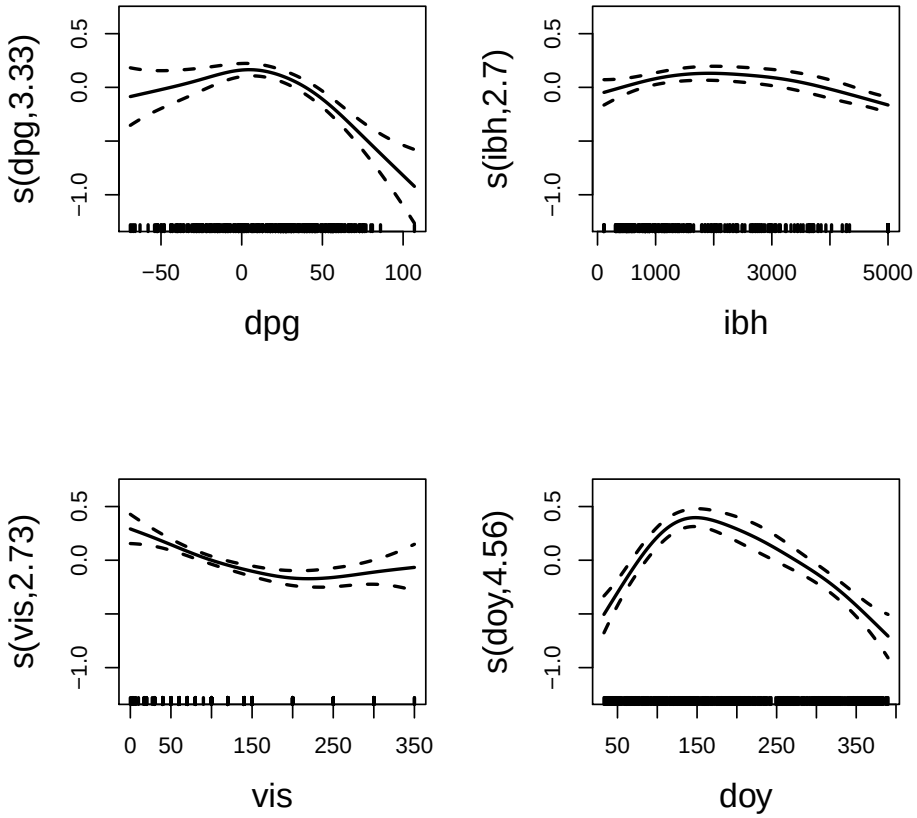


- Recall gam fit for “vh" and “wind", the edf=1, a linear relationship;
- “humidity" is not significant, in plot, the confidence band (dashed line) contains 0 uniformly;

Ozone Data

- Now we can visualize the fitting, last four functions

```
par(mfrow=c(2,2))
for(k in 5:8)
plot(amred,select=k,lwd=2,cex.lab=1.5)
```



Ozone Data

- From the previous fit, we want to refine the additive model a little bit
  - Manually forcing “vh" and “wind" to be linear functions;
  - Remove predictors “humidity" and “ibh" (you need to be more careful here in practice)
- So we now have a partial linear model with (a) 2 linear predictor and (b) 4 nonlinear components;

```
library(mgcv)
amred2 <- gam(log(O3)~vh+wind+s(temp)+s(dpg)
+s(vis)+s(doy),data=ozone)
> amred2
Formula:
log(O3) ~ vh + wind + s(temp) + s(dpg) + s(vis) + s(doy)
Estimated degrees of freedom:
5.66 4.25 4.50 4.31  total = 21.72
GCV score: 0.1141415
```

- After penalization, the effective degrees of freedom is $df = tr(\boldsymbol{A}(\boldsymbol{\lambda})) = 21.72$; (no big difference from before)

Ozone Data

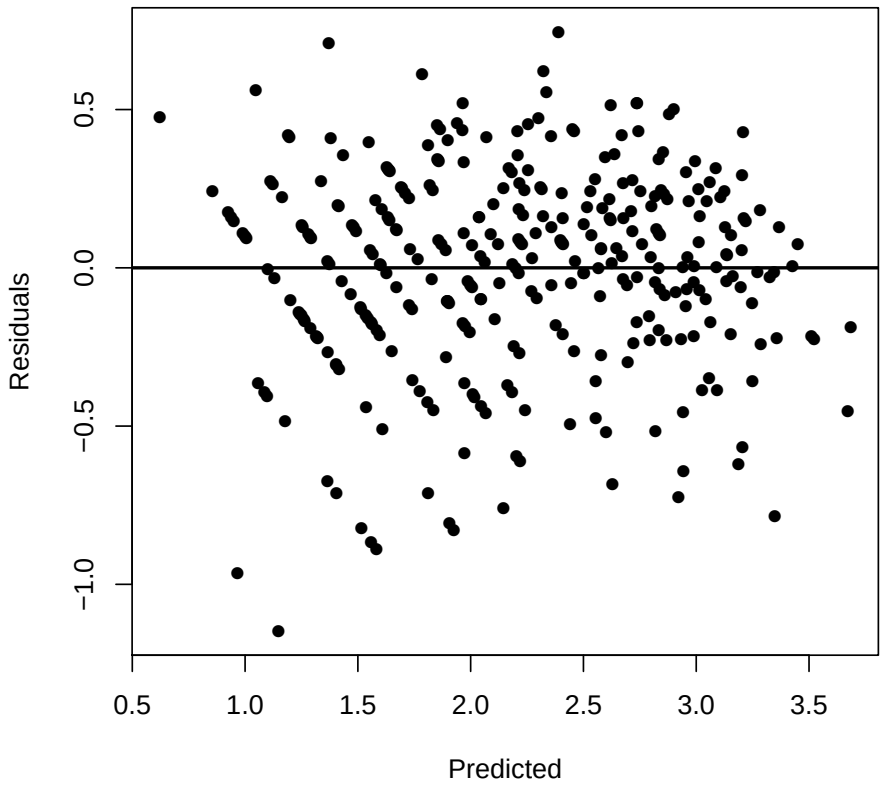
- Check out the fitted coefficients

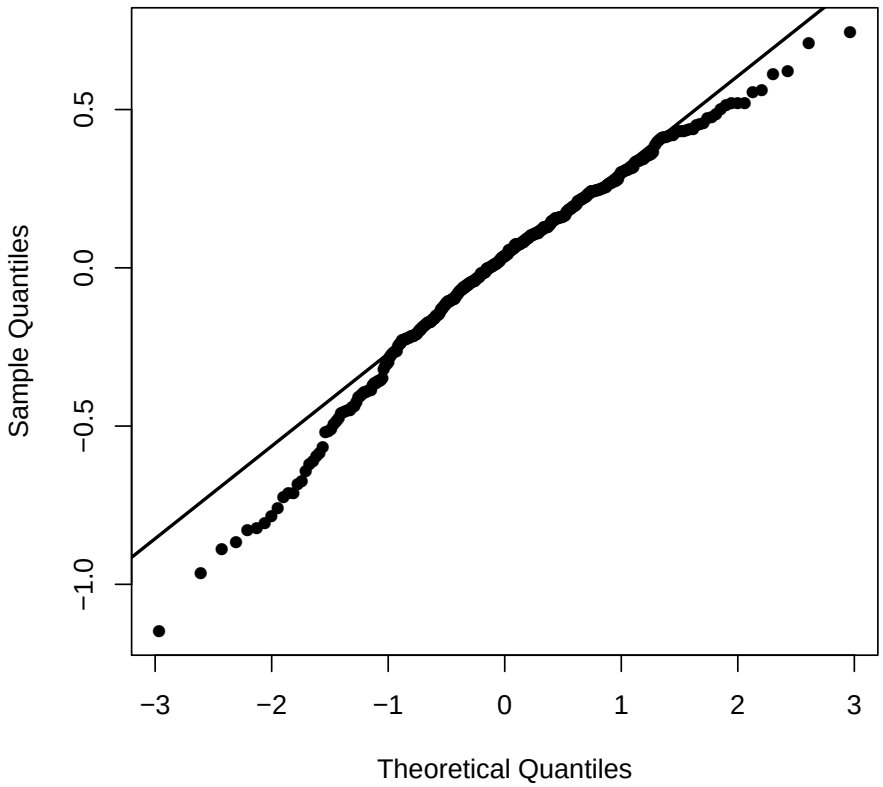
```
> summary(amred2)
Formula:
log(O3) ~ vh + wind + s(temp) + s(dpg) + s(vis) + s(doy)
Parametric coefficients:
Estimate Std. Error t value Pr(>|t|)
(Intercept) -8.9022861  2.3485798  -3.790 0.000181 ***
vh           0.0019632  0.0004067   4.827 2.19e-06 ***
wind        -0.0359052  0.0102836  -3.492 0.000550 ***
---
Approximate significance of smooth terms:
edf Ref.df      F  p-value
s(temp) 5.665  6.809  6.446 6.23e-07 ***
s(dpg)  4.250  5.320 17.921  < 2e-16 ***
s(vis)  4.500  5.556 11.385 8.82e-11 ***
s(doy)  4.305  5.393 30.866  < 2e-16 ***
R-sq.(adj) =   0.81   Deviance explained = 82.2%
GCV = 0.11414  Scale est. = 0.10663   n = 330
```

Ozone Data

- Now we can plot basic diagnostic plots as in the linear models;

```
plot(predict(amred2), residuals(amred2),xlab="Predicted",ylab="Residual
abline(h=0,lwd=2)
qqnorm(residuals(amred2), main="",pch=16)
qqline(residuals(amred2),lwd=2)
```



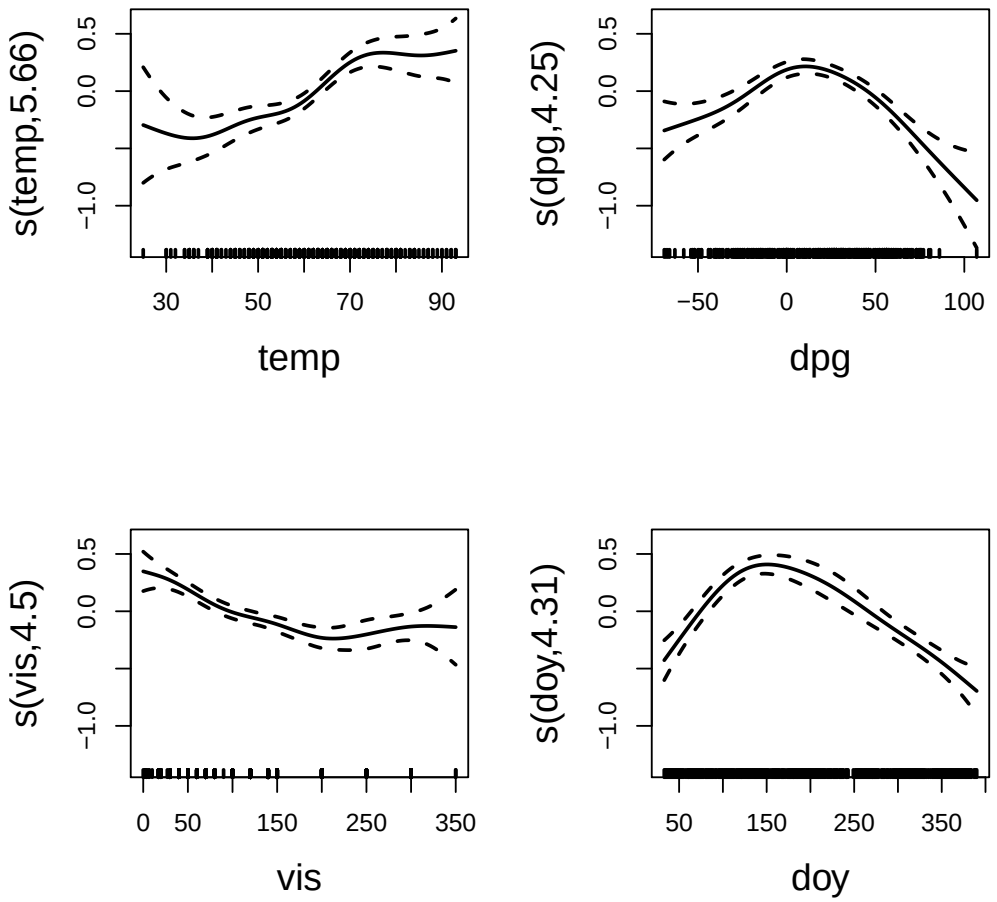


- Looks also ok...but with room to improve; for example, interactions?

Ozone Data

- Now we can visualize the fitting, first fitted four nonlinear functions

```
par(mfrow=c(2,2))
for(k in 1:4)
plot(amred2,select=k,lwd=2,cex.lab=1.5)
```



Ozone Data

- It is possible that there are some interactions between predictors;
- We need to fit a functional interaction models to see this;
- Which pair to look at? Pick it with intuition or expert's opinion;
- Here, we pick "temp" and "wind";
- In *gam()* function, use argument "s(temp,wind)";

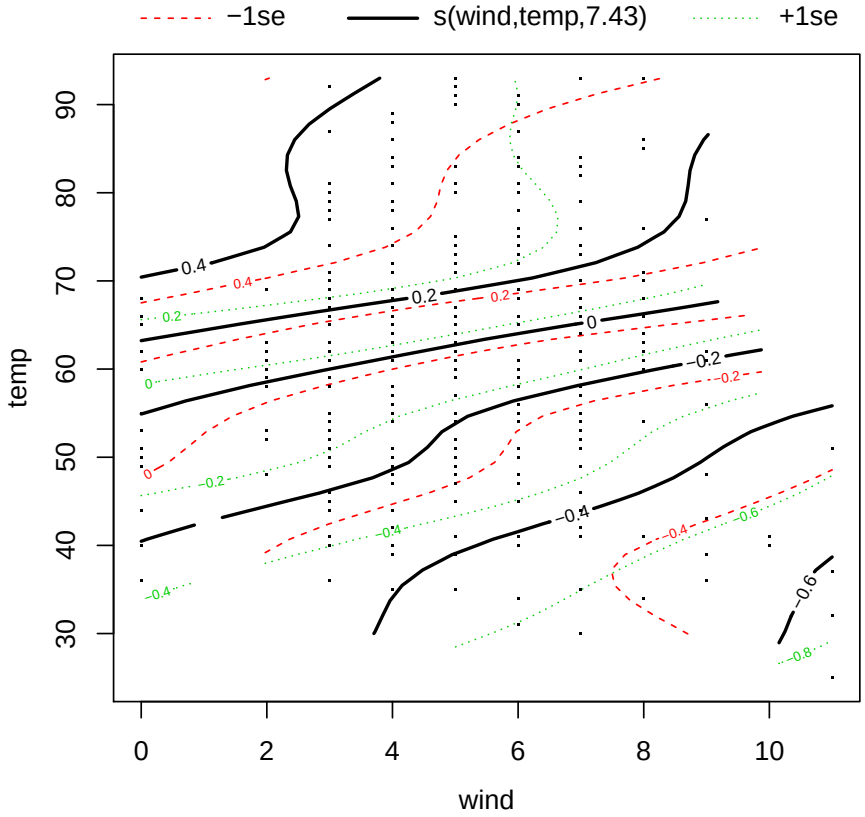
```
amint <- gam(log(O3)~vh+s(wind,temp)+s(dpg)+s(vis)+s(doy))
plot(amint,select=1)

> AIC(amred,amred2,amint)
              df      AIC
amred  23.61717 194.1516
amred2 22.72033 220.7995
amint  21.67309 221.6989
```

- Which model to use?

Ozone Data

- Now we can visualize the fitting of the interaction function between “temp" and “wind"

```
plot(amint,select=1)
```



- the contours appear almost parallel;
- we suspect that there is no interaction effect between “temperature" and “wind". (make sense?)

Ozone Data

- Like in the *lm()* function, we can use *predict()* to make predictions using the fitted additive model;
- option *predict(,se=T)* asks the function to return standard error of predicted values;
- We have fitted partial linear additive model for the ozone data;

```
amred2 <- gam(log(O3)~vh+wind+s(temp)+s(dpg)
+s(vis)+s(doy),data=ozone)
d1<-data.frame(vh=5750,wind=4.85,temp=62,dpg=18,vis=125,doy=210)
d2<-data.frame(vh=6000,wind=11,temp=93,dpg=108,vis=350,doy=390)
> predict(amred2,newdata=d1,se=T)
$fit
2.607106
$se.fit
0.08287519
> predict(amred2,newdata=d2,se=T)
$fit
1.031878
$se.fit
0.3622566
```

- Question: why do you think the standard error is so different?

Linear model V.S. Nonlinear model

- Linear model:
  - it will always exclude many plausible associations between y and $\boldsymbol{X}$;
  - it is more efficient than nonlinear model when the linear assumption is true; (much smaller MSE for estimated prameters)
  - parameters may also have better intuitive interpretations;
  - but if the model assumption is wrong, the estimates will be useless;
  - it's like a gamble, you take a bigger risk but may have much higher return;
- Nonlinear model:
  - it much more flexible in modelling associations between y and $\boldsymbol{X}$;
  - in a sense, it is less likely to be wrong than the linear model;
  - however, it is less efficient than the linear model when the linear assumption is true;
  - so, it is a safer bet but with relatively small returns (larger MSE for estimated functions;)
- Which one to use? Depends on how confident you are in making your assumptions.

Additive Logistic Regression

- The linear logistic model assumes that

$$\log \frac{P(Y=1|x)}{P(Y=0|x)} = \beta_0 + X_1\beta_1 + \cdots + X_p\beta_p$$

- The generalized additive logistic model assumes that

$$\log \frac{P(Y=1|x)}{P(Y=0|x)} = \beta_0 + f_1(X_1) + \cdots + f_p(X_p)$$

- All functions $f_1(X_1)$ can be approximated by some basis functions;
- To run a additive logistic regression model, we can use the
  - The *"gam()"* function in the "mgcv" package; (I prefer this one.)
  - The *"gam()"* function in the "gam" package; (The one used in the book.)
  - Everything else is the same with the additive model, but set *family=binomial*.

Spam Data

- We apply a generalized additive model to the *spam* data;
- The data consists of information from 4601 email messages, in a study to screen email for “spam" (i.e., junk email).
- The response variable is binary, with values *"nonspam"* (coded "0") or *"spam"* (coded "1");
- There are 57 predictors
  - 48 quantitative predictors: the percentage of words in the email that match a given word. Examples include *"business", "address", "internet", "free"*, and *george*.
  - 6 quantitative predictors: the percentage of characters in the email that match a given character. The characters are "ch;", "ch(", "ch[", "ch!", "ch$", and "ch#".
  - The average length of uninterrupted sequences of capital letters: "*CAPAVE*".
  - The length of the longest uninterrupted sequence of capital letters: "*CAPMAX*".
  - The sum of the length of uninterrupted sequences of capital letters: "*CAPTOT* ".

Spam Data

- Train 80%, Test 20%.
- A generalized additive model was fit, using a cubic smoothing spline;
- Most of the *"spam"* predictors have a very long-tailed distribution. Before fitting the GAM model, we log-transformed each variable (actually $\log(x + 0.1)$)

Spam Data

```
rm(list=ls())
library(kernlab)
library(mgcv)
library(caret)
set.seed(1)
data(spam)
data=spam[,-c(55,56,57)]
data[,-55]=log(data[,-55]+0.1)
data$type=factor(data$type)

n=dim(data)[1]# n=4601
n_train=round(n*0.8)
n_test=n-n_train
index_train=sample(1:n,n_train)
index_test=setdiff(1:n,index_train)
data_train=data[index_train,]
data_test=data[index_test,]

vname=colnames(data_train)[1:54]
fm=paste('s(', vname, ',bs="cr")', sep = "", collapse = ' + ')
fm=as.formula(paste('type ~', fm))
```

Spam Data

```
> fm
type ~ s(make, bs = "cr") + s(address, bs = "cr") + s(all, bs = "cr") +
    s(num3d, bs = "cr") + s(our, bs = "cr") + s(over, bs = "cr") +
    s(remove, bs = "cr") + s(internet, bs = "cr") + s(order,
    bs = "cr") + s(mail, bs = "cr") + s(receive, bs = "cr") +
    s(will, bs = "cr") + s(people, bs = "cr") + s(report, bs = "cr") +
    s(addresses, bs = "cr") + s(free, bs = "cr") + s(business,
    bs = "cr") + s(email, bs = "cr") + s(you, bs = "cr") + s(credit,
    bs = "cr") + s(your, bs = "cr") + s(font, bs = "cr") + s(num000,
    bs = "cr") + s(money, bs = "cr") + s(hp, bs = "cr") + s(hpl,
    bs = "cr") + s(george, bs = "cr") + s(num650, bs = "cr") +
    s(lab, bs = "cr") + s(labs, bs = "cr") + s(telnet, bs = "cr") +
    s(num857, bs = "cr") + s(data, bs = "cr") + s(num415, bs = "cr") +
    s(num85, bs = "cr") + s(technology, bs = "cr") + s(num1999,
    bs = "cr") + s(parts, bs = "cr") + s(pm, bs = "cr") + s(direct,
    bs = "cr") + s(cs, bs = "cr") + s(meeting, bs = "cr") + s(original,
    bs = "cr") + s(project, bs = "cr") + s(re, bs = "cr") + s(edu,
    bs = "cr") + s(table, bs = "cr") + s(conference, bs = "cr") +
    s(charSemicolon, bs = "cr") + s(charRoundbracket, bs = "cr") +
    s(charSquarebracket, bs = "cr") + s(charExclamation, bs = "cr") +
    s(charDollar, bs = "cr") + s(charHash, bs = "cr")
```

Spam Data

```
lmod=gam(fm,family=binomial,data=data_train) # extremely slow!

yhat=as.numeric(predict(lmod,newdata=data_test)>0)
ytrue=as.numeric(data_test$type)-1

> mean(ytrue==yhat)
[1] 0.9369565

> confusionMatrix(data=factor(yhat), reference=factor(ytrue))

          Reference
Prediction   0   1
         0 545  36
         1  22 317
```

Spam Data

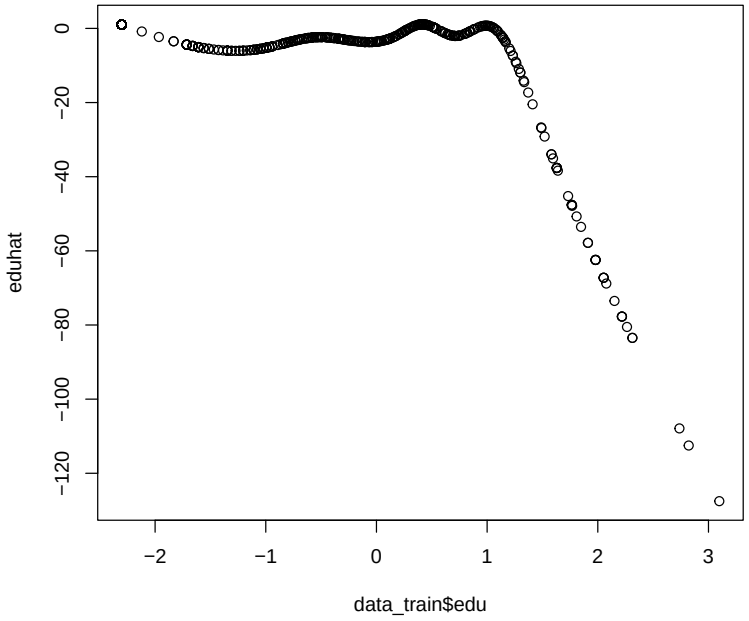
```
> summary(lmod)

Approximate significance of smooth terms:
                       edf Ref.df Chi.sq  p-value
s(make)              1.000  1.000  2.855 0.091116 .
s(address)           2.020  2.433  4.287 0.136373
s(all)               1.000  1.000  4.371 0.036561 *
s(num3d)             2.322  2.693  6.954 0.041239 *
s(our)               2.233  2.726 44.610  < 2e-16 ***
s(over)              5.363  6.255 26.207 0.000264 ***
s(remove)            1.314  1.548 49.405  < 2e-16 ***
s(internet)          7.239  7.968 13.250 0.106696
s(order)             5.176  5.984 15.363 0.018576 *
s(mail)              1.877  2.282  5.933 0.071254 .
s(receive)           5.442  6.330 14.615 0.028604 *
s(will)              1.978  2.415  8.443 0.020936 *
s(people)            1.358  1.612  2.949 0.217430
s(report)            2.452  2.985 13.438 0.003877 **
s(addresses)         1.917  2.283  4.078 0.144075
s(free)              1.000  1.000 44.549  < 2e-16 ***
s(business)          6.475  7.228 25.240 0.000879 ***
s(email)             1.000  1.000  0.380 0.537623
s(you)               1.000  1.000  1.009 0.315120
s(credit)            6.231  6.910 11.596 0.144927
s(your)              8.884  8.990 29.421 0.000549 ***
s(font)              1.895  2.267 10.760 0.010171 *
```

Spam Data

```
s(num000)            8.581  8.909 16.731 0.055605 .
s(money)             6.611  7.398 22.766 0.003377 **
s(hp)                1.467  1.757 45.249  < 2e-16 ***
s(hpl)               5.773  6.325 24.040 0.000752 ***
s(george)            2.965  3.390 38.578  < 2e-16 ***
s(num650)            4.798  5.635 29.597 3.44e-05 ***
s(lab)               1.000  1.001  1.704 0.191850
s(labs)              1.000  1.000  2.174 0.140348
s(telnet)            1.000  1.000  3.362 0.066737 .
s(num857)            1.000  1.000  0.339 0.560500
s(data)              1.000  1.000  0.484 0.486552
s(num415)            1.000  1.000  0.354 0.551838
s(num85)             1.000  1.000  0.848 0.357209
s(technology)        4.550  5.303  7.576 0.239679
s(num1999)           2.282  2.727 13.583 0.002651 **
s(parts)             2.244  2.767  7.401 0.050103 .
s(pm)                2.007  2.448  2.605 0.280172
s(direct)            1.058  1.111  0.881 0.347453
s(cs)                1.000  1.000  0.163 0.686555
s(meeting)           2.976  3.166  5.328 0.223767
s(original)          1.697  2.011 12.292 0.001922 **
s(project)           1.000  1.000  8.292 0.003984 **
s(re)                4.438  5.267 27.119 9.16e-05 ***
s(edu)               7.744  7.951 51.687  < 2e-16 ***
s(table)             1.000  1.000  0.758 0.384093
s(conference)        2.299  2.738  2.941 0.279127
s(charSemicolon)     3.382  4.172 10.247 0.041334 *
s(charRoundbracket)  6.214  7.059 11.677 0.116508
s(charSquarebracket) 1.000  1.000  0.006 0.939343
s(charExclamation)   4.342  5.164 86.665  < 2e-16 ***
s(charDollar)        4.395  5.292 71.970  < 2e-16 ***
s(charHash)          6.256  7.218 14.848 0.042625 *
```

Spam Data

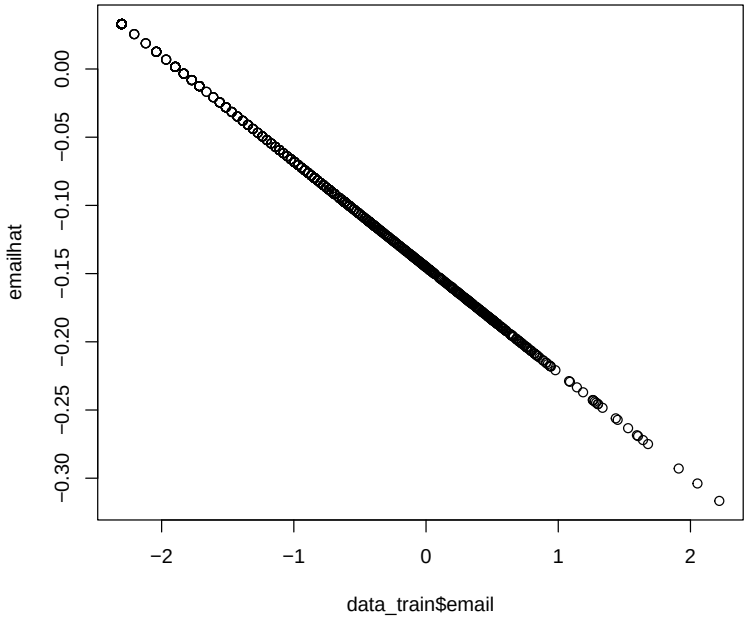
```
which(vname=="edu") # 46th column, pvalue<0.05
eduhat=predict(lmod,type="terms")[,46]
plot(data_train$edu, eduhat)
```



- nonlinear term, reject linear hypothesis

Spam Data

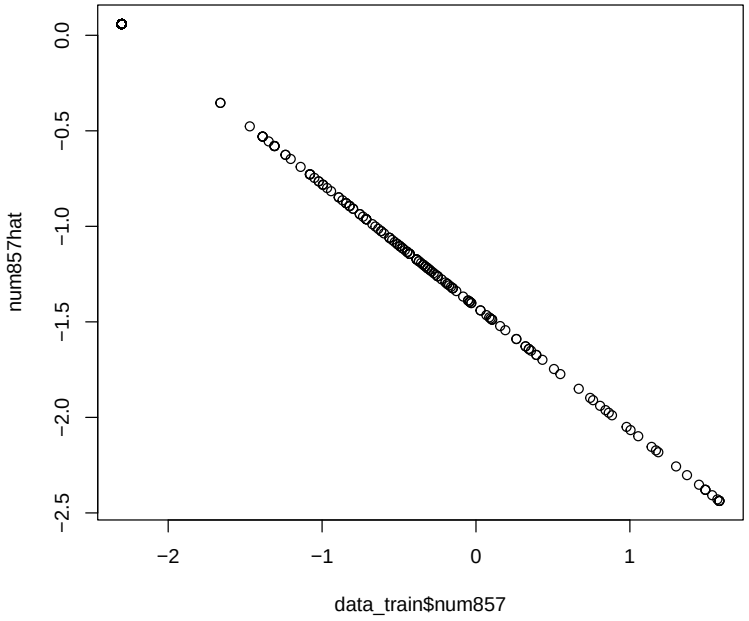
```
which(vname=="email") # 18th column, pvalue>0.05
emailhat=predict(lmod,type="terms")[,18]
plot(data_train$email, emailhat)
```



- A straight line, no nonlinear term, fails to reject linear hypothesis

Spam Data

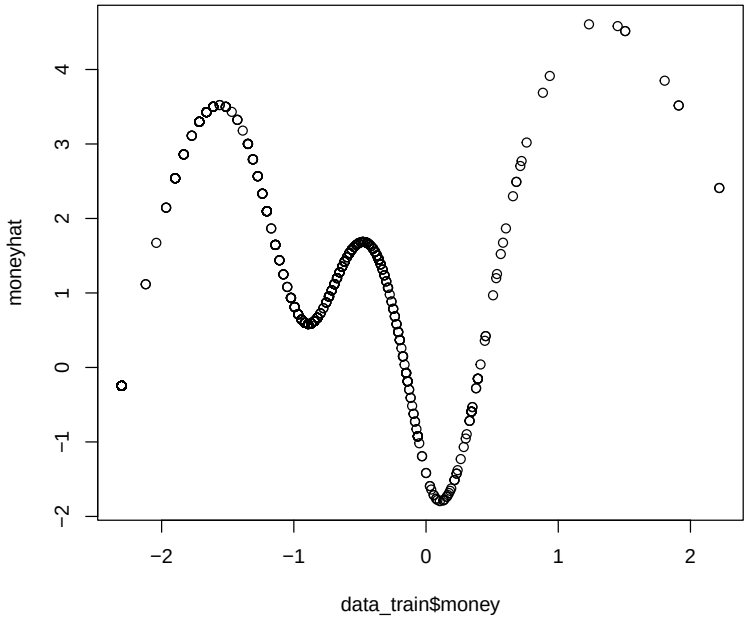
```
which(vname=="num857") # 32nd column, pvalue>0.05
num857hat=predict(lmod,type="terms")[,32]
plot(data_train$num857, num857hat)
```



- A straight line, no nonlinear term, fails to reject linear hypothesis

Spam Data

```
which(vname=="money") # 24th column, pvalue<0.5
moneyhat=predict(lmod,type="terms")[,24]
plot(data_train$money, moneyhat)
```



- nonlinear term, reject linear hypothesis hypothesis

Spam Data

- Fit a simple logistic model without nonlinear terms

```
rm(list=ls())
library(kernlab)
library(mgcv)
library(caret)
set.seed(1)
data(spam)
data=spam[,-c(55,56,57)]
data[,-55]=log(data[,-55]+0.1)
data$type=factor(data$type)

n=dim(data)[1]# n=4601
n_train=round(n*0.8)
n_test=n-n_train
index_train=sample(1:n,n_train)
index_test=setdiff(1:n,index_train)
data_train=data[index_train,]
data_test=data[index_test,]

lmod=glm(type~.,data=data_train,family=binomial)
yhat=as.numeric(predict(lmod,newdata=data_test)>0)
ytrue=as.numeric(data_test$type)-1

> mean(ytrue==yhat)
[1] 0.9358696 # Additive Logistic Regression with 0.9369565
> confusionMatrix(data=factor(yhat), reference=factor(ytrue))
          Reference
Prediction   0   1
         0 539  31
         1  28 322
```

- What can we do to further improve the performance?